



Università
di Catania

UNIVERSITY OF CATANIA

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

A MASTER OF SCIENCE DEGREE IN COMPUTER SCIENCE.

Alfio Spoto

Blockchain and Cryptocurrencies

LECTURE NOTES

Professor: Dario Catalano
Professor: Mario Di Raimondo

Academic Year 2024 - 2025

Contents

I	Lezioni Teoriche	6
1	Lezione 1	7
1.1	Cryptographic Hash Functions	7
1.1.1	Le Tre Colonne della Sicurezza	7
1.1.2	Commitment Schemes: Promesse Crittografiche	8
1.1.3	SHA-256: Il Motore di Bitcoin	9
2	Lezione 2	10
2.1	Hash Pointer e Strutture Dati	10
2.2	Registro a Prova di Manomissione e Blockchain	11
2.2.1	Meccanismo di Sicurezza	11
2.2.2	Applicazioni nella Blockchain	12
2.3	Merkle Tree: Struttura Dati per Verifica Rapida	12
2.3.1	Verifica dell'Integrità e Prova di Appartenenza	13
2.3.2	Merkle Tree Ordinato e Prova di Non-Appartenenza	13
3	Lezione 3	14
3.1	Digital Signatures	14
3.2	API per firme digitali	14
3.3	Requisiti di sicurezza	14
3.4	Elliptic Curve Digital Signature Algorithm (ECDSA): Principi e Funzionamento	16
3.4.1	Algoritmo di Firma	16
3.4.2	Algoritmo di Verifica	17
3.5	Aspetti Critici di Sicurezza	17
3.6	Public Keys as Identities	18
3.6.1	Controllo dell'Identità	18
3.7	Creazione di Nuove Identità	18
3.8	Gestione Decentralizzata delle Identità	19
3.8.1	Autonomia di Creazione	19
3.8.2	Implementazione in Bitcoin	19
3.9	Pseudonimity	19
3.10	Cryptocurrencies	20
3.10.1	Proprietà fondamentali di una moneta digitale	20
3.10.1.1	Coins immutabili (Immutable Coins)	20
3.10.1.2	Mancata dipendenza da autorità centralizzate	20
3.10.2	Principali sfide nell'implementazione	20

3.10.2.1	Attacchi di double spending	20
3.10.2.2	Decentralizzazione e coordinamento	21
3.11	Centralization vs Decentralization	21
3.11.1	Aspetti decentralizzati in Bitcoin	21
3.11.2	Centralizzazione pratica	22
3.12	Consenso Distribuito	22
3.12.1	Il problema fondamentale di Bitcoin	22
3.12.2	Definizione formale di consenso	22
3.12.3	Il meccanismo di consenso in Bitcoin	22
3.12.3.1	Protocollo operativo	23
3.12.4	Obiettivi e ostacoli nel raggiungere il consenso	23
3.12.4.1	Resultati di impossibilità teorica	23
3.12.5	Bitcoin: un compromesso tra teoria e pratica	24
3.13	Consensus without identity: the block chain	24
3.13.1	Il meccanismo di consenso implicito	24
3.13.2	Algoritmo di consenso semplificato	25
3.13.3	Attacchi da parte di nodi maliziosi	25
3.13.3.1	Attacco di Stealing Bitcoin	25
3.13.3.2	Attacco di negazione del servizio (DoS)	26
3.13.3.3	Double Spending	26
3.13.3.4	Prevenzione tramite conferme multiple	26
3.13.4	Sicurezza e validità delle transazioni in Bitcoin	27
4	Lezione 4	28
4.1	Incentivi e Proof-of-Work	28
4.1.1	Problemi con l'ipotesi di onestà	28
4.1.2	Incentivi per i nodi	28
4.1.2.1	Incentivo 1: Block Reward	28
4.1.2.2	Incentivo 2: Transaction Fee	29
4.1.3	Problemi da risolvere	29
4.1.4	Proof-of-Work (PoW): Soluzione unica	29
4.1.4.1	Funzione del PoW	29
4.1.4.2	Il puzzle di hash	30
4.1.4.3	Proprietà del PoW	30
4.1.5	Assunzione di sicurezza	30
4.1.6	Economia del Mining	31
4.2	Mettendo tutto insieme	31
4.2.1	Attacchi al 51%	32
4.2.2	Transazioni Bitcoin	32
4.2.2.1	Registro Basato su Conti (Account-Based)	33
4.2.2.2	Registro Basato su Transazioni (UTXO)	34
5	Lezione 5	36
5.1	Consenso di Nakamoto	36
5.1.1	Fondamenti del Consenso Distribuito	36
5.1.2	Meccanismo del Consenso di Nakamoto	36
5.1.3	Proprietà Formali del Consenso	36
5.1.4	Limitazioni del Consenso di Nakamoto	37

5.1.5	Intuizione della Consistenza	37
5.1.6	Binomial Random Walk	38
5.1.7	Considerazioni sulla Difficoltà di Mining	39
5.1.8	Scelta del Parametro di Difficoltà del Mining	39
5.1.9	Condizioni di Sicurezza	39
5.1.10	Proprietà della Blockchain di Nakamoto	40
5.1.11	Block-Withholding Attacks (Selfish Mining)	40
5.1.11.1	Strategia di Base del Selfish Mining	41
5.1.11.2	Dinamiche Competitive e Race Condition	41
5.1.11.3	Condizioni di Redditività dell'Attacco	41
5.1.11.4	Implicazioni per il Modello di Sicurezza	42
6	Lezione 6	43
6.1	Fondamenti dell'Anonimato	43
6.1.1	Definizione di Anonimato	43
6.1.2	Pseudonimia e Unlinkability	43
6.1.3	Quantificazione dell'Anonimato	44
6.1.4	Motivazioni per Criptovalute Anonime	44
6.1.5	Il Dilemma Etico-Tecnologico	44
6.2	Storia del Contante Elettronico Anonimo	45
6.2.1	Le Blind Signatures di David Chaum	45
6.3	Storia del Contante Elettronico Anonimo	45
6.3.1	Il Conflitto tra Anonimato e Decentralizzazione	46
6.4	Tecniche di De-anonimizzazione in Bitcoin	47
6.4.1	Clustering degli Indirizzi	47
6.4.2	Indirizzi di Resto e "Idioms of Use"	48
6.4.3	Attribuzione di Identità ai Cluster	49
6.4.4	Identificazione degli Individui	49
6.4.5	De-anonimizzazione a Livello di Rete	50
6.5	Soluzioni per Migliorare l'Anonimato: Tor	50
6.5.1	Utilizzo di Tor per le Transazioni Bitcoin	50
6.5.2	Funzionamento di Tor	51
7	Lezione 7	52
7.1	Mixing e Anonimato nelle Criptovalute	52
7.1.1	Online Wallets come Meccanismi di Mixing Implicito	52
7.1.2	Servizi di Mixing Dedicati	53
7.2	Principi di Progettazione per Servizi di Mixing Efficienti	53
7.2.1	Utilizzo di una Serie di Mix	53
7.2.2	Uniformità delle Transazioni	54
7.2.3	Automazione del Lato Client	55
7.2.4	Commissioni con Meccanismo All-or-Nothing	55
7.3	Stato Attuale dei Servizi di Mixing	55
7.4	Mixing Decentralizzato	56
7.4.1	Motivazioni per il Mixing Decentralizzato	56
7.4.2	CoinJoin: Principi e Funzionamento	57
7.4.2.1	Algoritmo di CoinJoin	57
7.4.3	Problematiche Residue di CoinJoin	59

7.4.4	Soluzioni per l'Anonimato tra Peer	59
7.4.4.1	Soluzione Elementare	59
7.4.4.2	Soluzione Avanzata	59
7.4.5	Mitigazione del Denial of Service	60
8	Lezione 8	61
8.1	Prove Interattive e Zero Knowledge Proofs	61
8.1.1	Il Concetto di Dimostrazione	61
8.1.2	Requisiti delle Prove Interattive	62
8.2	La Nozione di Conoscenza e Zero Knowledge	62
8.2.1	Acquisizione di Conoscenza	62
8.2.2	Sistemi di Prove Interattive	62
8.3	Zero Knowledge Proofs	63
8.3.1	Perfect Zero Knowledge	63
8.4	Zero Knowledge Proofs: Esempi e Applicazioni	63
8.4.1	Isomorfismo tra Grafi	63
8.4.2	Il Protocollo Diffie-Hellman	65
8.4.2.1	Meccanismo Fondamentale del Protocollo	65
8.4.2.2	Zero Knowledge Proofs nel Contesto Diffie-Hellman	66
8.5	Teorema Fondamentale sulle Zero Knowledge Proofs	67
8.6	Prove di Conoscenza	67
8.6.1	Formalizzazione della Conoscenza	67
9	Lezione 9	69
9.1	ZeroCoin e Zerocash: Soluzioni per l'Anonimato nelle Criptovalute	69
9.2	ZeroCoin: Fondamenti Teorici e Innovazione Concettuale	69
9.2.1	Modello Concettuale: La Bacheca Pubblica	69
9.2.2	Il Processo di Coniazione (Minting): Creazione dell'Anonimato	70
9.2.3	Il Processo di Riscatto (Redeeming): La Magia delle Prove zero knowledge	70
9.3	La Sfida dell'Efficienza: Accumulatori Crittografici	71
9.3.1	Accumulatori Crittografici: Compressione Intelligente dei Dati	72
9.3.2	Implementazione RSA degli Accumulatori: Il Cuore Tecnico	72
9.3.2.1	Generazione dei Parametri (Acc.Setup)	73
9.3.2.2	Accumulazione (Accumulate)	73
9.3.2.3	Generazione del Testimone (GenWitness)	73
9.3.2.4	Verifica dell'Appartenenza (Acc.Verify)	74
9.3.3	Integrazione in ZeroCoin: Ottimizzazione delle Prove di Riscatto	74
9.3.4	Limitazioni e Considerazioni Pratiche	74
9.4	Le Sfide dell'Integrazione: Limiti e Problematiche di ZeroCoin	75
9.4.1	Incompatibilità con il Modello di Script di Bitcoin	75
9.4.2	Divergenza dai Paradigmi di Sicurezza di Bitcoin	75
9.4.3	Il Problema del Setup Fidato	75
9.5	Zerocash: Evoluzione e Superamento dei Limiti di ZeroCoin	76
9.5.1	zk-SNARKs: Fondamento Crittografico	76
9.5.2	Transazioni Completamente Anonime	77
9.5.3	La Problematica del Setup Fidato	77
9.6	Il Continuum dell'Anonimato nelle Criptovalute	78

10 Lezione 10	79
10.1 Alternative Mining Puzzles	79
10.2 Requisiti Essenziali dei Mining Puzzles	79
10.2.1 Verifica Economica	79
10.2.2 Difficoltà Adattiva	80
10.2.3 Proporzionalità alla Potenza di Calcolo	80
10.2.4 Assenza di Vantaggio Sproporzionato per Grandi Operatori	80
10.2.5 Compenso Proporzionale anche per Piccoli Operatori	80
10.3 L'Inefficacia dei Puzzle Sequenziali	81
10.3.1 L'Importanza della Proprietà "Progress-Free"	81
10.4 Puzzle Resistenti agli ASIC	82
10.4.1 Motivazioni per la Resistenza agli ASIC	82
10.4.1.1 Democratizzazione del Mining	82
10.4.1.2 Prevenzione della Dominanza Manifatturiera	82
10.4.2 Puzzle Memory-Hard	83
10.4.2.1 Script: Fondamenti e Implementazione	83
10.4.2.2 Compromesso Tempo-Memoria in Script	85
10.4.3 Critica e Limitazioni di Script	85
10.4.4 Approcci Alternativi alla Resistenza agli ASIC	86
10.4.4.1 Funzioni di Hash Complesse	86
10.4.4.2 Target Mobile	86
10.4.5 Contrargomentazione: SHA-256 è Adeguato	86
10.5 Proof-of-Useful-Work	87
10.5.1 Il Concetto di Computazione Utile	87
10.5.1.1 Candidati per Computazioni Utili	87
10.5.1.2 Sfide Implementative	87
10.5.2 Primecoin: Un Caso di Studio	87
10.5.2.1 Catene di Cunningham	87
10.5.2.2 Algoritmo del Puzzle	88
10.5.2.3 Risultati e Utilità	88

Part I

Lezioni Teoriche

Chapter 1

Lezione 1

1.1 Cryptographic Hash Functions

Le funzioni hash sono il cuore pulsante delle tecnologie blockchain. Immaginatele come macchine matematiche che prendono un input di qualsiasi dimensione – un documento, una transazione, o persino un'intera blockchain – e lo trasformano in un'impronta digitale unica di 256 bit (nel caso di SHA-256). Questa trasformazione non è casuale: segue regole precise che garantiscono sicurezza ed efficienza.

1.1.1 Le Tre Colonne della Sicurezza

Una funzione hash crittografica deve soddisfare tre proprietà fondamentali:

1. Resistenza alle Collisioni (Collision-Free): È praticamente impossibile trovare due input diversi che producano lo stesso output. Formalmente, dati due messaggi $x \neq y$, non è computazionalmente possibile eseguire un algoritmo per cui $H(x) = H(y)$.

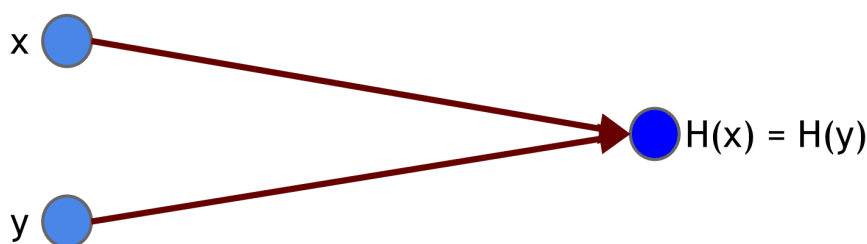


Figure 1.1: Esempio di collisione.

2. Proprietà di Hiding: Dato un'output $H(r|x)$, dove r è un valore casuale scelto da una distribuzione ad alta entropia, non è possibile risalire all'input x . Questo meccanismo protegge informazioni sensibili. Per esempio, in Bitcoin, quando si genera un indirizzo wallet, la chiave pubblica viene hashata nascondendo la vera identità.

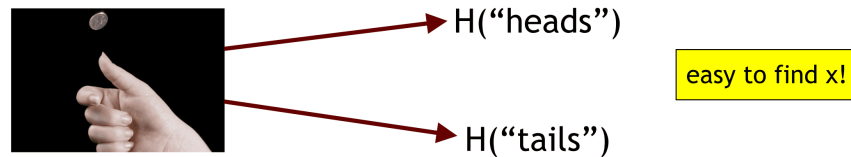


Figure 1.2: Dato $H(x)$ è facile trovare x .

3. Puzzle-Friendly: Una funzione hash H è detta puzzle-friendly se, dato un qualsiasi valore di output y e un input fisso x , non esiste un metodo migliore della forza bruta per trovare un parametro k tale che $H(x\|k) = y$, dove $\|$ rappresenta la concatenazione. In altre parole, anche conoscendo x , y e la funzione H , l'unico modo per trovare k è provare un gran numero di possibilità casuali, senza scorciatoie.

Questa proprietà è fondamentale per il mining nei sistemi proof-of-work. In questo contesto, i miner ricevono un valore fisso x (che rappresenta l'header del blocco) e cercano un valore k (chiamato nonce) tale che $H(x\|k)$ soddisfi una certa condizione, ad esempio che l'hash sia minore di un determinato target. Poiché il risultato non è prevedibile, i miner devono provare molteplici valori di k in modo casuale, facendo brute force fino a trovare una soluzione valida.

1.1.2 Commitment Schemes: Promesse Crittografiche

I commitment scheme sono protocolli a due fasi che permettono di "impegnarsi" a un valore senza rivelarlo immediatamente. Un esempio pratico è un'asta dove i partecipanti inviano l'offerta cifrata prima della rivelazione.

```

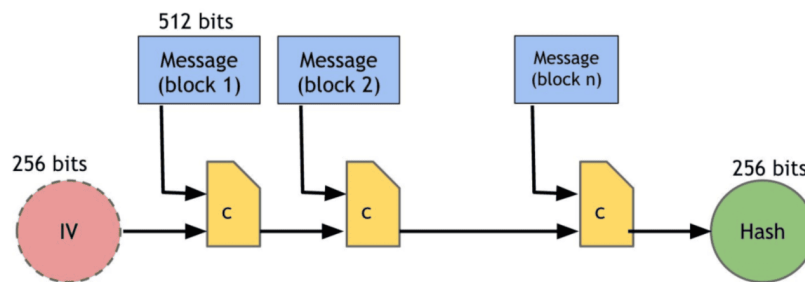
1 # Fase 1: Commit
2 commitment := commit(message, randomness)
3
4 # Fase 2: Verify
5 1\0 := verify(commitment, message, randomness):

```

Due proprietà sono essenziali:

- *Hiding*: Il commitment non rivela informazioni sul messaggio.
- *Binding*: Non è possibile cambiare il messaggio dopo averlo impegnato.

1.1.3 SHA-256: Il Motore di Bitcoin



Lo SHA-256 non è solo una funzione hash: è un'opera d'ingegneria crittografica. Ecco come lavora:

1. **Padding:** Il messaggio viene esteso a una lunghezza multipla di 512 bit, aggiungendo un '1', seguita da zeri e infine la lunghezza originale a 64 bit.
2. **Divisione in blocchi:** Il messaggio viene suddiviso in blocchi da 512 bit.
3. **Compressione iterativa:** Ogni blocco viene processato da una funzione di compressione che modifica lo stato interno (256 bit).
4. **Output finale:** Lo stato dopo l'ultimo blocco è l'hash risultante.

Theorem 1.1.1 (Sicurezza di SHA-256). *Se la funzione di compressione c è resistente alle collisioni, allora l'intera struttura SHA-256 eredita questa proprietà. Formalmente:*

$$\exists \text{ collisione in SHA-256} \implies \exists \text{ collisione in } c$$

Chapter 2

Lezione 2

2.1 Hash Pointer e Strutture Dati

Gli **hash pointer** rappresentano una componente fondamentale nelle strutture dati sicure e distribuite. Non si tratta semplicemente di un puntatore tradizionale, ma di un riferimento che integra due elementi essenziali:

1. Il puntatore vero e proprio, che indica la posizione in memoria o il luogo di archiviazione dell'informazione.
2. Il valore hash calcolato sull'informazione puntata, che funge da impronta digitale per verificarne l'integrità.

Questa duplice funzione consente non solo di recuperare i dati, ma anche di garantire che essi non siano stati alterati. Infatti, ogni variazione, anche minima, dei dati originali comporta una modifica sostanziale del valore hash, grazie alle proprietà crittografiche delle funzioni hash (in particolare la collision-free property). Di conseguenza, l'utilizzo degli hash pointer è cruciale per costruire sistemi che richiedono elevati standard di sicurezza e integrità.

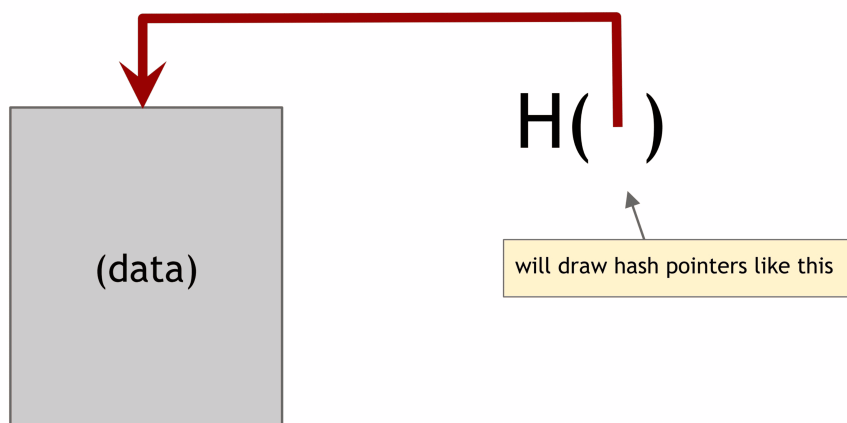


Figure 2.1: Rappresentazione schematica di un Hash Pointer.

Questa tecnologia viene impiegata per realizzare strutture dati che siano non solo efficienti, ma anche resistenti a tentativi di manomissione, fungendo da base per applicazioni

crittografiche avanzate come blockchain e registri a prova di frode.

2.2 Registro a Prova di Manomissione e Blockchain

Un'applicazione pratica degli hash pointer si riscontra nella costruzione di registri a prova di manomissione, che sono alla base della tecnologia blockchain. In un registro di questo tipo, ogni blocco di dati contiene un hash pointer che lo collega al blocco precedente, formando una catena continua.

2.2.1 Meccanismo di Sicurezza

La creazione di un nuovo blocco nella blockchain avviene attraverso un processo strutturato che garantisce l'integrità e la sicurezza dei dati. Quando un blocco viene generato, il suo contenuto, comprensivo delle transazioni e di altri metadati, viene elaborato tramite una funzione hash crittografica. Il risultato di questa operazione è un valore hash univoco, che rappresenta una firma digitale del blocco stesso. Questo valore hash viene poi integrato nel blocco successivo sotto forma di **hash pointer**, stabilendo così un legame indissolubile tra i blocchi consecutivi. In pratica il blocco successivo include l'hash complessivo del blocco precedente come parte del proprio contenuto e questo garantisce che ogni blocco sia legato al precedente in modo sicuro e immutabile.

Qualora un attaccante tentasse di alterare il contenuto di un blocco intermedio, tale modifica comprometterebbe l'integrità della catena. Infatti, l'alterazione del contenuto di un blocco comporterebbe il calcolo di un nuovo hash che non corrisponderebbe al valore memorizzato come hash pointer nel blocco successivo. Questa incongruenza sarebbe immediatamente rilevabile da tutti i nodi della rete.

Per garantire una manomissione consistente della catena, un eventuale attaccante dovrebbe ricalcolare non solo il hash del blocco modificato, ma anche tutti gli hash pointer dei blocchi successivi fino al blocco di testa. Tale procedura richiederebbe una quantità computazionale elevata e diventerebbe sempre più onerosa man mano che si procede lungo la catena. Inoltre, il blocco di testa, spesso conservato in ambienti sicuri o distribuito in modo decentralizzato, costituisce un punto di riferimento inalterabile. Pertanto, qualsiasi tentativo di manipolazione della blockchain verrebbe inevitabilmente rilevato dalla rete.

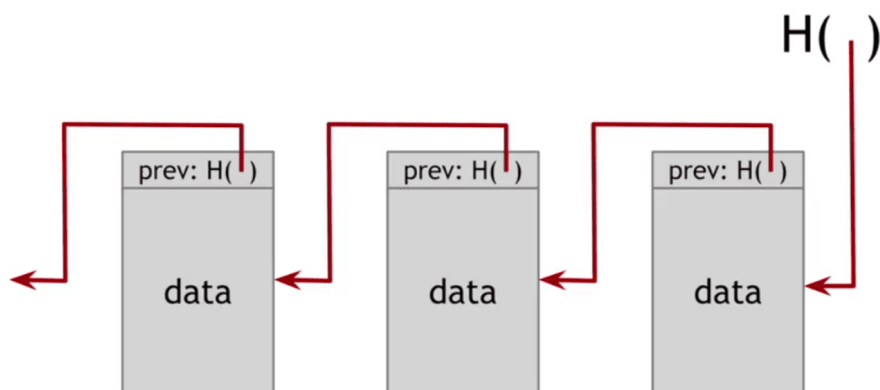


Figure 2.2: Lista concatenata dotata di Hash Pointer, fondamento per la costruzione della blockchain.

2.2.2 Applicazioni nella Blockchain

La blockchain sfrutta il meccanismo degli hash pointer per garantire un registro immutabile e trasparente. Ogni blocco della blockchain contiene:

- I dati o le transazioni.
- L'hash pointer che collega il blocco al blocco precedente.
- Eventuali metadati aggiuntivi utili alla verifica dell'integrità.

Questo schema permette di creare una catena di blocchi in cui l'integrità complessiva è garantita dalla difficoltà di riconciliare eventuali incongruenze senza alterare l'intera sequenza, rendendo la blockchain estremamente robusta contro attacchi retroattivi.

2.3 Merkle Tree: Struttura Dati per Verifica Rapida

La verifica dell'esistenza di una specifica transazione all'interno di un blocco può diventare, con il crescere del numero di transazioni, un'operazione computazionalmente intensiva e poco efficiente. Per far fronte a questo problema, si è introdotto il Merkle Tree (o Albero di Merkle), una struttura dati ideata da Ralph Merkle e ampiamente utilizzata in contesti crittografici, tra cui protocolli come Bitcoin.

Il Merkle Tree rappresenta una forma di albero binario gerarchico che organizza le transazioni contenute in un blocco in modo strutturato ed efficiente. Ogni singola transazione viene prima convertita nel proprio hash crittografico, tipicamente attraverso algoritmi come SHA-256. Successivamente, gli hash vengono combinati a due a due e nuovamente hashati per formare un nuovo livello superiore. Questo processo prosegue iterativamente fino a raggiungere un'unica radice, nota come Merkle Root, che costituisce un riferimento sintetico e immutabile per tutte le transazioni presenti nel blocco.

La Merkle Root viene quindi inclusa nell'intestazione del blocco, fungendo da rappresentazione compatta e sicura delle transazioni associate. Tale approccio non solo riduce significativamente i requisiti computazionali necessari per la verifica delle transazioni, ma garantisce anche l'integrità dei dati tramite meccanismi crittografici robusti.

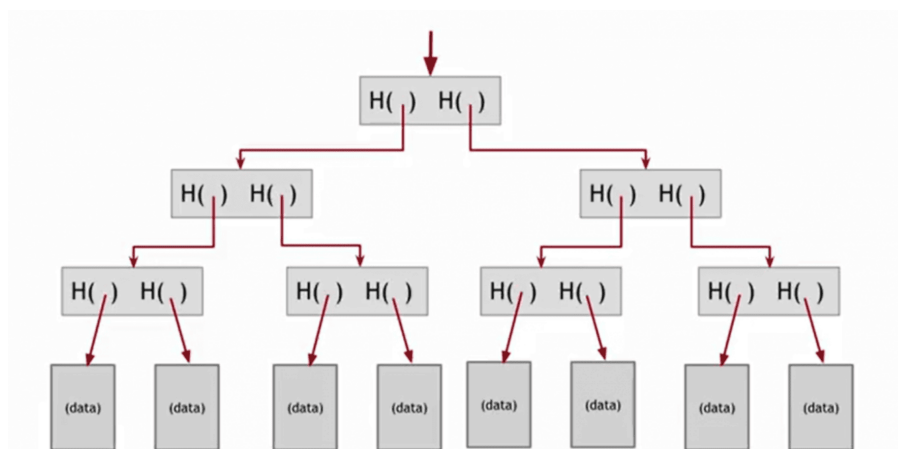


Figure 2.3: Schema rappresentativo di un Merkle Tree.

2.3.1 Verifica dell'Integrità e Prova di Appartenenza

Uno dei principali vantaggi dell'utilizzo di un Merkle Tree è la possibilità di verificare in modo efficiente se un determinato blocco di dati è incluso nell'albero, senza dover ricostruire o analizzare l'intera struttura. Questa verifica si effettua attraverso una *Merkle proof*, ovvero una sequenza di hash che collega la foglia contenente il dato da verificare fino alla *Merkle Root*. A ogni livello, è sufficiente conoscere l'hash del *fratello* del nodo corrente per ricostruire, passo dopo passo, l'intero percorso verso la radice. Il numero di hash necessari per questa verifica cresce in modo logaritmico rispetto al numero totale di foglie, rendendo l'operazione estremamente scalabile anche su grandi volumi di dati.

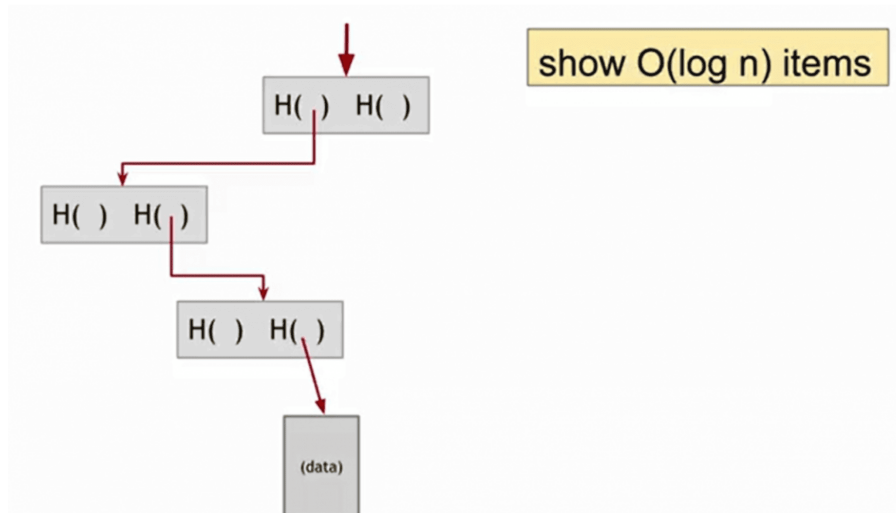


Figure 2.4: La verifica dell'appartenenza in un Merkle Tree richiede un numero di hash pari a $O(\log n)$.

2.3.2 Merkle Tree Ordinato e Prova di Non-Appartenenza

Una variante interessante del Merkle Tree è quella del **Merkle Tree ordinato**. In questa configurazione, le foglie sono disposte secondo un criterio predefinito (ad esempio, in ordine alfabetico o numerico). Tale ordinamento consente di effettuare prove non solo di appartenenza, ma anche di **non-appartenenza**. Per dimostrare che un determinato dato non è presente nell'albero, basta dimostrare che esiste una sequenza consecutiva di foglie tra le quali il dato in questione non potrebbe inserirsi. Questa caratteristica è particolarmente utile in applicazioni dove è necessario garantire in modo verificabile l'assenza di certe informazioni. h

Chapter 3

Lezione 3

3.1 Digital Signatures

Le firme digitali sono meccanismi crittografici progettati per garantire due proprietà fondamentali: *autenticità* e *non ripudio*. La prima garantisce che solo il titolare della chiave segreta possa generare una firma valida per un documento, mentre la seconda assicura che tale firma non possa essere né contraffatta né riutilizzata su altri documenti. Inoltre, qualsiasi utente in possesso della chiave pubblica può verificare l'autenticità della firma, ma non può generarne una nuova.

3.2 API per firme digitali

Il funzionamento di un sistema di firme digitali è formalizzato attraverso un'interfaccia di base costituita da tre operazioni:

- `generateKeys(keysize)`: Genera una coppia di chiavi (sk, pk) , dove sk è la chiave segreta di firma e pk è la chiave pubblica di verifica.
- `sign(sk, message)`: Genera una firma σ applicando l'algoritmo di firma alla coppia $(sk, message)$.
- `verify(pk, message,  $\sigma$ )`: Restituisce `true` se la firma σ è valida per il messaggio $message$ e la chiave pubblica pk .

Tali algoritmi sono spesso *randomizzati*, ovvero dipendono da una sorgente di casualità di qualità per evitare vulnerabilità legate a determinismo.

3.3 Requisiti di sicurezza

Per essere considerate sicure, le firme digitali devono soddisfare due condizioni fondamentali:

1. Validità delle firme

Ogni firma generata correttamente deve essere accettata dalla funzione di verifica:

```
1 verify(pk, message, sign(sk, message)) == true
```

Questa proprietà garantisce che l'algoritmo di firma non dia luogo a falsi negativi durante la verifica.

2. Resistenza alle contraffazioni

Un *avversario* che conosce la chiave pubblica pk e può ottenere firme su messaggi a scelta non deve essere in grado di generare una firma valida per un messaggio non precedentemente firmato. Questo scenario è modellato formalmente come un *attacco di contraffazione* (forgery attack), descritto nel seguente schema (Figura 3.1):

- L'attaccante ha accesso a un oracolo di firma che restituisce $\text{sign}(sk, m)$ per ogni messaggio m richiesto.
- L'obiettivo dell'attaccante è produrre una coppia (m^*, σ^*) tale che $\text{verify}(pk, m^*, \sigma^*) = \text{true}$, dove m^* non è mai stato richiesto all'oracolo.

Un sistema di firma è considerato sicuro se la probabilità di successo di tale attacco è trascurabile.

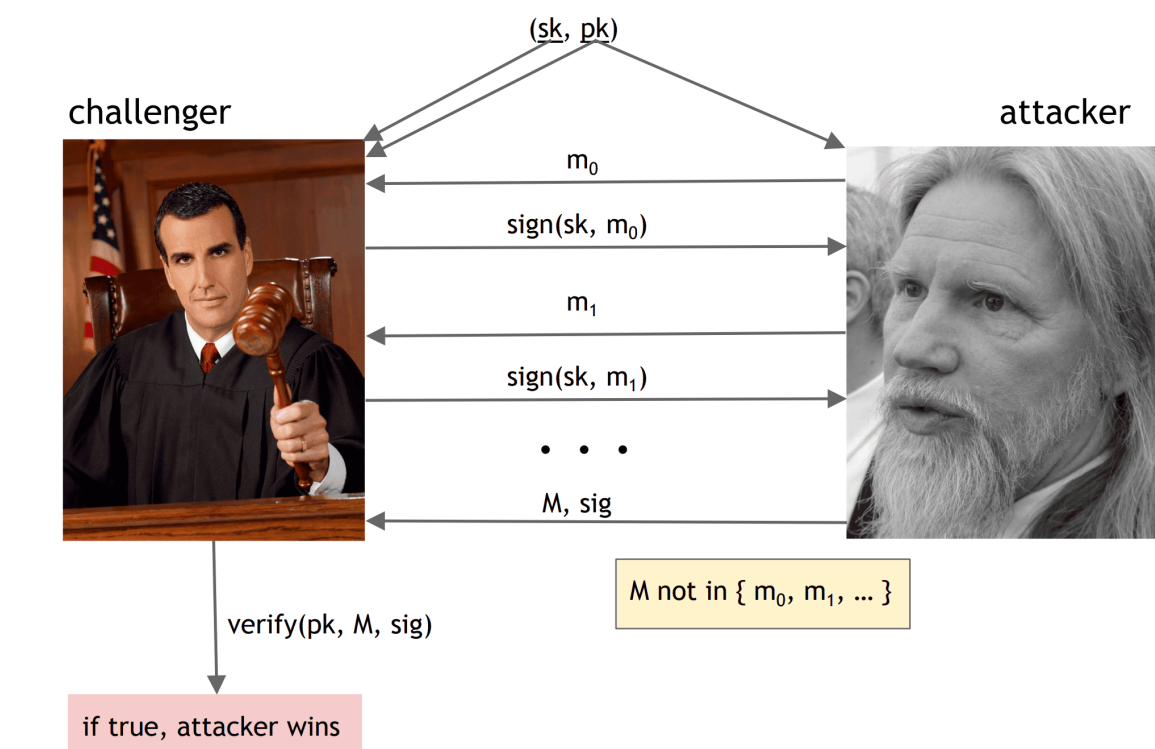


Figure 3.1: Schema challenger-attacker per attacchi di contraffazione.

Considerazioni pratiche

Randomness di qualità

Gli algoritmi randomizzati richiedono una sorgente di casualità cryptographically secure. Un fallimento in questa fase (ad esempio, ripetizione di valori casuali) può portare a:

- Divulgazione della chiave segreta sk durante la generazione delle chiavi.
- Vulnerabilità di firma riproducibile, che facilita l'attacco forzato.

Limitazione della dimensione del messaggio

Per gestire messaggi di lunghezza variabile, si applica la firma alla *hash digest* del messaggio invece che al messaggio stesso:

$$\sigma = \text{sign}(sk, H(\text{message}))$$

Dove H è una funzione hash cryptographically secure (ad esempio, SHA-256). Questa pratica garantisce:

- *Compressione*: riduzione della complessità computazionale.
- *Resistenza alle collisioni*: impossibilità di trovare due messaggi diversi con stessa hash.

3.4 Elliptic Curve Digital Signature Algorithm (ECDSA): Principi e Funzionamento

L'**Elliptic Curve Digital Signature Algorithm (ECDSA)** è un protocollo crittografico standardizzato utilizzato per la generazione e verifica di firme digitali. Si basa sulla crittografia a curve ellittiche e sfrutta la difficoltà computazionale del problema del logaritmo discreto su curve ellittiche (*ECDLP*) per garantire la sicurezza delle operazioni.

L'ECDSA prevede due algoritmi principali: uno per la generazione della firma e l'altro per la sua verifica. Di seguito vengono descritti questi processi in modo sintetico.

3.4.1 Algoritmo di Firma

La procedura per firmare un messaggio m con ECDSA è illustrata nel seguente pseudocodice:

```

1 // Generate a signature (sign(sk, m))
2 function sign(d, m):
3     h = H(m)
4     repeat:
5         k = random_integer_in_range(1, n-1)
6         (x1, y1) = scalar_multiply(G, k)
7         r = x1 mod n
8         if r != 0:
9             break
10    s = modular_inverse(k, n) * (h + d * r) mod n
11    if s == 0:
12        return sign(d, m)
13    return (r, s)

```

Per generare una firma, viene prima calcolato l'hash del messaggio m . Successivamente, si genera un numero casuale k , che viene utilizzato per calcolare un punto ellittico (x_1, y_1) tramite la moltiplicazione scalare del punto base G . Dalla componente x_1 del punto

risultante si estrae r , mentre s viene calcolato combinando k , h , e la chiave privata d . Infine, la coppia (r, s) rappresenta la firma digitale.

3.4.2 Algoritmo di Verifica

La procedura per verificare una firma $\sigma = (r, s)$ per un dato messaggio m e una chiave pubblica Q è illustrata nel seguente pseudocodice:

```

1 // Verify a signature (verify(pk, m, \sigma))
2 function verify(Q, m, r, s):
3     if r < 1 or r >= n or s < 1 or s >= n:
4         return False
5     h = H(m)
6     w = modular_inverse(s, n)
7     u1 = (h * w) mod n
8     u2 = (r * w) mod n
9     (x1, y1) = point_add(scalar_multiply(G, u1), scalar_multiply(Q, u2))
10    if is_point_at_infinity((x1, y1)):
11        return False
12    v = x1 mod n
13    return v == r

```

Il processo di verifica inizia controllando che i valori r e s siano compresi nell'intervallo valido. Viene quindi calcolato l'hash del messaggio m e determinato l'inverso modulare di s . Successivamente, si combinano le componenti u_1 e u_2 per calcolare un punto ellittico (x_1, y_1) . Infine, la verifica riesce se la componente x_1 del punto ottenuto corrisponde a r .

3.5 Aspetti Critici di Sicurezza

Dependenza dalla Casualità

Un elemento fondamentale per la sicurezza dell'ECDSA è la corretta generazione del parametro casuale k . Se k viene ripetuto o è prevedibile, un attaccante potrebbe recuperare la chiave privata d . Pertanto, è essenziale utilizzare una sorgente di casualità crittograficamente sicura per generare k .

Resistenza agli Attacchi

Per garantire la sicurezza dell'ECDSA, è necessario rispettare alcune condizioni:

- Utilizzare una curva ellittica ben studiata e resistente ai metodi di attacco noti.
- Scegliere una funzione hash robusta e collision-resistant.
- Evitare errori di implementazione, come la gestione errata dei numeri o la mancanza di validazione degli input.

Bitcoin utilizza l'ECDSA per firmare le transazioni. Ogni transazione è accompagnata da una firma digitale che garantisce:

- Solo il proprietario della chiave privata può autorizzare una transazione.

- L'integrità strutturale della transazione è preservata grazie all'utilizzo dell'hash del messaggio nella firma.

3.6 Public Keys as Identities

Le chiavi pubbliche sono fondamentali per la gestione delle identità in sistemi crittografici **decentralizzati**. Questo documento analizza come le chiavi pubbliche fungano da identità univoci, il processo di creazione di nuove identità, e gli aspetti di privacy associate a tali meccanismi, con particolare riferimento al contesto di Bitcoin.

Una chiave pubblica (public key) rappresenta un'identità digitale autenticabile. La verifica di una firma digitale `verify(pk, msg, sig)` che restituisce `true` implica che la chiave privata associata (`sk`) abbia firmato il messaggio `msg`. Tale operazione può essere interpretata come l'affermazione: "la chiave pubblica `pk` afferma: `[msg]`".

3.6.1 Controllo dell'Identità

Il controllo di un'identità basata su chiavi pubbliche è strettamente legato al possesso della corrispondente chiave privata (`sk`). In pratica, solo il proprietario della chiave privata è in grado di generare firme digitali valide per l'identità associata alla chiave pubblica (`pk`). Questo meccanismo garantisce che l'autenticità dell'identità dipenda esclusivamente dal possesso esclusivo della chiave privata. Pertanto, la sicurezza dell'intero sistema crittografico dipende dalla capacità del proprietario di proteggere la propria chiave privata da accessi non autorizzati.

3.7 Creazione di Nuove Identità

La creazione di una nuova identità in un sistema crittografico decentralizzato avviene attraverso la generazione casuale di una coppia di chiavi crittografiche (`(sk, pk)`). Il processo è semplice e autonomo: ogni utente può generare qualsiasi numero di coppie di chiavi senza richiedere autorizzazione o intervento di terze parti. Una volta generata, la chiave pubblica (`pk`) diventa il "nome" pubblico dell'identità.

In contesti come Bitcoin, si preferisce utilizzare un hash della chiave pubblica (ad esempio tramite SHA-256) per ridurre la lunghezza dell'identificatore e migliorarne la sicurezza. L'utilizzo di un hash ha diversi vantaggi:

- Riduce la complessità dell'identificatore, rendendolo più facile da gestire e memorizzare.
- Aggiunge un ulteriore strato di sicurezza, poiché l'hash non rivela direttamente la chiave pubblica sottostante.

L'uso di una coppia di chiavi crittografiche implica due caratteristiche cruciali:

- Il controllo dell'identità è esclusivo del proprietario della chiave privata (`sk`), garantendo che nessun altro possa agire a nome dell'identità associata.
- L'identità appare "casuale" a chi non conosce il legame tra `sk` e `pk`, garantendo anonimato iniziale e rendendo difficile correlare diverse identità ad uno stesso utente.

3.8 Gestione Decentralizzata delle Identità

3.8.1 Autonomia di Creazione

Un aspetto distintivo dei sistemi decentralizzati come le blockchain è la possibilità per ogni utente di generare un numero illimitato di identità in modo indipendente. Non esiste alcuna autorità centrale di registrazione o validazione; le identità (denominate *address* in Bitcoin) sono automaticamente valide una volta utilizzate all'interno del sistema. Questa caratteristica offre una grande flessibilità e autonomia agli utenti, permettendo loro di creare nuove identità per diverse finalità (ad esempio, per separare transazioni personali da quelle professionali).

3.8.2 Implementazione in Bitcoin

Nel contesto di Bitcoin, un indirizzo (*address*) è comunemente derivato dall'hash di una chiave pubblica. Questo approccio risponde a due obiettivi principali:

- Riduce la complessità dell'identificatore, rendendo gli indirizzi più brevi e pratici da gestire.
- Aggiunge una barriera aggiuntiva di sicurezza tramite l'utilizzo di funzioni di hashing crittografiche, che proteggono la chiave pubblica originale da eventuali attacchi di recupero diretto.

Tuttavia, è importante notare che l'utilizzo di indirizzi derivati da hash non elimina completamente i rischi di correlazione tra attività associate allo stesso utente.

3.9 Pseudonymity

Le identità basate su chiavi pubbliche sono *pseudonime*, poiché non sono direttamente legate a identità reali. Tuttavia, questa pseudonimia presenta limiti significativi. Un osservatore esterno può correlare le attività associate a una stessa chiave pubblica (o al suo hash) attraverso il tempo, tracciando transazioni e comportamenti all'interno del sistema. Sebbene le identità siano tecnicamente anonime all'inizio, l'analisi di pattern transazionali può rivelare informazioni sensibili sulle abitudini e le interazioni dell'utente.

Bitcoin non garantisce *anonimato*, ma solamente pseudonimità. Tutte le transazioni sono pubbliche e immutabili, registrate permanentemente sulla blockchain. Ciò significa che un'analisi di rete può correlare transazioni successive allo stesso indirizzo, rivelando informazioni sulle transazioni e sul comportamento dell'utente. Ad esempio:

- Se un indirizzo viene associato a un'attività nota (come un acquisto presso un negozio online), tutte le altre transazioni correlate a quel medesimo indirizzo possono essere tracciate.
- Tecniche avanzate di analisi di rete, come il clustering di indirizzi, possono rivelare gruppi di indirizzi controllati dallo stesso utente, compromettendo ulteriormente la privacy.

3.10 Cryptocurrencies

Nel tentativo di creare una nuova moneta digitale, si riscontrano sfide significative legate alla sicurezza, alla scalabilità e alla decentralizzazione. Alcuni approcci falliscono a causa di attacchi di **double spending** (spesa doppia) o di vulnerabilità nel meccanismo di validazione delle transazioni. Questa sezione analizza i requisiti necessari per una moneta digitale funzionale e gli ostacoli che si presentano nel soddisfarli.

3.10.1 Proprietà fondamentali di una moneta digitale

Per essere accettata come moneta valida, una criptovaluta deve soddisfare i seguenti principi fondamentali:

3.10.1.1 Coins immutabili (Immutable Coins)

Ogni unità di valuta (coin) deve essere **irreversibile e indivisibile**. In pratica, ciò significa che:

- Non è possibile modificare il contenuto di una coin una volta emessa.
- Non è consentita la suddivisione o la combinazione fisica di monete.

Per simulare l'effetto di una suddivisione, si utilizzano transazioni che creano nuovi output. Ad esempio, per dividere un importo, un utente può:

1. Consumare una coin esistente.
2. Generare due nuove transazioni che assegnano due nuove coins a se stesso o a un destinatario.

Questo meccanismo mantiene l'immaginario di *coin* come unità indivisibile, mentre la flessibilità viene garantita attraverso la concatenazione di transazioni.

3.10.1.2 Mancata dipendenza da autorità centralizzate

Un sistema di criptovaluta deve operare in modo **trustless** (senza fiducia in soggetti centrali). Questo richiede:

- Un meccanismo di consenso distribuito per validare le transazioni.
- Un sistema anti-censorship per prevenire l'interferenza di terzi.

La domanda fondamentale è: *Come garantire l'integrità del sistema senza un'autorità centrale di controllo?*

3.10.2 Principali sfide nell'implementazione

3.10.2.1 Attacchi di double spending

Il problema più critico è la **spesa doppia**: un utente malintenzionato potrebbe tentare di spendere la stessa coin in due transazioni differenti. Questo avviene quando:

- Viene inviata una transazione a due nodi diversi della rete.
- Viene modificata una transazione già inclusa in un blocco precedente.

Per prevenirlo, il sistema deve garantire che ogni transazione sia:

- **Univoca:** non può essere duplicata o alterata.
- **Inviolabile:** non può essere modificata retroattivamente.

3.10.2.2 Decentralizzazione e coordinamento

L'assenza di un'autorità centrale introduce sfide come:

- **Consenso di massa:** come raggiungere un accordo tra migliaia di nodi?
- **Prevenzione di attacchi di potere computazionale:** come evitare che un attaccante con risorse elevate possa sovvertire il sistema?

Questi problemi richiedono algoritmi come *Proof of Work* o *Proof of Stake*, che vanno oltre la semplice struttura di base di una criptovaluta.

3.11 Centralization vs Decentralization

3.11.1 Aspetti decentralizzati in Bitcoin

Bitcoin cerca di minimizzare la centralizzazione, ma non è del tutto immune a essa. I cinque aspetti fondamentali sono:

Gestione del ledger

Il libro mastro (blockchain) è mantenuto da una rete peer-to-peer distribuita. Ogni nodo ha una copia completa, ma la validazione è decentralizzata grazie al consenso di massa.

Autorità sulle transazioni

La validità delle transazioni è determinata dal consenso distribuito dei nodi, che applicano regole predefinite (es. firma digitale corretta).

Creazione di nuove monete

I nuovi Bitcoin vengono emessi tramite il processo di *mining*, un meccanismo incentivato ma aperto a chiunque possieda potere di calcolo.

Modifiche ai protocolli

Le modifiche richiedono il consenso della maggioranza della rete. Tuttavia, i **sviluppatori principali** (core developers) esercitano influenza significativa sulla direzione tecnica.

Valore delle criptovalute

Il valore di Bitcoin deriva dalla sua accettazione come mezzo di scambio e riserva di valore, ma dipende da strutture centralizzate (es. borse di scambio) che gestiscono la liquidità.

3.11.2 Centralizzazione pratica

Anche in sistemi decentralizzati, tendenze centralizzanti emergono in:

- **Miner:** La maggior parte del potere di calcolo è concentrato in poche organizzazioni (mining pools).
- **Servizi intermediari:** Portafogli, borse e nodi di rete sono spesso gestiti da entità centralizzate.
- **Governance:** Le decisioni critiche spesso riflettono le opinioni di un piccolo gruppo di sviluppatori.

3.12 Consenso Distribuito

3.12.1 Il problema fondamentale di Bitcoin

Il **consenso distribuito** rappresenta la sfida centrale per le criptovalute decentralizzate. In Bitcoin, il problema si pone come segue: *Come raggiungere un accordo su quale sequenza di transazioni è valida, senza ricorrere ad autorità centralizzate?* Questo obiettivo richiede un protocollo capace di coordinare migliaia di nodi indipendenti, garantendo simultaneamente sicurezza, scalabilità e resistenza agli attacchi.

3.12.2 Definizione formale di consenso

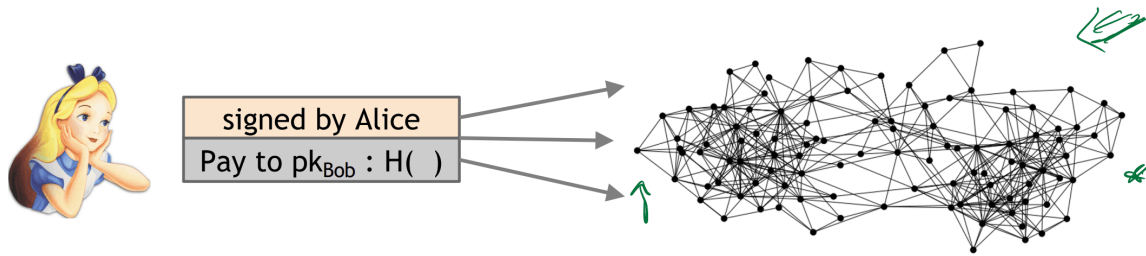
Un protocollo di consenso distribuito è considerato funzionale se soddisfa due requisiti fondamentali:

1. **Terminazione:** Il processo di decisione deve concludersi in un tempo finito, permettendo alla rete di raggiungere uno stato stabile.
2. **Aggiustamento:** Tutti i nodi corretti devono concordare su un unico valore proposto da almeno un nodo integro. Questo garantisce l'uniformità delle informazioni tra tutti i partecipanti.

Questi principi costituiscono la base teorica per qualsiasi sistema di consenso decentralizzato.

3.12.3 Il meccanismo di consenso in Bitcoin

La blockchain di Bitcoin è una sequenza gerarchica di blocchi di transazioni. Ogni blocco contiene un insieme di transazioni validate, un hash crittografico che lo collega al blocco precedente e un nonce generato durante il Proof of Work. I nodi concordano sulla *blockchain principale* (la più lunga), che rappresenta la versione accettata della storia delle transazioni. Questa struttura garantisce un ordine temporale e una traccia immutabile delle operazioni.



3.12.3.1 Protocollo operativo

Il processo di consenso funziona in tre fasi:

1. **Accumulo di transazioni:** I nodi memorizzano in una *mempool* le transazioni non ancora incluse in un blocco. Queste sono validate per conformità alle regole del protocollo (es. firma digitale corretta).
2. **Generazione dei blocchi:** I miner selezionano transazioni dalla mempool per creare un nuovo blocco. Risolvono un problema computazionale (Proof of Work) per dimostrare un contributo di risorse, rendendo il blocco candidato alla validazione.
3. **Propagazione e conferma:** Il blocco valido viene propagato alla rete. I nodi accettano il blocco se estende la blockchain corrente, aggiornando così la versione accettata. Le transazioni incluse vengono confermate progressivamente con il raggiungimento di nuovi blocchi successivi (6 conferme sono considerate standard per la sicurezza).

Questo ciclo ripetuto stabilisce un ordine univoco delle transazioni e garantisce che la rete converga su una visione condivisa.

3.12.4 Obiettivi e ostacoli nel raggiungere il consenso

Il consenso distribuito è minacciato da diverse fonti di instabilità:

- **Nodi difettosi o malintenzionati:** Nodi che falliscono o agiscono in modo corrotto (es. inviando transazioni false) possono creare conflitti.
- **Rete imperfetta:** Ritardi, divisioni della rete (fork) e perdita di pacchetti introducono incertezza nell'ordine delle transazioni.
- **Assenza di tempo globale:** Senza un orologio sincronizzato tra nodi, è impossibile definire un ordine assoluto delle operazioni.

Questi fattori rendono il consenso una sfida complessa, richiedendo soluzioni resistenti a tali condizioni.

3.12.4.1 Risultati di impossibilità teorica

Due teoremi fondamentali delineano i limiti del consenso deterministico:

- Il **Problema dei Generali Bizantini** dimostra che, in presenza di nodi maliziosi, il consenso deterministico è impossibile.
- Il teorema di **Fischer-Lynch-Paterson (1985)** afferma che, persino con un solo nodo difettoso, il consenso deterministico è irrealizzabile in sistemi asincroni.

Tali risultati si applicano a modelli teorici deterministici, ma Bitcoin adotta un approccio probabilistico che evita tali vincoli.

3.12.5 Bitcoin: un compromesso tra teoria e pratica

Bitcoin supera i limiti teorici mediante tre innovazioni chiave:

- **Incentivi economici:** I miner sono motivati a cooperare grazie ai premi di blocco e alle commissioni, creando un interesse al mantenimento della rete.
- **Randomness e tempo scalare:** Il Proof of Work introduce casualità nel processo di generazione dei blocchi, mentre il consenso diventa probabilistico e si rafforza col tempo (più blocchi confermano la validità).
- **Tolleranza alla latenza:** Il consenso è raggiunto in tempi prolungati (circa un'ora per 6 conferme), accettando un certo ritardo per guadagnare sicurezza.

Questo modello abbandona la determinazione immediata a favore di una convergenza asintotica, resistenti a attacchi teorici come il 51%.

I teoremi di impossibilità mantengono un ruolo cruciale:

- Identificano vulnerabilità (es. attacchi 51%), guidando misure di prevenzione.
- Stimolano la ricerca di protocolli probabilistici più efficienti, pur se ancora in fase di sviluppo per modelli come Bitcoin.

Così, mentre la teoria delinea i limiti, la pratica Bitcoin dimostra che soluzioni basate su incentivi e casualità possono funzionare in contesti reali.

3.13 Consensus without identity: the block chain

In molti protocolli decentralizzati, l'utilizzo di identità di nodi è fondamentale per garantire sicurezza e autenticazione. Tuttavia, Bitcoin adotta una strategia alternativa: **non utilizza identità esplicite** per i nodi. Questa scelta è motivata da due fattori chiave:

- **Resistenza agli attacchi Sybil:** Un attaccante (adv) potrebbe creare molteplici nodi falsi (Sybil attack) per dominare la rete.
- **Pseudonimità:** Bitcoin mira a mantenere l'anonimato degli utenti, evitando il legame tra transazioni e identità reali.

A causa di questi requisiti, Bitcoin opera con **ipotesi più deboli**: anziché fidarsi di identità controllate, seleziona casualmente i nodi per la generazione dei blocchi, come in un *lotteria*.

3.13.1 Il meccanismo di consenso implicito

Il consenso in Bitcoin è basato su un processo **implicito** che funziona come segue:

1. **Selezione casuale:** In ogni round, un nodo è scelto casualmente per proporre un nuovo blocco. Questa selezione è effettuata tramite il *Proof of Work*, che funge da "biglietto" per partecipare alla lotteria.

2. **Proposta di blocchi:** Il nodo selezionato propone un blocco che estende la blockchain corrente, includendo transazioni validate.
3. **Accettazione implicita:** Gli altri nodi accettano il blocco estendendolo con un nuovo blocco (se validi) o ignorandolo per proseguire da un precedente. Questo meccanismo crea una gerarchia di trust basata sulla lunghezza della catena.

Ogni blocco contiene un *hash* del blocco precedente, creando una catena crittograficamente legata. La validità di una transazione dipende quindi dalla posizione nel ramo principale della blockchain.

3.13.2 Algoritmo di consenso semplificato

Il processo di consenso è formalizzato in cinque fasi:

1. **Diffusione delle transazioni:** Le nuove transazioni vengono propagate a tutti i nodi.
2. **Raccolta in blocchi:** Ogni nodo accumula transazioni in una *mempool* e le organizza in blocchi candidati.
3. **Selezione casuale:** Un nodo è scelto casualmente per generare il blocco successivo risolvendo un problema computazionale (Proof of Work).
4. **Validazione:** I nodi accettano il blocco solo se tutte le transazioni sono valide (es. firme digitali corrette, utxo non spesi).
5. **Estensione:** I nodi confermano l'accettazione includendo il *hash* del blocco nel successivo.

Questo ciclo ripetuto garantisce che la rete converga verso una visione condivisa della blockchain.

3.13.3 Attacchi da parte di nodi maliziosi

3.13.3.1 Attacco di Stealing Bitcoin

Un utente malizioso può tentare di rubare bitcoin impedendo a un'altra persona di spostare i propri fondi. Questo avviene quando un nodo malizioso rifiuta deliberatamente di propagare una transazione valida verso il resto della rete. Ad esempio, supponiamo che Alice voglia trasferire alcuni bitcoin al proprio indirizzo, ma un nodo controllato da un attaccante blocchi questa transazione. Una transazione nel protocollo Bitcoin è considerata valida solo dopo essere stata inclusa in un blocco e aggiunta al blockchain. Tuttavia, per essere inclusa, la transazione deve essere prima propagata correttamente tra i nodi onesti della rete. Se un gruppo di nodi maliziosi decide di non propagare le transazioni di Alice, queste potrebbero non raggiungere mai i minatori, rimanendo così indefinite ed invalidando il tentativo di Alice di spostare i propri fondi. Ovviamente, è estremamente improbabile che tutti i nodi della rete collaborino per bloccare le transazioni di uno specifico utente. Al massimo, un piccolo gruppo di nodi maliziosi potrebbe solo ritardare temporaneamente l'inclusione delle transazioni, senza causare danni permanenti. Gli utenti possono sempre ripropagare manualmente le proprie transazioni o attendere che vengano accolte da nodi onesti.

3.13.3.2 Attacco di negazione del servizio (DoS)

Un nodo malizioso può rifiutare di includere transazioni specifiche (es. di un utente *Bob*). Tuttavia, altri nodi onesti includeranno quelle transazioni in successivi blocchi, limitando l'impatto dell'attacco.

3.13.3.3 Double Spending

Il più grave attacco nel contesto di Bitcoin è il **double spending**, ovvero la situazione in cui un utente malizioso (Alice) tenta di spendere lo stesso denaro in due transazioni distinte. Il processo si svolge come segue:

1. Alice invia una transazione a Bob, che riceve quindi la richiesta di pagamento. In cambio della transazione, Bob fornisce un bene o servizio.
2. Se Alice viene selezionata come miner, può proporre un blocco contenente una transazione alternativa che ripristina i fondi nel suo portafoglio. Ad esempio, potrebbe creare un blocco che utilizza gli stessi bitcoin spesi per pagare Bob, ma li reindirizza verso se stessa o verso altre transazioni.
3. Un altro miner estenderà la blockchain aggiungendo un blocco successivo, scegliendo tra le due transazioni proposte da Alice. Di conseguenza, solo una delle due transazioni verrà confermata sulla blockchain principale, mentre l'altra sarà scartata.

In questo scenario, solo una delle due transazioni diventa parte della blockchain, causando una perdita per Bob se la transazione legittima viene scartata. In sostanza, Alice ha cercato di usare lo stesso denaro per effettuare due pagamenti distinti.

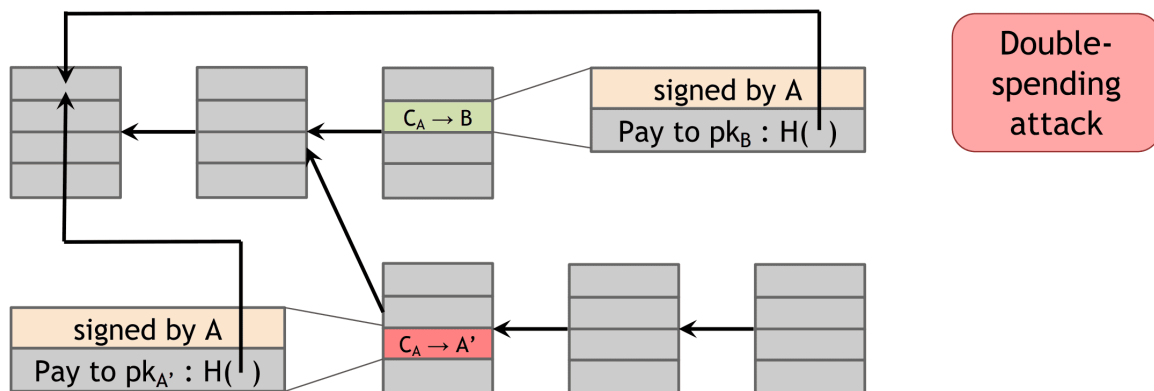


Figure 3.2: Schema di un attacco di double spending.

3.13.3.4 Prevenzione tramite conferme multiple

Per ridurre il rischio di subire un attacco di double spending, gli utenti attendono un numero sufficiente di conferme multiple prima di ritenere una transazione sicura. Nello specifico, Bob non fornirà il bene o servizio ad Alice fino a quando non vi saranno abbastanza conferme che garantiscano l'inclusione definitiva della transazione nella blockchain. Le conferme funzionano nel modo seguente:

- Ogni conferma corrisponde all'inclusione della transazione in un blocco successivo.
- La probabilità di successo di un attacco di double spending diminuisce esponenzialmente con il numero di conferme.

La pratica standard prevede di attendere almeno 6 conferme, il che richiede circa 1 ora, riducendo il rischio a livelli trascurabili.

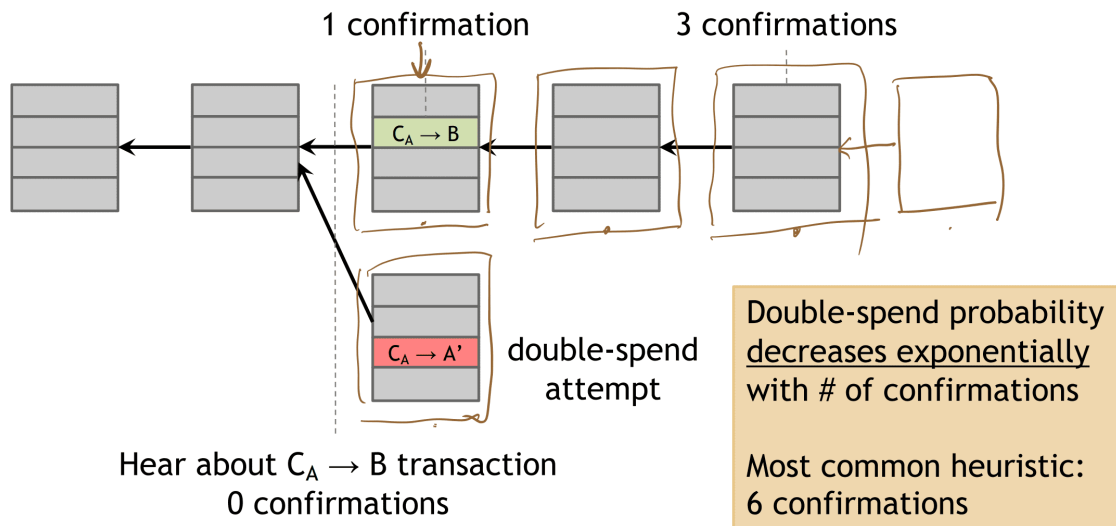


Figure 3.3: Al crescere del numero di conferme, decresce significativamente la probabilità di mettere a segno un double-spending.

3.13.4 Sicurezza e validità delle transazioni in Bitcoin

La sicurezza del sistema Bitcoin si basa su due principi fondamentali:

- **Validità delle transazioni:** Garantita da meccanismi di crittografia avanzati, come le firme digitali che attestano l'autenticità dell'utente emittente.
- **Prevenzione del double spending:** Garantita dal consenso distribuito della rete, che coordina i nodi per evitare lo spesa doppia di fondi.

Tuttavia, nessuna transazione è mai 100% sicura: la garanzia della validità è *probabilistica* e dipende dalla lunghezza della catena di blocchi. Gli utenti e i nodi onesti operano secondo il principio del **ramo più lungo** (longest chain rule), che stabilisce che:

- La rete converga sempre verso una visione univoca della blockchain, evitando conflitti.
- Gli attacchi messi in atto da attaccanti con una percentuale minore di potere computazionale nella rete falliscano automaticamente.

Questa architettura rende la sicurezza di Bitcoin intrinsecamente legata al costo economico e computazionale necessario per controllare la maggioranza della rete. Solo un attaccante in grado di dominare oltre il 50% delle risorse di calcolo potrebbe alterare la blockchain, un evento economicamente irrealizzabile grazie alla decentralizzazione del sistema.

Chapter 4

Lezione 4

4.1 Incentivi e Proof-of-Work

4.1.1 Problemi con l'ipotesi di onestà

L'ipotesi che i nodi partecipanti al network blockchain agiscano onestamente è problematica, poiché non è sufficiente per garantire la sicurezza del sistema. Per incentivare i nodi a seguire le regole, è necessario introdurre meccanismi che li motivino a comportarsi correttamente. In particolare, è stato proposto di premiare i nodi che contribuiscono alla creazione di blocchi validi (*block creation*), in modo che la loro onestà sia legata direttamente al proprio interesse economico.

4.1.2 Incentivi per i nodi

Due incentivi principali sono stati introdotti nel protocollo Bitcoin:

4.1.2.1 Incentivo 1: Block Reward

Il creatore di un blocco ha il diritto di includere una transazione speciale (*coin-creation transaction*) all'interno del blocco stesso. Questa transazione assegna nuovi Bitcoin a un indirizzo specificato dal minatore (di solito il proprio). La validità di questa transazione è vincolata al fatto che il blocco in cui è incluso diventi parte della catena di consenso (*consensus chain*). Questo meccanismo garantisce che i miner abbiano un interesse economico a non alterare il blocco, poiché altrimenti non riceverebbero il premio.

Il valore del *block reward* è fissato per periodi di 4 anni e viene dimezzato (*halving*) ogni 4 anni. Il blocco genitore (*Genesis block*), creato il 3 gennaio 2009, ha assegnato un reward di 50 BTC. Dopo 9 anni (circa 2018), il reward era sceso a 12.5 BTC grazie a due dimezzamenti (2013 e 2017). La quantità totale di Bitcoin è limitata a 21 milioni, prevista per essere esaurita intorno al 2140.

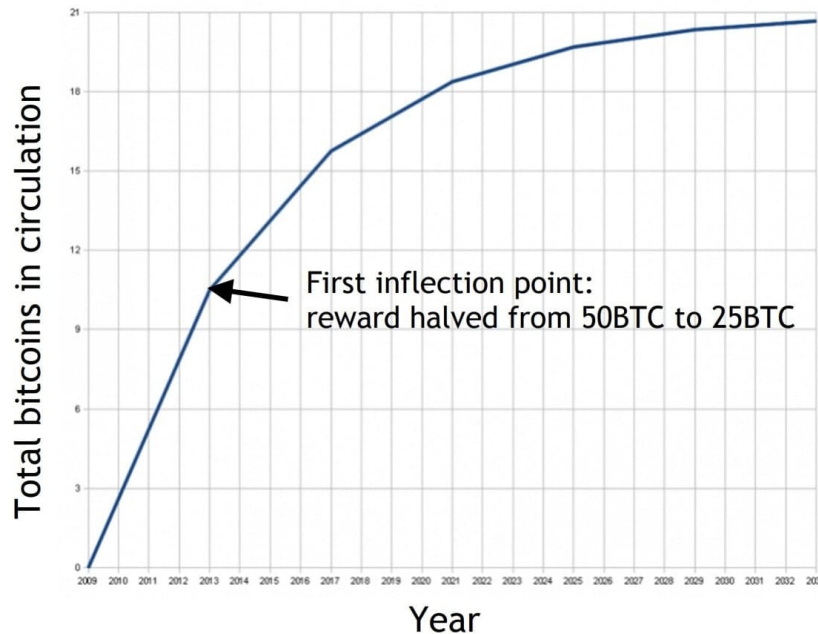


Figure 4.1: Andamento del numero totale di Bitcoin in circolazione.

4.1.2.2 Incentivo 2: Transaction Fee

I creatori di transazioni possono scegliere di assegnare un valore di output inferiore a quello dell'input, con il resto che funge da *transaction fee* assegnato al creatore del blocco. Attualmente queste commissioni sono volontarie (come un "tip"), ma diventeranno obbligatorie dopo l'esaurimento dei Bitcoin (post-2140). Questo sistema incentiva i miner a includere transazioni con fee più elevate, garantendo fluidità al network.

4.1.3 Problemi da risolvere

Tre sfide chiave sono state identificate:

1. Selezionare casualmente un nodo per la creazione del blocco.
2. Evitare conflitti dovuti alla competizione per i premi.
3. Prevenire attacchi *Sybil* (creazione di molti nodi falsi per dominare il network).

4.1.4 Proof-of-Work (PoW): Soluzione unica

Tutti i problemi sono affrontati mediante il meccanismo *Proof-of-Work*, che utilizza la potenza di calcolo come risorsa vincolante.

4.1.4.1 Funzione del PoW

- **Selezione proporzionale:** Il nodo che propone il blocco è scelto in base alla propria quota di potenza di calcolo (*hash power*) rispetto al network globale. Questo impedisce che un attaccante acquisisca un vantaggio monopolistico.
- **Competizione per la creazione del blocco:** I miner competono risolvendo un puzzle matematico, dove il vincitore è chi per primo trova una soluzione valida.

- **Prevenzione degli attacchi Sybil:** L'esigenza di investire in hardware per aumentare la potenza di calcolo rende costoso e poco redditizio creare molti nodi falsi.

4.1.4.2 Il puzzle di hash

Il PoW richiede ai miner di trovare un *nonce* tale che:

$$H(\text{nonce} \parallel \text{prev_hash} \parallel \text{tx} \parallel \dots) < \text{target} \quad (4.1)$$

Questo processo richiede tentativi ripetuti, con una difficoltà parametrizzabile. Ad esempio, nel 2014, circa 10^{20} tentativi erano necessari per risolvere un puzzle.

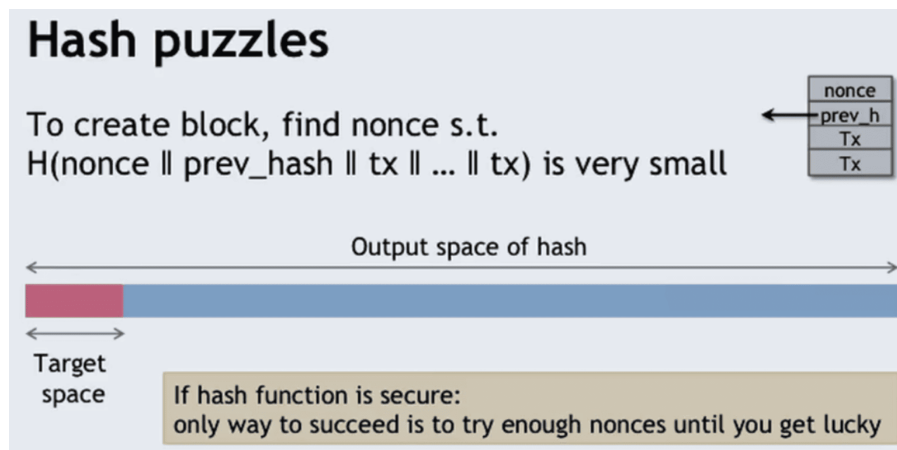


Figure 4.2: Il puzzle di hash richiede potenza di calcolo per trovare un nonce che generi un risultato inferiore al target.

4.1.4.3 Proprietà del PoW

1. **Costo computazionale elevato:** La difficoltà è regolata automaticamente ogni 2016 blocchi (circa 2 settimane) per mantenere il tempo medio tra blocchi a 10 minuti.
2. **Verificabilità rapida:** Altri nodi possono confermare rapidamente la validità della soluzione verificando l'equazione (1).
3. **Distribuzione di probabilità:** La probabilità che un miner vinca il blocco è proporzionale alla sua quota di *hash power*. Ad esempio, un miner con il 10% della potenza globale ha il 10% di probabilità di vincere.

4.1.5 Assunzione di sicurezza

La sicurezza del network è garantita solo se la maggioranza dei miner (in termini di *hash power*) segue le regole. Se un attaccante controllasse più del 50% della potenza di calcolo, potrebbe sovvertire il sistema modificando la catena. Tuttavia, l'investimento necessario per raggiungere tale livello è economicamente non sostenibile.

4.1.6 Economia del Mining

La sostenibilità economica del mining dipende dall'equilibrio tra ricavi e costi. I miner ottengono ricavi grazie alle ricompense per blocco (block reward) e alle commissioni sulle transazioni (Tx fees). Tuttavia, devono affrontare costi significativi, tra cui il consumo di elettricità, l'acquisto e la manutenzione dell'hardware.

$$\text{If mining reward (block reward + Tx fees)} > \text{hardware + electricity cost} \rightarrow \text{Profit}$$

Per essere redditizio, un'operazione di mining deve garantire che i ricavi superino i costi. Tuttavia, questa condizione è influenzata da vari fattori, tra cui:

- La volatilità del valore della criptovaluta minata, che può influenzare il valore dei ricavi.
- La difficoltà del mining, che dipende dall'hash rate globale e determina il tempo necessario per trovare un nuovo blocco.
- I costi energetici, che variano in base alla localizzazione geografica e all'efficienza dell'hardware utilizzato.

Il Proof-of-Work (PoW) assicura la sicurezza della rete distribuendo la potenza computazionale tra i miner, ma è stato criticato per il suo elevato consumo energetico. Alternative come il Proof-of-Stake (PoS) cercano di ridurre questi costi, ma PoW rimane attualmente il meccanismo di consenso più diffuso per le blockchain decentralizzate.

4.2 Mettendo tutto insieme

Il protocollo Bitcoin integra una serie di componenti interdipendenti che ne garantiscono il funzionamento decentralizzato. L'identità degli utenti viene gestita attraverso coppie di chiavi crittografiche asimmetriche, mentre le transazioni vengono validate e propagate attraverso una rete peer-to-peer (P2P). Il registro distribuito (blockchain) si fonda su un meccanismo di consenso proof-of-work, sostenuto da hash puzzles computazionalmente impegnativi risolti attraverso il processo di mining.

L'ecosistema si autosostiene attraverso tre pilastri interconnessi:

- La sicurezza della blockchain, garantita dalla potenza computazionale dedicata al mining
- La stabilità economica dell'ecosistema minerario
- Il valore intrinseco della valuta digitale come incentivo economico

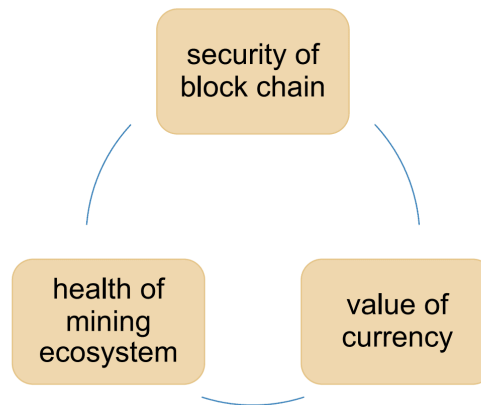


Figure 4.3: Schema di autosostentamento del protocollo Bitcoin

4.2.1 Attacchi al 51%

Un attaccante in controllo della maggioranza dell'hashrate della rete dispone di poteri circoscritti dal protocollo Bitcoin. La principale capacità operativa consiste nell'alterare temporaneamente la percezione dello stato della blockchain attraverso tre azioni principali:

- **Censura transazionale:** esclusione mirata di specifiche transazioni dai blocchi minati
- **Manipolazione dell'ordine:** modifica della sequenza delle transazioni non confermate
- **Riorganizzazione della catena:** invalidazione di blocchi recenti (tipicamente entro 5-6 conferme)

Le limitazioni fondamentali rimangono invariate:

- **Impossibilità di sottrazione fondi:** le firme crittografiche proteggono gli indirizzi esistenti
- **Immutabilità parametri protocollo:** reward minerario e supply totale non modificabili
- **Irreversibilità transazioni consolidate:** blocco con oltre 100 conferme risulta praticamente immutabile

L'effetto più rilevante sarebbe economico-sociale: la compromissione della fiducia nel sistema decentralizzato potrebbe innescare un deprezzamento del valore del bitcoin, minando gli incentivi economici alla base del mining. La sicurezza di Bitcoin deriva tuttavia dall'elevatissimo costo stimato per acquisire il 51% dell'hashrate globale (oltre \$20 miliardi in hardware) e dalla resilienza storica dimostrata dalla rete.

4.2.2 Transazioni Bitcoin

Il sistema di transazioni in Bitcoin rappresenta uno degli aspetti fondamentali dell'architettura della blockchain. Per comprenderne appieno il funzionamento, è utile analizzare i diversi

modelli teorici di registrazione delle transazioni, evidenziando le motivazioni che hanno portato all'adozione dell'attuale paradigma UTXO (Unspent Transaction Output).

4.2.2.1 Registro Basato su Conti (Account-Based)

L'approccio basato su conti rappresenta un modello teorico alternativo che, sebbene possa apparire intuitivo in quanto ispirato ai sistemi bancari tradizionali, non è stato adottato da Bitcoin per significative limitazioni di scalabilità. In questo paradigma, ogni utente possiede un account identificato univocamente a cui viene associato un saldo contabile che si aggiorna progressivamente con le transazioni effettuate.

Per comprendere le problematiche di questo approccio, si consideri un modello semplificato con una primitiva `create_coin()` che consente di generare monete valide. In uno scenario ipotetico, Alice crea 25 monete e ne trasferisce 17 a Bob, il quale successivamente spende 8 monete per pagare Carol. Quest'ultima trasferisce 5 monete ad Alice, che successivamente tenta di trasferire 15 monete a David. La validazione di quest'ultima transazione richiederebbe la ricostruzione completa dello stato del conto di Alice attraverso l'analisi di tutte le transazioni pregresse, per verificare l'effettiva disponibilità dei fondi necessari.

Questa architettura introduce una complessità computazionale lineare crescente: ogni nuovo partecipante alla rete deve ricostruire l'intero stato del sistema processando sequenzialmente tutte le transazioni a partire dal blocco genesis. Nella peggiore delle ipotesi, la verifica di una singola transazione comporterebbe l'esame dell'intero registro fino all'origine della blockchain, un processo computazionalmente oneroso che non scala efficacemente all'aumentare del numero di transazioni e utenti attivi nella rete.

Inoltre, la gestione degli aggiornamenti concorrenti diventa problematica in contesti decentralizzati, richiedendo meccanismi di locking degli account per prevenire race condition, un requisito difficilmente implementabile in un sistema distribuito come Bitcoin.

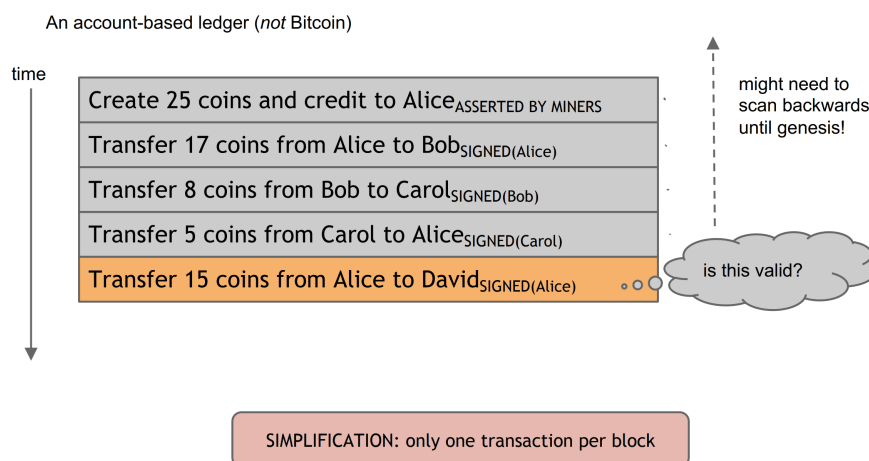


Figure 4.4: Modello account-based: gli account mantengono saldi aggiornati attraverso transazioni sequenziali

4.2.2.2 Registro Basato su Transazioni (UTXO)

Bitcoin implementa invece un modello basato su una rappresentazione a stati discreti denominato UTXO (Unspent Transaction Output). In questo paradigma, ogni transazione opera come una funzione di transizione di stato atomica: consuma output non spesi da transazioni precedenti (input) e genera nuovi output spendibili. Il concetto chiave è che non esistono "conti" persistenti, ma solo output di transazioni che possono essere utilizzati come input per transazioni successive.

In ogni transazione si specifica esplicitamente da quali UTXO precedenti provengono i fondi e gli output generati diventeranno potenziali input per future transazioni. Questo approccio offre un vantaggio significativo rispetto al modello account-based: per verificare una nuova transazione non è necessario ricostruire l'intero stato del sistema a partire dal blocco genesis, ma è sufficiente seguire i riferimenti (hash pointer) che collegano la transazione corrente alle transazioni precedenti che hanno generato gli UTXO utilizzati come input.

La transazione è identificata univocamente dall'hash del blocco stesso che la rappresenta. L'input contiene l'indice del blocco contenente la transazione che ha introdotto l'UTXO nel conto, ed in particolare l'indice del pagamento (poiché ci possono essere pagamenti multipli). Si precisa quale pagamento di quale transazione ha introdotto la disponibilità.

La verifica di validità di una transazione UTXO si articola in quattro fasi fondamentali:

1. Ogni input deve referenziare esplicitamente l'identificativo e l'indice di un UTXO esistente nella blockchain.
2. Si verifica che tali UTXO non siano già stati consumati in transazioni precedenti, prevenendo così la doppia spesa.
3. Le firme crittografiche associate a ciascun input vengono validate, garantendo che solo i legittimi proprietari possano spendere i fondi.
4. Si controlla la coerenza degli importi, assicurando che la somma degli input sia maggiore o uguale alla somma degli output, con la differenza eventualmente destinata come commissione per i minatori (fee mineraria).

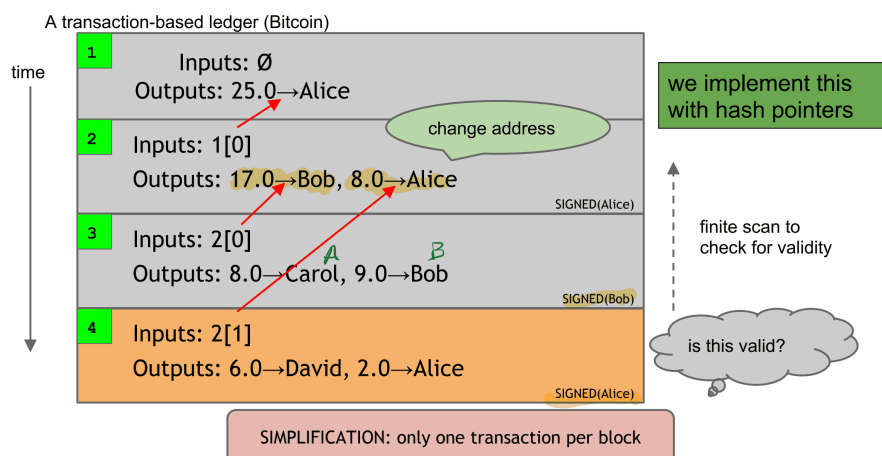


Figure 4.5: Gli output non spesi diventano input per nuove transazioni

Il modello UTXO abilita diverse proprietà avanzate che hanno contribuito al suo successo nell'ecosistema Bitcoin. L'aggregazione di valori permette a un utente di combinare multipli UTXO (anche di piccolo importo) in un'unica transazione, ottimizzando la gestione del portafoglio digitale. I pagamenti congiunti vengono realizzati attraverso script multi-firma che richiedono l'approvazione simultanea di più parti, implementando così funzionalità di controllo distribuito sui fondi.

Dal punto di vista della privacy, il modello beneficia della possibilità di generare output multipli indipendenti all'interno di una stessa transazione, dissociando potenzialmente gli indirizzi di input da quelli di output e rendendo più complessa l'analisi dei flussi transazionali. L'efficienza verificativa deriva dalla possibilità di validare transazioni in modo locale e indipendente, senza necessità di conoscere l'intero stato del sistema: è sufficiente dimostrare il possesso di UTXO validi e non spesi lungo il percorso dalla transazione origine al blocco corrente.

Questa architettura contribuisce significativamente alla scalabilità del sistema Bitcoin, permettendo verifiche parallele e indipendenti delle transazioni, un requisito fondamentale per un sistema di pagamento distribuito e decentralizzato.

Chapter 5

Lezione 5

5.1 Consenso di Nakamoto

5.1.1 Fondamenti del Consenso Distribuito

Il consenso distribuito rappresenta un elemento cruciale nei sistemi blockchain, caratterizzato da due proprietà fondamentali: la consistenza, che garantisce l'accordo tra i nodi onesti sul contenuto del registro distribuito, e la liveness, che assicura la progressione continua del sistema. L'implementazione di tali proprietà deve tenere conto dei diversi modelli di rete in cui il protocollo opera: si distinguono il modello sincrono, in cui i messaggi vengono consegnati istantaneamente, e il modello asincrono, in cui i messaggi possono arrivare in ordini potenzialmente diversi, introducendo complessità significative nella gestione del consenso.

5.1.2 Meccanismo del Consenso di Nakamoto

Il consenso di Nakamoto, elemento fondante della blockchain Bitcoin, si basa sul meccanismo di Proof of Work (PoW). In questo paradigma, i minatori competono per trovare un valore nonce tale che l'hash del blocco candidato risulti inferiore a un determinato target di difficoltà: $H(block) < Target$. Questa competizione computazionale è calibrata per produrre un nuovo blocco mediamente ogni 10 minuti, creando un ritmo regolare nell'espansione della blockchain.

Il protocollo implementa il principio della catena più lunga, secondo cui i minatori tentano sempre di estendere la catena con il maggior numero di blocchi di cui sono a conoscenza. Questo approccio euristico contribuisce alla convergenza del consenso in un ambiente decentralizzato. Un aspetto caratteristico del consenso di Nakamoto è la nozione di finalità probabilistica: in qualsiasi momento, tutti i blocchi eccetto gli ultimi k nella catena corrente sono considerati finali, con una probabilità di riorganizzazione che diminuisce esponenzialmente all'aumentare della profondità.

5.1.3 Proprietà Formali del Consenso

Le proprietà del consenso di Nakamoto possono essere formalizzate con maggiore precisione:

1. Consistency

La consistenza garantisce che i partecipanti onesti concordino su tutti i blocchi tranne gli ultimi k , con una probabilità di disaccordo che decresce esponenzialmente: $O(2^{-k})$. Questa proprietà assicura l'integrità storica della blockchain, nonostante la natura distribuita del sistema.

2. Chain quality

La qualità della catena (chain quality) impone che qualsiasi sequenza consecutiva di k blocchi contenga un numero "sufficientemente elevato" di blocchi generati da nodi onesti, con una probabilità di fallimento limitata a $O(2^{-k})$. In termini pratici, i minatori che controllano una frazione p della potenza computazionale dovrebbero produrre approssimativamente una frazione p dei blocchi, garantendo una rappresentazione proporzionale nel processo di consenso.

3. Chain growth

La crescita della catena (chain growth) assicura l'espansione della blockchain a un ritmo costante: se g rappresenta il parametro di crescita, la catena dovrebbe espandersi di k blocchi ogni k/g "round", mantenendo un progresso regolare indipendentemente dalle fluttuazioni nella partecipazione dei minatori.

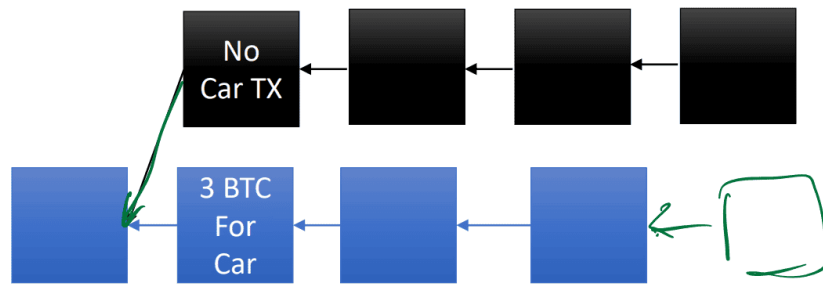
5.1.4 Limitazioni del Consenso di Nakamoto

Il consenso di Nakamoto presenta alcuni limiti operativi significativi. La necessità di attendere la conferma di circa 6 blocchi (corrispondenti a circa un'ora) prima che una transazione possa essere considerata definitivamente confermata introduce latenze considerevoli nell'elaborazione delle transazioni. Questa caratteristica rende il protocollo Bitcoin meno adatto per applicazioni che richiedono conferme rapide delle transazioni.

La naturale domanda che emerge è se sia possibile accelerare l'intero processo, ad esempio semplificando l'attività di mining. Tuttavia, come verrà analizzato successivamente, esistono vincoli fondamentali che limitano la possibilità di ridurre arbitrariamente la difficoltà del puzzle crittografico senza compromettere la sicurezza del sistema.

5.1.5 Intuizione della Consistenza

Per comprendere la proprietà di consistenza, si consideri uno scenario in cui un avversario A controlla una frazione $q < 1/2$ della potenza di hash totale. La domanda fondamentale è se A possa creare una catena alternativa più rapidamente rispetto ai partecipanti onesti, tentando di sovvertire l'ordine delle transazioni registrate.



5.1.6 Binomial Random Walk

Il problema può essere modellato attraverso passeggiate aleatorie binomiali. L'avversario A parte da un certo deficit rispetto alla catena principale. Un incremento di $+1$ indica che A ha estratto un blocco (riducendo il proprio divario di 1), mentre un decremento di -1 significa che i partecipanti onesti hanno estratto un blocco (aumentando il divario di 1). L'avversario A riesce nel proprio intento se riesce a raggiungere un determinato valore obiettivo z (partendo da 0) nella passeggiata aleatoria binomiale.

La probabilità P_z rappresenta la probabilità che A riesca mai a recuperare il divario z . Questa probabilità dipende crucialmente dal rapporto tra la potenza di calcolo dell'avversario e quella dei nodi onesti. Se indichiamo con p la probabilità che i nodi onesti estraggano un blocco e con q la probabilità che A estragga un blocco, la probabilità che A recuperi un divario di z blocchi è data da:

$$P_z = \begin{cases} \left(\frac{q}{p}\right)^z & \text{se } p > q \\ 1 & \text{altrimenti} \end{cases}$$

Questa formula rivela un aspetto fondamentale della sicurezza di Bitcoin: la probabilità di successo di un attacco diminuisce esponenzialmente con la profondità z dei blocchi, purché la potenza computazionale onesta superi quella dell'attaccante ($p > q$).

q=0.3	
z=1	P(z)=0.428571
z=2	P(z)=0.183674
z=3	P(z)=0.0787172
z=4	P(z)=0.033736
z=5	P(z)=0.0144583
z=6	P(z)=0.0061964
z=7	P(z)=0.0026556
z=8	P(z)=0.00113811
z=9	P(z)=0.000487763
z=10	P(z)=0.000209041

Figure 5.1: Con un rapporto $\frac{q}{p} = 0.3$, le probabilità di recupero per diversi valori di z sono:

5.1.7 Considerazioni sulla Difficoltà di Mining

Un'osservazione interessante è che l'analisi precedente non sembra implicare requisiti specifici sulla difficoltà del processo di mining. Ciò che risulta determinante è la proporzione di potenza computazionale onesta nel sistema. Sorge quindi spontanea la domanda: perché attendere 10 minuti per blocco? Non sarebbe possibile ridurre questo intervallo?

La risposta risiede nelle implicazioni che la difficoltà di mining ha sulla consistenza del protocollo. Non è possibile ridurre arbitrariamente la difficoltà del puzzle crittografico senza compromettere la sicurezza del sistema. Il fattore cruciale è il ritardo di rete (network delay), definito come *il numero di round necessari per propagare un blocco appena estratto tra tutti i nodi onesti*. Durante questo intervallo, i nodi onesti non possono contribuire efficacemente al consenso, mentre i nodi malevoli potrebbero non subire tale limitazione se ben organizzati. In particolare i giocatori onesti non lavorano su una versione aggiornata della blockchain fino a che non gli si propaga la versione che comprende il blocco aggiunto, per tanto tutta la potenza di calcolo che hanno utilizzato è andata sprecata. I giocatori malevoli potrebbero non essere sottoposti a questa condizione poiché presumibilmente meglio organizzati.

5.1.8 Scelta del Parametro di Difficoltà del Mining

Per comprendere la relazione tra difficoltà di mining e sicurezza del sistema, è necessario introdurre alcuni parametri fondamentali. In un sistema con n nodi partecipanti, la frazione α rappresenta la proporzione di nodi onesti, mentre p indica la probabilità che un singolo nodo onesto riesca a estrarre un blocco in un dato intervallo temporale.

La formalizzazione matematica di questo fenomeno rivela implicazioni sorprendenti. Se definiamo Δ come il tempo necessario per la propagazione di un blocco attraverso la rete e $\alpha p n \Delta$ come la probabilità che almeno un nodo onesto estragga un blocco in un singolo round, la potenza di mining effettiva degli utenti onesti risulta ridotta di un fattore proporzionale a $\alpha p n \Delta$.

Questa riduzione di efficienza può essere approssimata come $(1 - \alpha p n \Delta)\alpha n$, espressione che quantifica la frazione effettivamente utilizzabile della potenza di calcolo onesta quando si considera l'impatto del ritardo di rete.

5.1.9 Condizioni di Sicurezza

La sicurezza del sistema viene compromessa quando la potenza computazionale controllata da attori malevoli supera la potenza effettiva degli attori onesti. Definendo $\beta = (1 - \alpha p n \Delta)\alpha n$ come la frazione di nodi corrotti, la condizione critica di vulnerabilità può essere espressa come:

$$\beta n > (1 - \alpha p n \Delta)\alpha n$$

Questa relazione rivela un aspetto fondamentale: in presenza di ritardi di rete non trascurabili, la tradizionale condizione di maggioranza onesta ($\alpha > 0.5$) non è più sufficiente per garantire la sicurezza del protocollo. Il ritardo di propagazione Δ introduce un fattore

di degradazione che riduce l'efficacia della potenza di mining onesta, richiedendo quindi una maggioranza più ampia per mantenere la sicurezza.

Le analisi teoriche formali stabiliscono condizioni specifiche per la sicurezza del sistema Bitcoin. Il protocollo può essere considerato sicuro quando vengono soddisfatte simultaneamente le seguenti condizioni:

- La condizione generale: $2pn\Delta < \frac{1}{2}$, che limita l'impatto complessivo del ritardo di rete
- In presenza di latenza non nulla ($\Delta \neq 0$): $p < \frac{1}{4n\Delta}$, che vincola la probabilità di estrazione per singolo nodo

Questi vincoli matematici spiegano perché la difficoltà di mining non possa essere ridotta arbitrariamente: una diminuzione eccessiva comporterebbe un aumento di p , violando potenzialmente le condizioni di sicurezza e rendendo il sistema vulnerabile ad attacchi anche da parte di avversari che controllano meno della metà della potenza computazionale totale.

5.1.10 Proprietà della Blockchain di Nakamoto

La blockchain basata sul consenso di Nakamoto presenta caratteristiche distintive che ne determinano vantaggi e limiti operativi:

La partecipazione anonima consente a qualsiasi entità di contribuire al processo di consenso senza rivelare la propria identità, favorendo un'inclusività globale. Il sistema dimostra un'elevata scalabilità, permettendo ai nodi di unirsi e abbandonare la rete con facilità, senza procedure di registrazione centralizzate.

L'imprevedibilità del leader di blocco impedisce la manipolazione anticipata del processo di consenso, contribuendo alla sicurezza complessiva. Il protocollo può tollerare una significativa frazione di nodi disonesti, mantenendo le sue proprietà di sicurezza fintanto che la maggioranza della potenza computazionale rimane onesta.

D'altra parte, il sistema presenta limitazioni intrinseche: la lentezza nelle conferme delle transazioni lo rende inadatto per applicazioni che richiedono finalità rapida. Il processo di mining comporta un consumo di risorse elevato, sollevando preoccupazioni di sostenibilità economica e ambientale.

In presenza di ritardi di rete prolungati, le garanzie di sicurezza possono essere compromesse, rendendo possibili attacchi teorici. Tuttavia, è importante notare che tali vulnerabilità, sebbene teoricamente possibili, non sono state osservate nella pratica, testimoniando la robustezza empirica del protocollo Bitcoin.

5.1.11 Block-Withholding Attacks (Selfish Mining)

Il modello di sicurezza della blockchain di Nakamoto si basa tradizionalmente sull'assunzione che i minatori agiscano razionalmente annunciando immediatamente i blocchi estratti. Tuttavia, questa assunzione è stata messa in discussione dall'identificazione di strategie alternative potenzialmente più redditizie, note come attacchi di block-withholding o selfish mining. Questi attacchi rappresentano una deviazione significativa dal comportamento standard dei minatori e potrebbero compromettere le proprietà di sicurezza della

blockchain in determinate circostanze.

5.1.11.1 Strategia di Base del Selfish Mining

La strategia fondamentale del selfish mining consiste nel trattenere deliberatamente l'annuncio dei blocchi appena estratti, creando una catena privata parallela a quella pubblica. L'obiettivo dell'attaccante è accumulare un vantaggio in termini di lunghezza della catena, per poi rivelarlo strategicamente al momento opportuno. Questa tattica sfrutta la regola della "catena più lunga" del protocollo Bitcoin per invalidare blocchi legittimi estratti dalla comunità onesta, rendendo vano il loro lavoro computazionale.

Quando un avversario riesce a mantenere un vantaggio di due o più blocchi rispetto alla catena pubblica, si verifica una situazione particolarmente vantaggiosa: tutti gli altri minatori stanno investendo risorse computazionali su una catena destinata ad essere sostituita, sprecando effettivamente la loro potenza di calcolo. Questo comportamento non solo aumenta il guadagno relativo dell'attaccante rispetto alla sua effettiva potenza di mining, ma compromette anche l'equità distributiva della rete, potenzialmente alterando l'equilibrio economico dell'ecosistema.

5.1.11.2 Dinamiche Competitive e Race Condition

Un aspetto cruciale di questa strategia emerge quando l'attaccante si trova in vantaggio di un solo blocco rispetto alla catena pubblica. In questo scenario, la divulgazione di un nuovo blocco da parte della comunità onesta genera una condizione di competizione (race condition) tra due catene di pari lunghezza: quella dell'attaccante e quella pubblica.

L'esito di questa competizione dipende dalla propagazione dei blocchi nella rete. Il minatore che riesce a diffondere più rapidamente il proprio blocco successivo ottiene un vantaggio temporaneo, potenzialmente decisivo per la definizione della catena canonica. Questa dinamica introduce una dimensione strategica nell'attacco, dove fattori come la posizione nella topologia della rete e la velocità di connessione diventano elementi determinanti per il successo.

5.1.11.3 Condizioni di Redditività dell'Attacco

L'analisi matematica del selfish mining ha dimostrato che, in condizioni ideali dove l'attaccante vince ogni competizione di propagazione, la strategia risulta vantaggiosa per qualsiasi frazione α della potenza di mining sotto il controllo dell'avversario. Questa condizione ideale può essere ottenuta attraverso:

Una posizione ottimale nella topologia di rete, che consenta all'attaccante di propagare i propri blocchi più rapidamente rispetto ai nodi onesti, garantendo un vantaggio sistematico nelle race condition.

Meccanismi di corruzione (bribery), in cui l'attaccante incentiva economicamente altri minatori a unirsi alla propria strategia o a favorire la sua catena in caso di competizione, espandendo artificialmente la propria influenza sulla rete.

In scenari più realistici, dove l'attaccante ha una probabilità del 50% di vincere le competizioni di propagazione, l'analisi dimostra che la strategia di selfish mining diventa redditizia quando la frazione di potenza di mining controllata supera la soglia critica del 25% ($\alpha > 0.25$). Questo risultato rappresenta una significativa riduzione rispetto alla

tradizionale soglia di sicurezza del 50%, implicando che la blockchain di Bitcoin potrebbe essere vulnerabile ad attacchi da parte di entità che controllano meno della metà della potenza computazionale totale.

5.1.11.4 Implicazioni per il Modello di Sicurezza

L'identificazione del selfish mining rappresenta una sorprendente deviazione dalle assunzioni precedenti sulla razionalità dei minatori e sulla sicurezza del protocollo Bitcoin. Questo attacco teorico dimostra che, in determinate condizioni, seguire il protocollo come previsto non costituisce necessariamente la strategia ottimale per i partecipanti, contraddicendo un principio fondamentale su cui si basava la sicurezza della blockchain.

È importante sottolineare che, nonostante la sua rilevanza teorica, il selfish mining non è stato ancora osservato in pratica nell'ecosistema Bitcoin. Questa discrepanza tra teoria e pratica potrebbe essere attribuita a diversi fattori:

1. La difficoltà pratica di mantenere una posizione di rete sufficientemente vantaggiosa per garantire il successo nelle race condition.
2. I rischi reputazionali associati all'adozione di comportamenti evidentemente malevoli all'interno della comunità.
3. Considerazioni di lungo termine sulla stabilità e sul valore dell'ecosistema, che potrebbero dissuadere i grandi minatori dall'adottare strategie potenzialmente destabilizzanti.
4. La presenza di contromisure implementate nel protocollo o adottate dalla comunità per mitigare l'efficacia di tali attacchi.

La discrepanza tra la vulnerabilità teorica e la stabilità pratica del sistema solleva interessanti questioni sulla robustezza emergente dei sistemi decentralizzati e suggerisce che fattori socio-economici più ampi possano contribuire alla sicurezza complessiva del protocollo, oltre alle pure garanzie crittografiche e teoriche.

Chapter 6

Lezione 6

6.1 Fondamenti dell'Anonimato

L'anonimato rappresenta un aspetto controverso nel contesto delle criptovalute, in particolare per quanto riguarda Bitcoin. Da un lato, fonti come la pagina delle donazioni di WikiLeaks affermano che "Bitcoin è una valuta digitale sicura e anonima"; dall'altro, pubblicazioni come Wired UK sostengono che "Bitcoin non ti nasconderà dagli occhi indiscreti della NSA". Questa divergenza di opinioni evidenzia la necessità di definire rigorosamente cosa si intenda per anonimato nel contesto delle criptovalute.

6.1.1 Definizione di Anonimato

Il termine "anonimo", nella sua accezione letterale, significa "senza nome". Questa definizione può essere interpretata in due modi distinti nel contesto delle transazioni digitali:

- La possibilità di interagire senza utilizzare il proprio nome reale.
- La possibilità di interagire senza utilizzare alcun identificativo persistente.

Nel caso di Bitcoin, gli indirizzi utilizzati sono hash di chiavi pubbliche piuttosto che identità reali. Secondo la prima interpretazione, Bitcoin potrebbe essere considerato anonimo; tuttavia, secondo la seconda interpretazione, non lo è, poiché ogni transazione è associata a un indirizzo specifico che funge da pseudonimo persistente. Per questo motivo, gli informatici definiscono Bitcoin come un sistema **pseudonimo** piuttosto che anonimo.

6.1.2 Pseudonimia e Unlinkability

Nella scienza informatica, l'anonimato completo richiede due componenti: la pseudonimia e l'unlinkability (non collegabilità). Quest'ultima proprietà implica che diverse interazioni dello stesso utente con il sistema non dovrebbero essere riconducibili tra loro. La necessità dell'unlinkability deriva dal fatto che numerosi servizi Bitcoin richiedono l'identità reale degli utenti, e i profili collegati possono essere de-anonimizzati attraverso vari canali secondari di informazione.

Nel contesto specifico di Bitcoin, l'unlinkability si declina in tre proprietà fondamentali:

- La difficoltà di collegare diversi indirizzi appartenenti allo stesso utente.
- La difficoltà di collegare diverse transazioni effettuate dallo stesso utente.
- La difficoltà di collegare il mittente di un pagamento al suo destinatario.

Le prime due proprietà risultano intuitive, mentre la terza richiede una precisazione: non si riferisce alla singola transazione, in cui il collegamento è esplicito, ma al processo complessivo di trasferimento di Bitcoin dal mittente al destinatario, che potrebbe coinvolgere molteplici transazioni intermedie.

6.1.3 Quantificazione dell'Anonimato

L'unlinkability completa tra tutti gli indirizzi e le transazioni rappresenta un obiettivo difficilmente raggiungibile. Per quantificare il grado di anonimato, si introduce il concetto di *anonymity set*, ovvero l'insieme delle transazioni che un avversario non può distinguere dalle transazioni dell'utente target. Per calcolare questo insieme è necessario:

- Definire un modello di avversario con capacità e limitazioni specifiche.
- Analizzare attentamente cosa l'avversario conosce, non conosce e non può conoscere.

6.1.4 Motivazioni per Criptovalute Anonime

Le blockchain sono registri pubblici, permanenti e completamente tracciabili. Senza meccanismi di anonimato, la privacy degli utenti risulterebbe significativamente compromessa, persino in misura maggiore rispetto ai sistemi bancari tradizionali. L'anonimato nelle criptovalute risponde quindi a legittime esigenze di privacy:

- Individui che non desiderano condividere informazioni sensibili come il proprio stipendio.
- Organizzazioni con significative preoccupazioni relative alla riservatezza delle proprie operazioni finanziarie.

Tuttavia, l'anonimato solleva anche preoccupazioni legittime riguardo a potenziali utilizzi illeciti, come il riciclaggio di denaro. È importante notare che l'interfaccia tra valute digitali e valute fiat (tradizionali) non è anonima, ma anzi fortemente regolamentata, il che limita parzialmente questi rischi.

6.1.5 Il Dilemma Etico-Tecnologico

L'anonimato nelle criptovalute presenta un dilemma comune nella sicurezza informatica e nella privacy: usi moralmente molto diversi possono risultare tecnologicamente quasi indistinguibili. Un esempio analogo è rappresentato dalla rete Tor, che offre comunicazioni anonime utilizzate sia da giornalisti, attivisti e forze dell'ordine, sia da attori malintenzionati. Nonostante questo rischio, Tor è finanziato da varie entità, incluso il Dipartimento di Stato degli Stati Uniti, riconoscendo il valore complessivo di tali tecnologie.

6.2 Storia del Contante Elettronico Anonimo

6.2.1 Le Blind Signatures di David Chaum

Il concetto di contante elettronico anonimo risale agli anni '80, quando David Chaum introdusse le "blind signatures" (firme cieche). Questo meccanismo crittografico consente a una parte di firmare digitalmente un messaggio senza conoscerne il contenuto, analogamente a come si potrebbe firmare una busta sigillata senza vedere il documento all'interno.

6.3 Storia del Contante Elettronico Anonimo

Il concetto chiave alla base dell'ecash di Chaum è rappresentato dalle "blind signatures" (firme cieche), un meccanismo crittografico che consente a una parte di firmare digitalmente un messaggio senza conoscerne il contenuto, analogamente a come si potrebbe firmare una busta sigillata senza vedere il documento all'interno. Questo meccanismo consente di separare l'identità dell'utente dalla sua attività finanziaria, proteggendo la privacy dell'individuo.

Il funzionamento del sistema può essere concettualizzato attraverso l'analogia di una bacheca pubblica condivisa. Quando un utente desidera effettuare una transazione anonima, "affigge" un messaggio sigillato sulla bacheca: tutti possono vedere che il messaggio esiste, ma nessuno può conoscerne il contenuto. Successivamente, per recuperare i fondi, l'utente presenta una prova del proprio diritto a reclamare un messaggio, senza rivelare quale specifico messaggio stia reclamando.

Il processo operativo del sistema di Chaum si articola in tre fasi distinte che garantiscono l'anonimato delle transazioni attraverso l'implementazione delle blind signatures:

- **Fase di Prelievo (Withdrawal)**

Durante la fase iniziale, l'utente Alice genera un numero seriale casuale S che rappresenterà univocamente la moneta elettronica. Questo numero seriale viene successivamente "mascherato" attraverso un fattore di blinding r scelto casualmente, producendo un commitment $M = f(S, r)$ dove f rappresenta la funzione di blinding crittografico. Alice trasmette questo commitment mascherato alla banca insieme alla richiesta di prelievo per un determinato importo.

La banca, ricevendo il commitment M , non può determinare il valore del numero seriale S sottostante a causa delle proprietà crittografiche del blinding. Tuttavia, può verificare la validità della richiesta di prelievo controllando l'identità di Alice e la disponibilità di fondi nel suo conto. Una volta confermata la validità dell'operazione, la banca applica la propria firma digitale al commitment mascherato, generando $\text{Sign}_{\text{bank}}(M)$, e trasmette questa firma cieca ad Alice insieme alla detrazione dell'importo corrispondente dal suo conto.

- **Fase di Unblinding**

Alice, ricevendo la firma cieca $\text{Sign}_{\text{bank}}(M)$, applica la funzione di unblinding utilizzando il fattore r precedentemente generato. Questa operazione produce una firma valida $\text{Sign}_{\text{bank}}(S)$ applicata direttamente al numero seriale S , senza che la

banca abbia mai avuto conoscenza di tale valore. La proprietà fondamentale delle blind signatures garantisce che questa firma sia matematicamente valida e verificabile dalla banca, nonostante quest'ultima non abbia mai "visto" il numero seriale durante il processo di firma.

• Fase di Deposito (Deposit)

Quando Alice desidera spendere la moneta elettronica, presenta il numero seriale S insieme alla sua firma $\text{Sign}_{\text{bank}}(S)$ a un commerciante Bob. Quest'ultimo può verificare la validità della firma utilizzando la chiave pubblica della banca, confermando l'autenticità della moneta elettronica. Per completare la transazione, Bob trasmette la coppia $(S, \text{Sign}_{\text{bank}}(S))$ alla banca per il deposito.

La banca riceve il numero seriale S e verifica che la firma sia autentica. Successivamente, controlla che S non sia già presente nel database dei numeri seriali spesi, prevenendo così il problema della doppia spesa. Se il numero seriale risulta nuovo, la banca lo registra come utilizzato e accredita l'importo corrispondente sul conto di Bob.

La caratteristica fondamentale del sistema è che, durante il processo di deposito, la banca non può determinare quale utente abbia originariamente prelevato la moneta elettronica. Questo perché, durante la fase di emissione, la banca ha firmato un commitment e non il numero seriale diretto, e il fattore di blinding impedisce di correlare il commitment al numero seriale finale. Questa proprietà garantisce l'anonimato del pagatore, anche in un contesto in cui la banca funge da autorità centrale.

Questo sistema permette di implementare contanti elettronici anonimi, ma richiede un'autorità centrale fidata (come una banca) per gestire la contabilità e prevenire il double spending. Nel processo di prelievo, l'utente riceve una moneta con numero seriale firmato dalla banca. Durante il deposito, quando la moneta viene presentata, la banca registra il numero seriale come speso, ma non può collegare il prelievo al deposito, garantendo così l'anonimato.

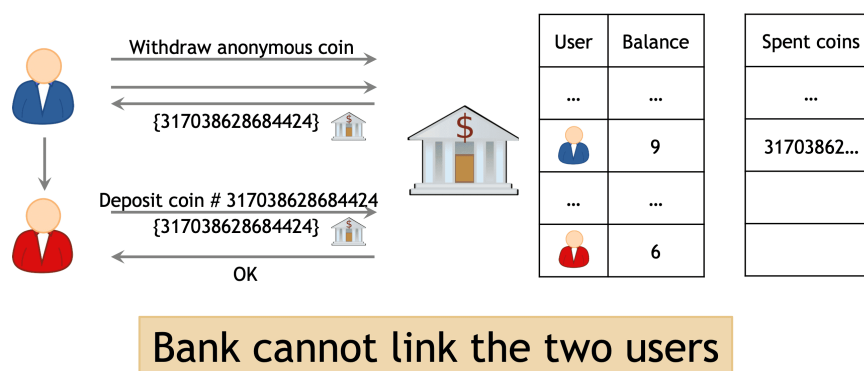


Figure 6.1: Contanti elettronici anonimi tramite blind signatures. La banca non può collegare le operazioni di prelievo e deposito, mantenendo così l'anonimato dell'utente.

6.3.1 Il Conflitto tra Anonimato e Decentralizzazione

I protocolli interattivi che coinvolgono un'autorità centrale come una banca sono difficili da decentralizzare. La decentralizzazione viene spesso ottenuta attraverso la tracciabilità

pubblica per garantire la sicurezza del sistema, creando una tensione fondamentale con l'obiettivo dell'anonimato. Questa tensione rappresenta una delle sfide principali nella progettazione di cryptocurrency che aspirano ad essere sia decentralizzate che anonime.

6.4 Tecniche di De-anonimizzazione in Bitcoin

Nonostante la percezione comune che Bitcoin offra anonimato, esistono numerose tecniche che possono compromettere questa proprietà. La facilità con cui si possono creare nuovi indirizzi e la best practice di utilizzare sempre indirizzi freschi per ricevere pagamenti suggeriscono un certo grado di unlinkability. Tuttavia, un'analisi più approfondita rivela vulnerabilità significative.

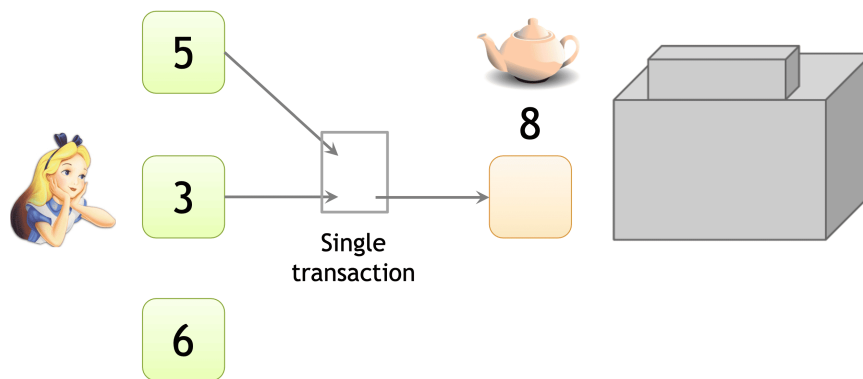


Figure 6.2: Esempio di transazione di Alice: l'acquisto di una teiera in un grande magazzino con una singola transazione Bitcoin.

6.4.1 Clustering degli Indirizzi

Una delle tecniche principali per compromettere l'anonimato in Bitcoin è il clustering degli indirizzi. Questo approccio si basa su due principi fondamentali:

- **Spesa condivisa (Shared spending):** Quando più indirizzi contribuiscono come input a una singola transazione, questo costituisce un'evidenza di controllo congiunto. In altre parole, se due o più indirizzi vengono utilizzati come input nella stessa transazione, è ragionevole supporre che appartengano allo stesso utente.
- **Transitività:** Gli indirizzi possono essere collegati transitivamente, permettendo di identificare interi cluster di transazioni riconducibili a un singolo utente. Ad esempio se l'indirizzo A è collegato all'indirizzo B (tramite una transazione condivisa) e l'indirizzo B è collegato ad un terzo indirizzo C allora anche l'indirizzo A è presumibilmente collegato all'indirizzo C.

Questa metodologia è stata formalizzata nel 2011 da Reid e Harrigan nel loro studio "An Analysis of Anonymity in the Bitcoin System", presentato alla conferenza PASSAT.

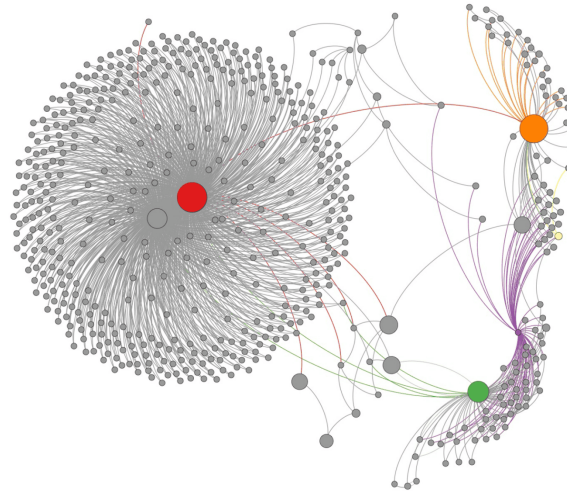


Figure 6.3: Clustering degli indirizzi: visualizzazione grafica di come gli indirizzi Bitcoin possono essere raggruppati attraverso l'analisi delle transazioni condivise.

6.4.2 Indirizzi di Resto e "Idioms of Use"

Un'altra vulnerabilità deriva dalle specifiche implementazioni dei wallet Bitcoin. Quando un utente effettua una transazione per un importo inferiore al totale disponibile, il sistema genera automaticamente una transazione di "resto" (change) verso un nuovo indirizzo. La sfida per l'analista è determinare quale degli output rappresenti il pagamento effettivo e quale il resto.

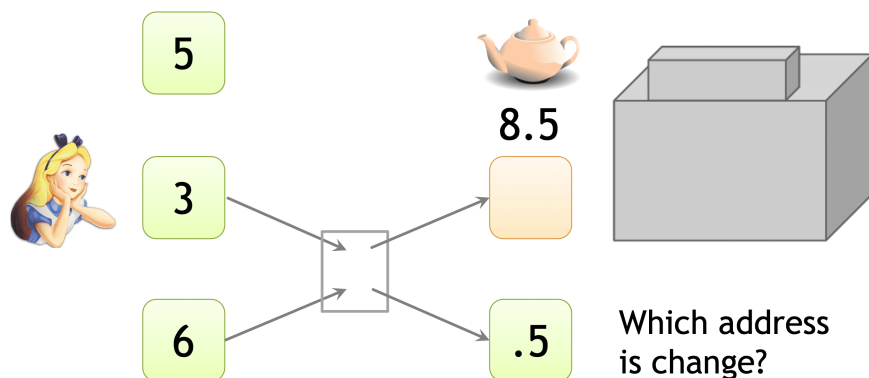


Figure 6.4: Problema dell'identificazione dell'indirizzo di resto: nel caso di Alice, quale dei due output della transazione rappresenta il change address? (Indirizzo di resto)

Le caratteristiche idiosincratice del software dei wallet, definite "idioms of use", possono essere sfruttate per questa distinzione. Ad esempio, alcuni wallet utilizzano ciascun indirizzo solo una volta come indirizzo di resto, creando pattern riconoscibili. Lo studio "A Fistful of Bitcoins: Characterizing Payments Among Men with No Names" di Meiklejohn et al. (2013) ha dimostrato come la combinazione di spesa condivisa e idioms of use permetta di identificare cluster significativi di indirizzi.

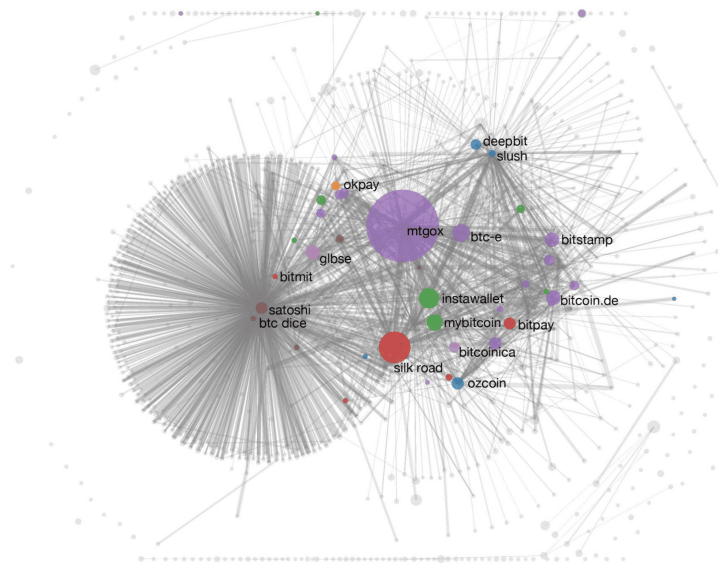


Figure 6.5: Clustering avanzato degli indirizzi: combinazione delle tecniche di shared spending e idioms of use per identificare gruppi di indirizzi appartenenti allo stesso utente.

6.4.3 Attribuzione di Identità ai Cluster

Una volta identificati i cluster di indirizzi, è possibile formulare ipotesi educate sulla loro appartenenza. Ad esempio, nel 2013:

- Cluster con numerose transazioni di piccolo valore potevano corrispondere a servizi di gioco d'azzardo come Satoshi Dice.
- Il cluster più grande apparteneva probabilmente a Mt. Gox, all'epoca il maggiore exchange di Bitcoin.

Per verificare queste ipotesi, è sufficiente interagire direttamente con il servizio e osservare i pattern di transazione.

6.4.4 Identificazione degli Individui

L'identificazione degli individui dietro le transazioni Bitcoin può avvenire attraverso diverse modalità:

1. **Transazioni dirette:** L'interazione diretta con un individuo può rivelare il suo indirizzo Bitcoin.
2. **Provider di servizi:** Molti servizi richiedono l'identità reale degli utenti per conformità normativa.
3. **Disattenzione:** Gli utenti spesso pubblicano i propri indirizzi Bitcoin su forum pubblici, ad esempio per ricevere donazioni.
4. **Evoluzione delle tecniche:** Con il tempo, i ricercatori identificano nuove tecniche di de-anonimizzazione sempre più sofisticate.

6.4.5 De-anonimizzazione a Livello di Rete

Un'ulteriore vulnerabilità riguarda l'architettura di rete di Bitcoin. Come evidenziato da Dan Kaminsky nella sua presentazione a Black Hat 2011, "il primo nodo a informarti di una transazione è probabilmente la fonte di essa". Questo principio permette di collegare indirizzi IP a transazioni specifiche, compromettendo ulteriormente l'anonimato.

Questa vulnerabilità deriva dal meccanismo di diffusione delle transazioni in Bitcoin, dove ogni nuova transazione generata viene inizialmente trasmessa ai nodi peer a cui il client dell'utente è direttamente connesso, per poi propagarsi attraverso la rete secondo un modello a cascata, con ogni nodo che inoltra l'informazione ai propri peer. Un attaccante che controlla strategicamente molteplici nodi distribuiti nella rete può monitorare l'ordine e i tempi di propagazione delle transazioni e, attraverso tecniche di triangolazione basate sui tempi di latenza, identificare con buona probabilità l'indirizzo IP originario della transazione.



Figure 6.6: Network layer de-anonymization: illustrazione di come i nodi nella rete Bitcoin possano essere utilizzati per identificare l'origine delle transazioni.

Questo metodo di de-anonimizzazione è particolarmente insidioso perché opera a un livello diverso rispetto alle protezioni implementate sulla blockchain, permettendo di correlare direttamente indirizzi IP (potenzialmente riconducibili a identità reali) con specifici indirizzi Bitcoin.

6.5 Soluzioni per Migliorare l'Anonimato: Tor

6.5.1 Utilizzo di Tor per le Transazioni Bitcoin

Una soluzione parziale ai problemi di de-anonimizzazione a livello di rete consiste nell'utilizzo di Tor, una rete di comunicazione anonima. È importante notare che Tor è progettato principalmente per attività a bassa latenza come la navigazione web, e reti di mixing potrebbero fornire un anonimato superiore. Tuttavia, Tor rappresenta una soluzione già implementata e funzionante.

6.5.2 Funzionamento di Tor

Tor opera secondo un modello di minaccia specifico e utilizza il principio della cifratura a strati. Il sistema è considerato relativamente sicuro se almeno uno dei router nel percorso è onesto. La sfida principale consiste nel nascondere le informazioni di routing, il che viene ottenuto attraverso la cifratura stratificata.

Alice deve comunicare con Bob utilizzando delle chiavi di cifratura valide per la sessione corrente per cifrare un messaggio n volte, dove n è il numero di macchine intermedie (partecipanti alla rete Tor) per cui passa il messaggio. Ogni macchina intermedia applica una chiave di cifratura per togliere uno strato di cifratura al messaggio fino all'ultima macchina del path prestabilito da Alice. A quel punto il messaggio, ormai in chiaro, passa dall'ultima macchina ad Alice.

Un effetto collaterale di questo approccio è che i contenuti risultano cifrati dalla sorgente (Alice) fino al nodo di uscita, ma non necessariamente dal nodo di uscita alla destinazione finale (Bob). Questo aspetto deve essere considerato quando si valuta il livello complessivo di sicurezza e privacy offerto dalla combinazione di Bitcoin e Tor.

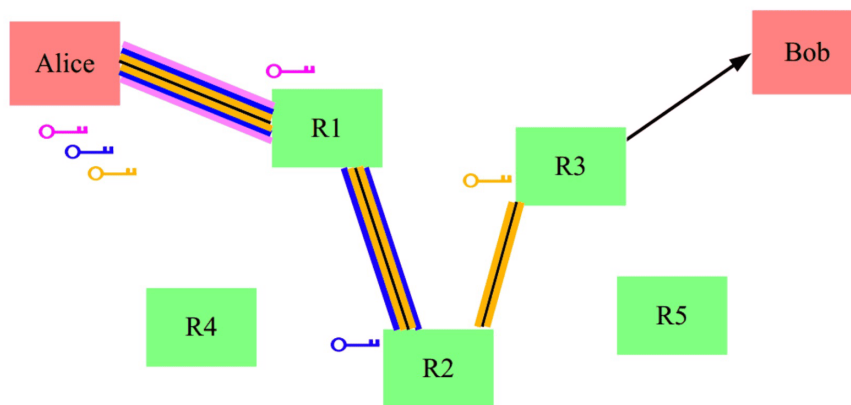


Figure 6.7: Cifratura a strati in Tor: il messaggio viene criptato in più livelli, ciascuno dei quali può essere decifrato solo dal nodo corrispondente nel percorso.

Si osservi che se l'avversario controlla tutte le macchine per cui passa il messaggio cifrato avrebbe informazioni su mittente, destinatario e conoscerebbe anche il messaggio quindi non si ha nessuna cifratura. Ma affinché valga l'unlinkability è sufficiente che ci sia almeno una macchina non controllata dall'avversario.

Chapter 7

Lezione 7

7.1 Mixing e Anonimato nelle Criptovalute

Il mixing rappresenta una tecnica fondamentale per migliorare l'anonimato nelle transazioni di criptovalute, operando come meccanismo di dissociazione tra indirizzi di input e output. Questa metodologia si configura come risposta alle vulnerabilità di de-anonimizzazione precedentemente analizzate, in particolare al clustering degli indirizzi e all'analisi dei flussi transazionali. Il principio fondamentale del mixing consiste nell'utilizzo di intermediari che interrompono la tracciabilità diretta delle transazioni, creando un livello di separazione tra mittente e destinatario.

7.1.1 Online Wallets come Meccanismi di Mixing Implicito

I wallet online rappresentano una prima forma di mixing, sebbene questo non sia generalmente il loro scopo primario. Questi servizi gestiscono collettivamente i fondi di numerosi utenti, facilitando implicitamente un certo grado di commistione tra i depositi. Quando un utente trasferisce Bitcoin a un wallet online e successivamente li preleva, tipicamente riceve unità di valuta diverse da quelle originariamente depositate. Questo processo introduce un elemento di dissociazione che rende più complessa l'analisi della provenienza dei fondi. Tuttavia, l'efficacia dei wallet online come strumenti di mixing presenta significative limitazioni:

1. **Assenza di garanzie esplicite:** La maggior parte dei wallet online non si impegna formalmente a implementare pratiche di mixing robuste. L'effetto di mixing deriva principalmente da procedure operative interne finalizzate all'ottimizzazione della gestione dei fondi, piuttosto che da un impegno esplicito verso la tutela della privacy degli utenti. Questa caratteristica potrebbe subire modifiche in risposta a pressioni normative o a cambiamenti nelle politiche di gestione.
2. **Conservazione dei registri:** Anche ipotizzando un efficace processo di mixing interno, i wallet online generalmente mantengono registri dettagliati delle transazioni effettuate dagli utenti. Questa pratica è adottata sia per ragioni di convenienza operativa, facilitando la risoluzione di controversie e la gestione del servizio, sia per ottemperare a requisiti normativi sempre più stringenti in materia di antiriciclaggio e contrasto al finanziamento del terrorismo.

La natura dei wallet online come entità commerciali regolamentate implica ulteriori considerazioni:

- **Requisiti di identificazione:** I wallet online di maggiore reputazione sono generalmente sottoposti a regolamentazione e richiedono la verifica dell'identità degli utenti (procedure KYC, Know Your Customer). Questo aspetto compromette intrinsecamente l'anonimato rispetto al provider del servizio, poiché stabilisce un collegamento diretto tra l'identità reale dell'utente e le sue transazioni.
- **Custodia prolungata dei fondi:** Gli utenti affidano i propri Bitcoin al wallet online per periodi potenzialmente estesi, il che amplifica l'efficacia del mixing rispetto a osservatori esterni. La permanenza dei fondi all'interno del sistema per intervalli significativi aumenta la dimensione dell'anonymity set, rendendo più complessa l'analisi dei flussi di entrata e uscita.

7.1.2 Servizi di Mixing Dedicati

In risposta alle limitazioni dei wallet online in termini di privacy, sono emersi servizi di mixing dedicati che si concentrano specificamente sull'offerta di transazioni anonime. Questi servizi si differenziano dai wallet online convenzionali per due caratteristiche fondamentali:

- **Impegno a non conservare registri:** I servizi di mixing dedicati dichiarano esplicitamente di non mantenere archivi delle transazioni processate, eliminando teoricamente la possibilità di ricostruire i collegamenti tra indirizzi di input e output. Questa caratteristica costituisce un vantaggio significativo rispetto ai wallet online, che tipicamente conservano dati dettagliati per ragioni operative e normative.
- **Assenza di requisiti di identificazione:** A differenza dei wallet regolamentati, i mixing service non richiedono la verifica dell'identità degli utenti, preservando l'anonimato anche rispetto al provider del servizio stesso.

L'analisi che segue si concentra sulle necessità degli utenti per i quali i requisiti di fiducia e le proprietà di anonimato offerte dai wallet online risultano insufficienti, e che pertanto potrebbero considerare l'utilizzo di servizi di mixing dedicati.

7.2 Principi di Progettazione per Servizi di Mixing Efficienti

La ricerca accademica, in particolare il lavoro "Mixcoin: Anonymity for Bitcoin with Accountable Mixes" presentato alla conferenza Financial Cryptography 2014 da J. Bonneau et al., ha identificato quattro principi fondamentali per la progettazione di servizi di mixing efficaci e robusti.

7.2.1 Utilizzo di una Serie di Mix

Il primo principio raccomanda l'implementazione di una sequenza di operazioni di mixing successive, piuttosto che affidarsi a un singolo intermediario. Questa architettura distribuita presenta vantaggi significativi:

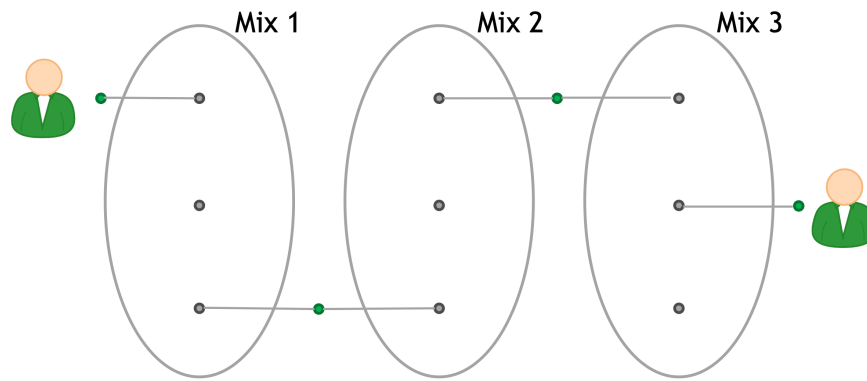


Figure 7.1: Rappresentazione schematica di una serie di mix in sequenza. Ogni passaggio incrementa l'anonimato rendendo più complessa la tracciabilità delle transazioni.

- **Riduzione del rischio di compromissione:** L'utilizzo di molteplici mix riduce drasticamente la vulnerabilità complessiva del sistema. Affinché la catena di anonimizzazione sia compromessa, tutti i mix coinvolti dovrebbero essere simultaneamente compromessi o collusi, evento la cui probabilità diminuisce esponenzialmente con il numero di mix indipendenti utilizzati.
- **Incremento dell'anonymity set:** Ogni passaggio attraverso un mix diverso espande l'insieme delle transazioni con cui i fondi dell'utente vengono mescolati, aumentando progressivamente la complessità dell'analisi necessaria per de-anonimizzare le transazioni.

Per facilitare l'implementazione di questo principio, Mixcoin propone l'adozione di un'API standardizzata che consenta l'interoperabilità tra diversi servizi di mixing, semplificando la creazione di catene di mix eterogenee.

7.2.2 Uniformità delle Transazioni

Il secondo principio enfatizza l'importanza della standardizzazione delle caratteristiche transazionali, in particolare dell'importo. Tutte le transazioni processate da un servizio di mixing dovrebbero presentare un valore identico, definito come "chunk size":

- **Prevenzione dell'analisi per valore:** La standardizzazione degli importi impedisce l'utilizzo del valore transazionale come elemento distintivo per correlare input e output. Se tutte le transazioni presentano lo stesso valore, diviene impossibile identificare corrispondenze basate su questo parametro.
- **Resistenza all'analisi temporale:** La normalizzazione riduce anche la vulnerabilità all'analisi temporale, poiché transazioni di uguale valore sono più difficilmente distinguibili sulla base dei pattern temporali di ingresso e uscita.

Questo principio implica che gli utenti devono suddividere preventivamente i propri fondi in "chunk" standardizzati prima di sottoporli al processo di mixing, eventualmente ricomponendoli successivamente in unità di valore diverso.

7.2.3 Automazione del Lato Client

Il terzo principio sottolinea l'importanza dell'automazione delle operazioni lato client, preferibilmente attraverso l'integrazione nei software di wallet desktop:

- **Riduzione dell'errore umano:** L'automazione minimizza la possibilità di errori operativi che potrebbero compromettere l'anonimato, come l'utilizzo inconsistente di tecniche di privacy o la gestione impropria degli indirizzi.
- **Standardizzazione dei comportamenti:** Il software automatizzato garantisce comportamenti coerenti tra utenti diversi, riducendo le idiosincrasie comportamentali che potrebbero essere utilizzate per l'identificazione.
- **Accessibilità:** L'integrazione nei wallet desktop rende le tecniche di mixing accessibili anche a utenti con limitate competenze tecniche, democratizzando l'accesso a strumenti di privacy avanzati.

L'implementazione di questo principio richiede lo sviluppo di software client che gestiscano autonomamente la complessità delle operazioni di mixing, offrendo un'interfaccia semplificata che nasconda i dettagli tecnici sottostanti.

7.2.4 Commissioni con Meccanismo All-or-Nothing

Il quarto principio propone un modello innovativo per la gestione delle commissioni dei servizi di mixing, basato su un meccanismo probabilistico piuttosto che su detrazioni proporzionali:

- **Commissioni probabilistiche:** Anziché applicare una detrazione percentuale su ogni transazione (es. 0,1% del valore), il mix trattiene l'intero importo ("chunk") con una probabilità equivalente (es. 0,1% di probabilità).
- **Preservazione dell'uniformità:** Questo approccio mantiene l'uniformità degli importi transazionali, evitando che le commissioni creino valori "dispari" che potrebbero essere utilizzati per tracciare i flussi.
- **Protezione dell'anonimato:** Una commissione proporzionale creerebbe un valore unico e potenzialmente identificabile, compromettendo l'anonimato dell'utente. Il modello probabilistico conserva invece l'indistinguibilità tra le transazioni.

Questo meccanismo di compensazione garantisce che tutte le transazioni in uscita mantengano lo stesso valore standardizzato di quelle in entrata, eliminando un potenziale vettore di correlazione.

7.3 Stato Attuale dei Servizi di Mixing

Nonostante la chiara articolazione di questi principi nella letteratura accademica, l'ecosistema dei servizi di mixing presenta significative discrepanze rispetto alle best practice teoriche. Al 2015 (data di riferimento nei documenti analizzati), la situazione poteva essere così sintetizzata:

- **Mancata implementazione dei principi:** I servizi di mixing allora disponibili non implementavano adeguatamente nessuno dei quattro principi fondamentali de-

lineati dalla ricerca, compromettendo potenzialmente l'efficacia dell'anonimizzazione offerta.

- **Assenza di servizi di reputazione consolidata:** Non esistevano servizi di mixing che godessero di una reputazione unanimemente riconosciuta in termini di affidabilità e sicurezza.
- **Rischio operativo significativo:** Come evidenziato dal disclaimer presente sul Bitcoin Wiki, i servizi di mixing operavano spesso essi stessi in regime di anonimato, eliminando qualsiasi possibilità di ricorso in caso di mancata consegna dei fondi o di negazione dell'accesso. Questa caratteristica introduceva un rischio significativo per gli utenti, che dovevano essenzialmente fidarsi di entità anonime per la gestione dei propri asset.

Il Bitcoin Wiki stesso raccomandava cautela con questa dichiarazione: "Attenzione: i servizi di mixing potrebbero essi stessi operare in anonimato. Pertanto, in caso di mancata consegna dell'output del mixing o di negazione dell'accesso ai fondi, non esiste alcun meccanismo di ricorso. Utilizzate questi servizi a vostro rischio e discrezione."

7.4 Mixing Decentralizzato

Le limitazioni intrinseche dei servizi di mixing centralizzati hanno stimolato lo sviluppo di approcci decentralizzati, che mirano a raggiungere livelli comparabili o superiori di anonimato senza richiedere la fiducia in un intermediario centrale. Questa evoluzione rappresenta un significativo progresso concettuale, allineandosi maggiormente con la filosofia decentralizzata che caratterizza la blockchain di Bitcoin.

7.4.1 Motivazioni per il Mixing Decentralizzato

L'adozione di protocolli di mixing decentralizzati è motivata da molteplici fattori che ne evidenziano i vantaggi rispetto alle controparti centralizzate:

- **Indipendenza da provider affidabili:** I protocolli decentralizzati eliminano la necessità di attendere l'emergere di servizi centralizzati di comprovata reputazione, un processo che si è rivelato problematico nell'ecosistema Bitcoin.
- **Eliminazione del rischio di furto:** Poiché gli utenti mantengono il controllo dei propri fondi durante l'intero processo di mixing, la possibilità di sottrazione fraudolenta da parte dell'intermediario viene strutturalmente eliminata.
- **Potenziale per un anonimato superiore:** I protocolli decentralizzati possono teoricamente offrire garanzie di privacy più robuste, implementando meccanismi crittografici avanzati che limitano la quantità di informazioni rivelate a qualsiasi partecipante.
- **Coerenza filosofica:** L'approccio decentralizzato risulta maggiormente allineato con i principi fondanti di Bitcoin, in particolare con l'ideale di disintermediazione e di resistenza alla censura.

Queste caratteristiche hanno catalizzato lo sviluppo di diversi protocolli di mixing decentralizzati, tra cui CoinJoin rappresenta uno degli approcci più significativi e concettualmente eleganti.

7.4.2 CoinJoin: Principi e Funzionamento

CoinJoin, proposto da Greg Maxwell (sviluppatore core di Bitcoin), rappresenta un protocollo di mixing decentralizzato che sfrutta una proprietà fondamentale delle transazioni Bitcoin: la possibilità di aggregare multiple firme indipendenti in un'unica transazione. Questo approccio consente a più utenti di collaborare alla creazione di una transazione congiunta senza cedere il controllo dei propri fondi.

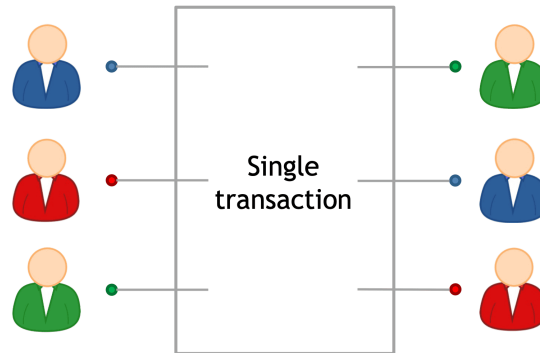


Figure 7.2: Schema di funzionamento del protocollo CoinJoin. Più partecipanti collaborano per creare una singola transazione con multiple input e output, rendendo difficile la correlazione tra mittenti e destinatari.

Il principio fondamentale di CoinJoin consiste nella creazione di una singola transazione che combina input provenienti da diversi utenti e genera output diretti verso nuovi indirizzi controllati dai partecipanti. La separazione tra le firme digitali di ciascun partecipante garantisce che ogni utente mantenga il controllo esclusivo sui propri fondi, eliminando il rischio di furto. Questa struttura può essere interpretata come un "round" di mixing, e può essere reiterata secondo i principi delineati precedentemente per incrementare l'efficacia dell'anonimizzazione.

7.4.2.1 Algoritmo di CoinJoin

L'obiettivo di CoinJoin è rompere il legame deterministico tra input e output di una transazione Bitcoin, aumentando la privacy dei partecipanti. Si articola tipicamente nelle seguenti fasi:

1. **Individuazione dei Peer e Registrazione al Round:** I partecipanti interessati a un round di mixing per un **importo specifico** (es. 0.1 BTC) devono trovarsi. Comunemente, ciò avviene tramite un **server di coordinamento** a cui ci si connette, spesso via **Tor**, per preservare l'anonimato dell'indirizzo IP. I partecipanti si registrano per un round specifico, indicando l'intenzione di mixare un determinato importo, senza ancora rivelare gli UTXO specifici.
2. **Fornitura Input e Output:** Una volta formato il gruppo per il round, i partecipanti comunicano al coordinatore (in modo sicuro e privato):
 - Gli **UTXO (Unspent Transaction Outputs) specifici** che intendono utilizzare come **input**. La somma dei loro valori deve coprire l'importo standardizzato del round più le commissioni.

- Un **indirizzo Bitcoin nuovo** (mai usato prima) come **output** desiderato per ricevere i fondi mixati.
- Eventuali indirizzi per il **resto**, se il valore degli input supera l'importo del round più le fee.

È **fondamentale** che tutti gli output destinati ai partecipanti (non quelli per il resto) abbiano un **valore identico e standardizzato** (l'importo del round), al netto delle fee proporzionali. Questa uniformità è la chiave per l'offuscamento.

3. **Costruzione della Transazione Collaborativa:** Il coordinatore (o un partecipante designato) **assembla** una singola **proposta di transazione Bitcoin non firmata**. Questa include:

- **Tutti gli input** (UTXO) forniti da tutti i partecipanti.
- **Tutti gli output** concordati: quelli di **valore identico** per i partecipanti, gli eventuali output per il **resto**, e gli output per le **commissioni** (di rete e, se prevista, del coordinatore).

Il coordinatore non ha potere di modificare gli input/output a questo punto senza invalidare il processo.

4. **Verifica e Firma Individuale:** La proposta di transazione non firmata viene distribuita a **ciascun partecipante**. **Prima di firmare**, ogni partecipante deve eseguire una **verifica cruciale** lato client:

- Controllare che i propri UTXO di input siano correttamente inclusi.
- Verificare che il proprio indirizzo di output (e l'eventuale indirizzo di resto) sia presente con l'importo esatto concordato.
- Assicurarsi che non vi siano input o output non autorizzati o sospetti.

Solo dopo questa verifica positiva, il partecipante **firma la transazione usando la chiave privata** associata ai propri input. Questa firma **autorizza crittograficamente solo ed esclusivamente lo spostamento dei propri fondi** verso le destinazioni verificate. La firma viene restituita al coordinatore.

5. **Propagazione della Transazione Completa:** Una volta che il coordinatore ha raccolto **tutte le firme valide** necessarie da ogni partecipante (una per ogni input), la transazione è **completa e crittograficamente valida**. A questo punto, il coordinatore (o chiunque abbia la transazione completa) la **trasmette (propaga)** alla rete Bitcoin per l'inclusione nella blockchain.

Questo processo garantisce che **nessun partecipante, né il coordinatore, possa rubare i fondi altrui**, poiché ogni firma autorizza specificamente solo il movimento degli input corrispondenti alla chiave privata utilizzata e solo verso gli output presenti nella transazione verificata dal firmatario. La **sicurezza dei fondi è garantita dal protocollo Bitcoin stesso**.

Dal punto di vista della privacy, la struttura risultante della transazione **rompe il legame deterministico** tra input e output. In particolare, la creazione di molteplici output di **valore identico e standardizzato** rende estremamente difficile per osservatori esterni applicare euristiche di tracciamento comuni (come l'analisi degli importi), aumentando

significativamente l'**anonymity set** (il gruppo di possibili origini/destinazioni indistinguibili) per ogni output mixato.

7.4.3 Problematiche Residue di CoinJoin

Nonostante l'eleganza concettuale di CoinJoin, il protocollo presenta alcune criticità che ne limitano l'efficacia pratica:

- **Individuazione dei peer:** Un primo ostacolo significativo consiste nell'identificazione di altri utenti interessati al mixing, mantenendo contemporaneamente l'anonimato. Questo richiede l'implementazione di meccanismi di coordinamento che non compromettano la privacy.
- **Conoscenza della mappatura input-output:** A differenza dei mix centralizzati, dove teoricamente solo l'intermediario conosce la corrispondenza tra input e output, in CoinJoin i partecipanti possono osservare le associazioni tra gli indirizzi di ciascun utente. Questa caratteristica rappresenta una vulnerabilità più grave rispetto ai sistemi centralizzati, in quanto espande il numero di entità potenzialmente in possesso di informazioni sensibili.
- **Denial of service:** Partecipanti malevoli potrebbero compromettere l'efficacia del protocollo rifiutandosi di firmare la transazione nelle fasi finali, causando un fallimento del processo di mixing dopo che gli altri utenti hanno già investito tempo e risorse.

Queste problematiche hanno stimolato la ricerca di soluzioni specifiche per migliorare la robustezza e l'efficacia dei protocolli di mixing decentralizzato.

7.4.4 Soluzioni per l'Anonimato tra Peer

La protezione dell'anonimato tra i partecipanti al protocollo CoinJoin ha generato diverse proposte metodologiche:

7.4.4.1 Soluzione Elementare

Un approccio semplificato per preservare l'anonimato dei partecipanti prevede:

1. Lo scambio iniziale degli indirizzi di input tra i partecipanti.
2. La disconnessione e la successiva riconnessione attraverso la rete Tor, garantendo che le nuove connessioni non siano associabili alle precedenti.
3. Lo scambio degli indirizzi di output nella nuova sessione, impedendo così il collegamento diretto tra indirizzi di input e output.

Questa metodologia, sebbene concettualmente semplice, presenta limitazioni in termini di efficienza e robustezza.

7.4.4.2 Soluzione Avanzata

Un approccio più sofisticato consiste nell'implementazione di meccanismi di routing anonimo specificamente progettati per il contesto del mixing decentralizzato. Questi sistemi

sfruttano principi crittografici avanzati per garantire che nessun partecipante possa correlare gli indirizzi di input con quelli di output degli altri utenti, preservando l'anonimato reciproco anche in presenza di partecipanti potenzialmente malevoli.

7.4.5 Mitigazione del Denial of Service

Per affrontare la vulnerabilità al denial of service, sono state proposte diverse strategie:

- **Proof of Work:** I partecipanti potrebbero essere tenuti a dimostrare un investimento computazionale prima di unirsi al protocollo, aumentando il costo di comportamenti malevoli.
- **Proof of Burn:** Un approccio più diretto prevede la dimostrazione di un impegno economico attraverso la "distruzione" di una piccola quantità di criptovaluta, creando un incentivo economico alla partecipazione onesta.
- **Esclusione gestita:** In configurazioni che prevedono un server di coordinamento, questo potrebbe identificare e escludere partecipanti che manifestano comportamenti sospetti o non cooperativi.
- **Protocolli di attribuzione della colpa:** Approcci crittografici avanzati come quelli proposti in "CoinShuffle: Practical Decentralized Coin Mixing for Bitcoin" (T. Ruffing et al., PETS 2014) permettono di identificare in modo verificabile i partecipanti che violano il protocollo, senza compromettere l'anonimato del processo complessivo.

Chapter 8

Lezione 8

8.1 Prove Interattive e Zero Knowledge Proofs

Nell'ambito della crittografia moderna, emerge frequentemente uno scenario in cui due entità, che non condividono un rapporto di fiducia reciproca, necessitano di scambiare informazioni riservate in modo sicuro. Un caso paradigmatico è rappresentato dalla necessità di una parte (convenzionalmente denominata Alice) di dimostrare a un'altra parte (Bob) il possesso di determinati requisiti o conoscenze, senza tuttavia rivelare informazioni supplementari che potrebbero compromettere la propria sicurezza o privacy.

Questo problema fondamentale solleva due interrogativi cruciali:

- Quale nozione di "dimostrazione" è appropriata in contesti crittografici?
- Come possiamo formalizzare e verificare che una dimostrazione non divulghi informazioni eccessive?

8.1.1 Il Concetto di Dimostrazione

Una dimostrazione, nel senso più ampio, è un processo che convince un osservatore della validità di un'asserzione. Mentre nella matematica tradizionale questo concetto è tipicamente interpretato in senso statico, attraverso una sequenza di passaggi logici rigorosi che conducono da premesse accettate a una conclusione inevitabile, in altri ambiti applicativi la nozione di dimostrazione può assumere carattere dinamico.

In particolare, nella crittografia moderna, le dimostrazioni possono manifestarsi sotto forma di processi interattivi, in cui due entità distinte scambiano informazioni secondo un protocollo prestabilito. Queste entità sono convenzionalmente denominate:

- **Prover** (P): l'entità che intende dimostrare un'asserzione
- **Verifier** (V): l'entità che valuta l'attendibilità della dimostrazione

Un aspetto particolarmente rilevante nel contesto crittografico è l'asimmetria intrinseca tra dimostrazione e verifica. La verifica di un'asserzione è generalmente più accessibile computazionalmente rispetto alla generazione della dimostrazione stessa. Questa asimmetria è elegantemente formalizzata dalla classe di complessità NP (Non-deterministic Polynomial time), che comprende tutti i problemi per i quali esiste una procedura di

verifica efficiente (eseguibile in tempo polinomiale) per asserzioni del tipo " x appartiene al linguaggio L ", anche se determinare autonomamente l'appartenenza può risultare computazionalmente oneroso.

8.1.2 Requisiti delle Prove Interattive

Affinché un sistema di prove interattive possa essere considerato valido nel contesto crittografico, deve soddisfare due proprietà fondamentali:

- **Completezza (Completeness):** Un prover onesto deve essere in grado di convincere il verifier della validità di asserzioni veritiere con probabilità prossima a 1. Formalmente, per ogni $x \in L$, la probabilità che l'interazione tra P e V porti all'accettazione è uguale a 1.
- **Soundness:** Un prover disonesto non deve essere in grado di convincere il verifier della validità di asserzioni false, se non con probabilità trascurabile. Formalmente, per ogni $x \notin L$ e per qualsiasi strategia potenzialmente fraudolenta B , la probabilità che l'interazione tra B e V porti all'accettazione è limitata superiormente da una costante inferiore a 1 (tipicamente $1/3$).

8.2 La Nozione di Conoscenza e Zero Knowledge

Un requisito aggiuntivo che si rivela particolarmente significativo in molti contesti applicativi è che il verifier non acquisisca alcuna conoscenza sostanziale oltre alla validità dell'asserzione verificata. Questo solleva interrogativi filosofici e tecnici sulla natura della conoscenza e sulle condizioni in cui si può affermare che un'entità ha acquisito nuova conoscenza durante un'interazione.

8.2.1 Acquisizione di Conoscenza

Per formalizzare questa nozione intuitiva, consideriamo uno scenario elementare: Alice comunica un'informazione a Bob. In questo contesto:

- Alice, in quanto emittente dell'informazione, non acquisisce nuova conoscenza.
- Bob, in quanto ricevente, non necessariamente acquisisce conoscenza significativa. Ad esempio, se Alice comunica a Bob che " $5 + 7 = 12$ ", questa informazione potrebbe essere già nota a Bob o direttamente derivabile dalle sue conoscenze matematiche esistenti.

Possiamo quindi stabilire che Bob acquisisce effettivamente nuova conoscenza se e solo se la sua capacità computazionale viene significativamente ampliata dall'interazione con Alice. In altri termini, Bob acquisisce conoscenza se riceve il risultato di una computazione che non era in grado di eseguire autonomamente con le risorse a sua disposizione.

8.2.2 Sistemi di Prove Interattive

Formalmente, un sistema di prove interattive (P, V) per un linguaggio L è definito come una coppia di algoritmi interattivi, dove V è polinomialmente limitato, che soddisfano le seguenti condizioni:

- **Completeness:** $\forall x \in L, \Pr[\langle P, V \rangle(x) = 1] = 1$
- **Soundness:** $\forall x \notin L, \forall B, \Pr[\langle B, V \rangle(x) = 1] \leq \frac{1}{3}$

8.3 Zero Knowledge Proofs

Un sistema di prove interattive (P, V) per un linguaggio L è detto "zero knowledge" (a conoscenza zero) se qualsiasi informazione calcolabile interagendo con P su input $x \in L$ può essere calcolata anche solo a partire da x , senza l'interazione. Intuitivamente, questo implica che il verifier non acquisisce alcuna conoscenza aggiuntiva rispetto alla semplice validità dell'asserzione.

Per formalizzare questa nozione, definiamo *view* come la variabile che rappresenta ciò che il verifier "vede" durante l'esecuzione del protocollo $(P, V)(x)$, comprensiva dei messaggi scambiati tra P e V e dell'eventuale input privato h di V .

8.3.1 Perfect Zero Knowledge

Theorem 8.3.1. *Un sistema di prove interattive (P, V) fornisce Perfect Zero Knowledge (PZK) per il linguaggio L se esiste una macchina probabilistica a tempo polinomiale M tale che, per ogni $x \in L$, per ogni $a > 0$, e per ogni input ausiliario h con $|h| < |x|^a$, le distribuzioni di probabilità di $M(x, h)$ e *view* sono identiche.*

La macchina M è convenzionalmente denominata "simulatore", poiché la sua funzione è simulare l'interazione tra P e V senza effettivamente disporre delle informazioni riservate di P . L'esistenza di un simulatore efficiente dimostra che il verifier non può estrarre informazioni significative dall'interazione, poiché avrebbe potuto generare autonomamente la stessa distribuzione di viste.

Analogamente, si possono definire versioni più rilassate di zero knowledge:

- **Statistical Zero Knowledge (SZK):** Le distribuzioni di $M(x, h)$ e *view* sono statisticamente indistinguibili.
- **Computational Zero Knowledge (CZK):** Le distribuzioni di $M(x, h)$ e *view* sono computazionalmente indistinguibili, ovvero nessun algoritmo polinomiale può distinguerle con probabilità non trascurabile.

8.4 Zero Knowledge Proofs: Esempi e Applicazioni

8.4.1 Isomorfismo tra Grafi

L'isomorfismo tra grafi rappresenta un esempio paradigmatico di problema per il quale è stato sviluppato un protocollo a conoscenza zero efficace. Il problema fondamentale consiste nel determinare se, dati due grafi G_1 e G_2 , ciascuno con n nodi, esista una permutazione dei nodi che trasformi G_1 in G_2 . Questo problema occupa una posizione peculiare nella teoria della complessità computazionale: appartiene alla classe NP, ma

non è stato ancora dimostrato appartenere a P, né è stato provato essere NP-completo, collocandosi in una sorta di “limbo” algoritmico.

La caratteristica che rende l'isomorfismo tra grafi particolarmente adatto alle Zero-Knowledge Proofs è la netta asimmetria tra la difficoltà di trovare una soluzione e quella di verificarla. Individuare una permutazione π tale che $\pi(G_1) = G_2$ risulta computazionalmente oneroso, specialmente per grafi di dimensioni significative. Al contrario, data una permutazione candidata, la verifica della sua correttezza richiede semplicemente l'applicazione della permutazione a G_1 e il confronto del risultato con G_2 , operazione realizzabile in tempo polinomiale.

Il protocollo a conoscenza zero per l'isomorfismo tra grafi si articola nelle seguenti fasi:

1. Il prover P possiede la conoscenza di una permutazione π tale che $\pi(G_1) = G_2$.
2. P genera casualmente una permutazione σ e calcola $H = \sigma(G_1)$, creando così un grafo isomorfo tanto a G_1 quanto a G_2 .
3. P trasmette H a V , senza rivelare alcuna informazione sulle permutazioni σ o π .
4. V seleziona casualmente $b \in \{1, 2\}$ e lo comunica a P , richiedendo implicitamente la dimostrazione dell'isomorfismo tra H e G_b .
5. Se $b = 1$, P trasmette la permutazione τ tale che $\tau(G_1) = H$, che corrisponde a σ .
6. Se $b = 2$, P trasmette la permutazione τ tale che $\tau(G_2) = H$, che corrisponde a $\sigma \circ \pi^{-1}$.
7. V verifica l'effettiva validità di $\tau(G_b) = H$ e accetta la dimostrazione se la verifica ha esito positivo.

L'analisi formale del protocollo rivela che esso soddisfa le proprietà fondamentali richieste a un sistema di prove interattive a conoscenza zero:

Completeness. Un prover onesto, in possesso della permutazione π , può rispondere correttamente a qualsiasi sfida lanciata dal verifier. Se viene richiesto di dimostrare l'isomorfismo con G_1 , il prover fornisce semplicemente σ ; se viene richiesto di dimostrare l'isomorfismo con G_2 , fornisce la composizione $\sigma \circ \pi^{-1}$. In entrambi i casi, il verifier sarà sempre convincibile della validità dell'asserzione.

Soundness. Un prover disonesto, che tenta di affermare falsamente l'isomorfismo tra G_1 e G_2 non isomorfi, affronta un dilemma strategico nella generazione di H :

- Se costruisce H come rimescolamento di G_1 (cioè $H = \sigma(G_1)$), potrà rispondere correttamente solo alla sfida $b = 1$, ma non a $b = 2$.
- Se costruisce H come rimescolamento di G_2 (cioè $H = \sigma(G_2)$), potrà rispondere correttamente solo alla sfida $b = 2$, ma non a $b = 1$.

Poiché la sfida viene lanciata dopo la generazione di H , e poiché il prover non può predire quale valore di b verrà selezionato, la probabilità di successo in un singolo round è limitata a $\frac{1}{2}$. Attraverso la ripetizione sequenziale del protocollo, la probabilità che un prover disonesto riesca a ingannare il verifier si riduce esponenzialmente: dopo k round,

tale probabilità diventa $\frac{1}{2^k}$, rendendo il protocollo arbitrariamente sicuro all'aumentare di k .

Zero-Knowledge. La proprietà più sottile e affascinante del protocollo è la sua capacità di non rivelare alcuna informazione significativa sulla permutazione π . Questo può essere dimostrato formalmente attraverso la costruzione di un simulatore che, senza conoscere π , genera trascrizioni dell'interazione indistinguibili da quelle generate da un'interazione reale tra P e V . Il simulatore opera nel seguente modo:

1. Estrae dall'oracolo il valore b che V intende utilizzare come sfida.
2. Genera casualmente una permutazione τ e calcola $H = \tau(G_b)$.
3. Simula l'invio di H da P a V .
4. Simula la ricezione della sfida b da V .
5. Fornisce τ come risposta alla sfida.

La distribuzione di probabilità delle trascrizioni generate dal simulatore è identica a quella delle trascrizioni generate da un'interazione reale, dimostrando che il protocollo non rivela alcuna informazione sulla permutazione π oltre al fatto che essa esiste.

8.4.2 Il Protocollo Diffie-Hellman

Il protocollo Diffie-Hellman rappresenta un fondamentale metodo crittografico per lo scambio di chiavi su canali non sicuri. La sua sicurezza si basa sulla difficoltà computazionale del problema del logaritmo discreto in gruppi ciclici appropriatamente scelti.

8.4.2.1 Meccanismo Fondamentale del Protocollo

Il protocollo Diffie-Hellman, nella sua formulazione originale, prevede i seguenti passaggi:

1. Definizione di parametri pubblici: un gruppo ciclico G di ordine m nel quale il problema del logaritmo discreto è considerato intrattabile, e un generatore g di G .
2. Alice seleziona casualmente un esponente privato $x \in \mathbb{Z}_m$ e calcola il valore pubblico g^x .
3. Bob seleziona casualmente un esponente privato $y \in \mathbb{Z}_m$ e calcola il valore pubblico g^y .
4. Alice trasmette g^x a Bob attraverso il canale non sicuro.
5. Bob trasmette g^y ad Alice attraverso il canale non sicuro.
6. Alice calcola la chiave condivisa come $(g^y)^x = g^{xy}$.
7. Bob calcola la chiave condivisa come $(g^x)^y = g^{xy}$.

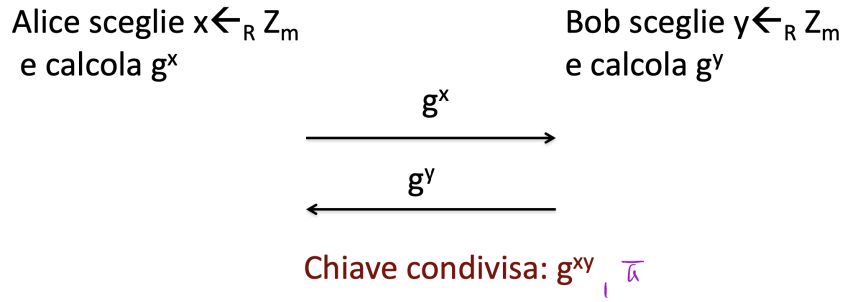


Figure 8.1: Scambio di chiavi nel protocollo Diffie-Hellman: Alice e Bob stabiliscono una chiave segreta condivisa g^{xy} attraverso uno scambio di messaggi su un canale pubblico, sfruttando le proprietà matematiche dell'esponenziazione modulare.

8.4.2.2 Zero Knowledge Proofs nel Contesto Diffie-Hellman

In scenari crittografici avanzati, emerge spesso la necessità di verificare che una determinata quadrupla (g, g^a, g^b, g^c) costituisca una valida quadrupla Diffie-Hellman, ovvero che $g^c = g^{ab}$, senza rivelare gli esponenti segreti a e b . Questa verifica si riduce essenzialmente a dimostrare che $DL_g(g^c) = DL_g(g^a) \cdot DL_g(g^b)$, dove DL_g denota il logaritmo discreto in base g .

Un protocollo a conoscenza zero per questa dimostrazione può essere costruito nel seguente modo:

1. Il prover P conosce i valori segreti a e b tali che $g^c = g^{ab}$.
2. P seleziona casualmente $r_1, r_2 \in \mathbb{Z}_m$ e calcola $t_1 = g^{r_1}$, $t_2 = g^{r_2}$, $t_3 = (g^b)^{r_1} \cdot (g^a)^{r_2} \cdot g^{-r_1 r_2}$.
3. P invia la tripla (t_1, t_2, t_3) a V .
4. V seleziona casualmente un elemento di sfida $e \in \mathbb{Z}_m$ e lo invia a P .
5. P calcola $s_1 = r_1 + e \cdot a \pmod m$ e $s_2 = r_2 + e \cdot b \pmod m$.
6. P invia la coppia (s_1, s_2) a V .
7. V verifica le seguenti equazioni:
 - $g^{s_1} = t_1 \cdot (g^a)^e$
 - $g^{s_2} = t_2 \cdot (g^b)^e$
 - $(g^b)^{s_1} \cdot (g^a)^{s_2} \cdot g^{-s_1 s_2} = t_3 \cdot (g^c)^e$
8. V accetta la dimostrazione se tutte le verifiche hanno esito positivo.

Questo protocollo soddisfa le proprietà fondamentali delle Zero Knowledge Proofs:

Completeness. Un prover onesto, in possesso dei valori a e b tali che $g^c = g^{ab}$, può sempre costruire risposte che soddisfano le equazioni di verifica.

Soundness. Un prover disonesto, che non conosce valori a e b tali che $g^c = g^{ab}$, non può, se non con probabilità trascurabile, produrre risposte che soddisfino contemporaneamente tutte le equazioni di verifica per una sfida arbitraria e .

Zero-Knowledge. È possibile costruire un simulatore che, senza conoscere a e b , generi trascrizioni dell'interazione statisticamente indistinguibili da quelle prodotte da un'interazione reale tra P e V . Il simulatore opera "all'indietro", selezionando prima la sfida e e i valori s_1 e s_2 , e poi calcolando i valori t_1, t_2, t_3 in modo che soddisfino le equazioni di verifica.

8.5 Teorema Fondamentale sulle Zero Knowledge Proofs

Un risultato di fondamentale importanza nel campo delle zero knowledge proofs è il seguente teorema:

Theorem 8.5.1. *Se esistono funzioni unidirezionali, allora ogni linguaggio in NP ammette una zero knowledge proof computazionale.*

Questo teorema, sebbene qui formulato in modo non del tutto rigoroso, rappresenta un risultato di straordinaria generalità e potenza. Esso stabilisce che, sotto l'ipotesi dell'esistenza di funzioni unidirezionali (una congettura ampiamente accettata in crittografia), è possibile costruire zero knowledge proofs per qualsiasi problema in NP.

La portata di questo risultato è difficile da sovrastimare: esso implica che, per qualsiasi problema per il quale esistono soluzioni verificabili efficientemente, è possibile dimostrare il possesso di una soluzione senza rivelare alcuna informazione sulla soluzione stessa, oltre al fatto che essa esiste.

8.6 Prove di Conoscenza

Un concetto correlato, ma distinto, è quello di "prova di conoscenza" (proof of knowledge). In questo contesto, l'obiettivo non è semplicemente dimostrare l'appartenenza di un elemento a un linguaggio, ma dimostrare il possesso effettivo di una "testimonianza" (witness) che certifica tale appartenenza.

Ad esempio, nel caso di un grafo hamiltoniano, una prova di conoscenza richiederebbe non solo di dimostrare che il grafo contiene un ciclo hamiltoniano, ma di dimostrare di conoscere effettivamente tale ciclo.

8.6.1 Formalizzazione della Conoscenza

La definizione formale di "conoscenza" in questo contesto è sottile e richiede l'introduzione del concetto di "estrattore" (extractor). Consideriamo relazioni in NP, ovvero linguaggi L che ammettono una funzione (o relazione) polinomiale ρ tale che, per ogni $x \in L$, esiste y che soddisfa $\rho(x, y) = 1$.

Una prova di conoscenza per una relazione ρ richiede l'esistenza di un algoritmo estrattore E che, dato accesso a x e alla strategia del prover P (attraverso un oracolo), è in grado di estrarre un y tale che $\rho(x, y) = 1$. L'estrattore E può scegliere la randomness utilizzata da P e può interagire con P in modo più flessibile rispetto a un verifier standard, ad esempio costringendo P a utilizzare sempre la stessa randomness in interazioni successive.

Intuitivamente, se E riesce a estrarre y interagendo con P , questo dimostra che P "conosce" effettivamente y , nel senso che la sua strategia incorpora implicitamente tale conoscenza, anche se P potrebbe non essere disposto a rivelare y direttamente.

È importante sottolineare che la formalizzazione completa del concetto di "prova di conoscenza" è significativamente più complessa di quanto esposto qui. Le definizioni rigorose coinvolgono nozioni come l'"estrattore a conoscenza" (knowledge extractor) e il "fattore di conoscenza" (knowledge factor), che specificano la probabilità con cui l'estrattore può estrarre la testimonianza in funzione della probabilità di successo del prover.

Inoltre, come nel caso delle zero knowledge proofs, è possibile definire varianti come le "prove di conoscenza a conoscenza zero" (zero-knowledge proofs of knowledge), che combinano i requisiti di entrambi i concetti: dimostrare la conoscenza di una testimonianza senza rivelare alcuna informazione su di essa.

Chapter 9

Lezione 9

9.1 Zerocoin e Zerocash: Soluzioni per l'Anonimato nelle Criptovalute

L'anonimato nelle transazioni finanziarie rappresenta un diritto fondamentale per molti utenti di criptovalute. Nelle lezioni precedenti abbiamo analizzato come Bitcoin offra solamente pseudonimità, con transazioni potenzialmente tracciabili attraverso tecniche di analisi del grafo e clustering degli indirizzi. Questa vulnerabilità ha stimolato lo sviluppo di soluzioni crittografiche più avanzate.

In questo contesto, Zerocoin e la sua evoluzione Zerocash emergono come protocolli particolarmente significativi, rappresentando non semplici miglioramenti ma vere rivoluzioni concettuali nell'approccio all'anonimato, attraverso l'utilizzo innovativo di primitive crittografiche avanzate.

9.2 Zerocoin: Fondamenti Teorici e Innovazione Concettuale

Zerocoin è stato proposto nel 2013 da ricercatori della Johns Hopkins University come estensione crittografica per Bitcoin. Il protocollo introduce un meccanismo di mixing crittografico fondamentalmente diverso dai mixer tradizionali: anziché affidarsi a intermediari, implementa un mixing basato esclusivamente su primitive crittografiche, eliminando il problema della fiducia negli operatori.

L'innovazione centrale di Zerocoin risiede nell'utilizzo di prove zero knowledge, che consentono di dimostrare la conoscenza di un'informazione senza rivelare l'informazione stessa, interrompendo il legame tra origine e destinazione dei fondi.

9.2.1 Modello Concettuale: La Bacheca Pubblica

Il funzionamento di Zerocoin può essere concettualizzato attraverso l'analogia di una bacheca pubblica condivisa. Quando un utente desidera rendere anonimi i propri fondi, "affigge" un messaggio sigillato sulla bacheca (blockchain): tutti possono vedere che il

messaggio esiste, ma nessuno può conoscerne il contenuto. Successivamente, per recuperare i fondi, l'utente presenta una prova del proprio diritto a reclamare un messaggio, senza rivelare quale specifico messaggio stia reclamando.

Questa metafora cattura l'essenza dell'innovazione: la creazione di uno spazio comune dove le transazioni sono verificabili pubblicamente mantenendo al contempo un livello di privacy che impedisce di tracciare il percorso dei fondi.

9.2.2 Il Processo di Coniazione (Minting): Creazione dell'Anonimato

La coniazione rappresenta la fase iniziale nel ciclo di vita di uno Zerocoin, dove un utente converte un'unità di valuta base in uno Zerocoin.

Il processo si articola in questi passaggi:

1. L'utente genera un numero seriale casuale S che identificherà univocamente la moneta, mantenendolo inizialmente segreto.
2. Calcola un commitment crittografico $C = \text{Com}(S, r)$ usando il numero seriale S e un valore casuale r (randomness). Il commitment funziona come una cassaforte digitale con la proprietà di **hiding** (impossibilità di estrarre S) e **binding** (possibilità di verificare successivamente che C contenga S).
3. Pubblica il commitment C sulla blockchain insieme a un'unità della valuta base tramite una transazione di tipo "Mint".
4. I nodi della rete verificano che il commitment sia correttamente formato e che l'unità di valuta base sia autentica.

Gli Zerocoin vengono emessi in denominazioni standard, creando un "anonymity set" omogeneo che impedisce il tracciamento basato sui valori delle transazioni.

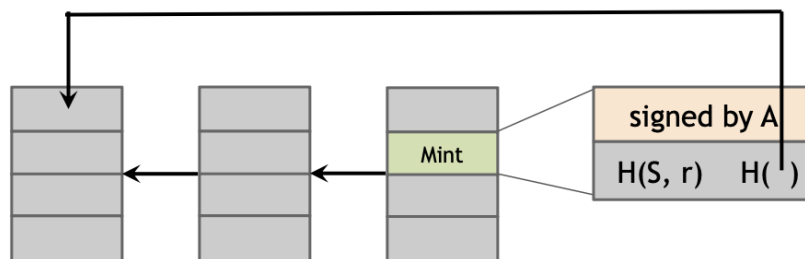


Figure 9.1: Rappresentazione schematica dell'inserimento di un commitment Zerocoin nella blockchain tramite una transazione di tipo Mint. La transazione include il commitment $H(S, r)$ ed è firmata dal mittente.

9.2.3 Il Processo di Riscatto (Redeeming): La Magia delle Prove zero knowledge

Il riscatto è la fase in cui l'utente spende uno Zerocoin precedentemente coniato, generando una transazione non collegabile alla coniazione originale.

Il processo di riscatto si sviluppa così:

1. L'utente esamina la blockchain per identificare tutti i commitment Zerocoin registrati, costruendo l'insieme $(C_1, C_2, \dots, C_n)$ che include il proprio commitment.
2. Genera una prova zero knowledge non interattiva π che dimostra due fatti: (1) conosce un commitment C presente nell'insieme pubblico, e (2) conosce un valore r tale che C si apre correttamente sul numero seriale S che sta per rivelare.
3. Pubblica sulla blockchain una transazione di spesa contenente la coppia (S, π) : il numero seriale e la prova zero knowledge.
4. I validatori verificano che il numero seriale non sia già stato utilizzato (prevenendo la doppia spesa) e che la prova π sia matematicamente valida.

La proprietà "zero-knowledge" della prova garantisce che, pur dimostrando la validità del riscatto, non rivela quale specifico commitment l'utente stia utilizzando. Sebbene tutte le transazioni siano pubbliche, risulta matematicamente impossibile collegare il numero seriale rivelato a uno specifico commitment.

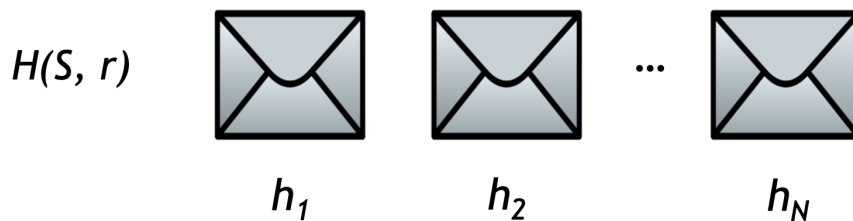


Figure 9.2: Rappresentazione concettuale dell'anonimato in Zerocoin: risulta impossibile determinare quale commitment (rappresentati come buste) corrisponda al numero seriale rivelato.

9.3 La Sfida dell'Efficienza: Accumulatori Crittografici

L'implementazione pratica di Zerocoin si scontra con una sfida fondamentale di efficienza computazionale che potrebbe comprometterne l'adozione su larga scala. Il protocollo richiede la capacità di dimostrare, in modo efficiente e privato, che un determinato impegno crittografico (commitment) appartiene a un insieme precedentemente stabilito, senza rivelare quale specifico elemento sia. Questa operazione, nella sua formulazione naïve, comporterebbe una complessità computazionale lineare rispetto alla dimensione dell'insieme.

Formalmente, una prova di riscatto richiederebbe la dimostrazione di una disgiunzione logica estesa a tutti i commitment presenti nel sistema:

$$\text{"Conosco } r \text{ tale che } H(S, r) = h_1 \text{ OR } H(S, r) = h_2 \text{ OR } \dots \text{ OR } H(S, r) = h_N" \quad (9.1)$$

Questa formulazione presenta due problemi critici:

- **Complessità computazionale:** Il costo di generazione della prova cresce linearmente con il numero di commitment nel sistema, raggiungendo rapidamente livelli proibitivi con l'espansione del network.

- **Dimensione della prova:** La prova stessa avrebbe dimensione proporzionale al numero di commitment, rendendo inefficiente la sua trasmissione e verifica all'interno della blockchain.

Queste limitazioni renderebbero Zerocoin praticamente inutilizzabile in contesti reali, dove l'insieme dei commitment potrebbe facilmente raggiungere dimensioni dell'ordine di milioni di elementi.

9.3.1 Accumulatori Crittografici: Compressione Intelligente dei Dati

Per superare queste limitazioni fondamentali, Zerocoin implementa un sofisticato meccanismo crittografico noto come "accumulatore", introdotto originariamente da Benaloh e de Mare nel 1993 e successivamente perfezionato da Baric e Pfitzmann. Un accumulatore crittografico costituisce una struttura dati avanzata che permette di comprimere un insieme arbitrariamente grande di elementi in un singolo valore di dimensione costante, mantenendo simultaneamente la possibilità di generare prove efficienti di appartenenza per qualsiasi elemento dell'insieme.

La funzione degli accumulatori essenziale è trasformare l'insieme di commitment $(C_1, \dots, C_n)$ in un singolo digest crittografico A la cui dimensione rimane costante indipendentemente dalla cardinalità dell'insieme.

Le proprietà fondamentali di un accumulatore crittografico includono:

- **Compattezza:** La dimensione dell'accumulatore rimane costante indipendentemente dal numero di elementi accumulati.
- **Efficienza di verifica:** La verifica dell'appartenenza di un elemento all'accumulatore ha complessità computazionale sublineare (idealmente costante o logaritmica) rispetto alla dimensione dell'insieme.
- **Unidirezionalità:** È computazionalmente intrattabile derivare gli elementi originali dal valore dell'accumulatore.
- **Resistenza alle collisioni:** È computazionalmente intrattabile trovare un elemento valido che non è stato incluso nell'accumulatore.

Nel contesto di Zerocoin, gli accumulatori permettono di riformulare la prova di appartenenza in modo drammaticamente più efficiente. Anziché dimostrare una disgiunzione su tutti i commitment, l'utente deve semplicemente provare di conoscere un "testimone" (witness) w che attesti l'appartenenza del proprio commitment c all'accumulatore A .

9.3.2 Implementazione RSA degli Accumulatori: Il Cuore Tecnico

Zerocoin implementa gli accumulatori crittografici basandosi su primitive crittografiche associate al problema RSA, sfruttando le proprietà matematiche dell'esponenziazione modulare in gruppi di ordine nascosto. Questa implementazione richiede un ambiente a parametri pubblici (common reference string) e opera attraverso quattro operazioni fondamentali:

9.3.2.1 Generazione dei Parametri (Acc.Setup)

La fase di inizializzazione dell'accumulatore stabilisce i parametri fondamentali del sistema:

1. Generazione di un modulo RSA $N = pq$ dove p e q sono numeri primi di grande dimensione (tipicamente 1024 bit ciascuno).
2. Selezione di un elemento casuale $w \in \mathbb{Z}_N^*$ appartenente al gruppo moltiplicativo residuo modulo N .
3. Calcolo della base dell'accumulatore $u = w^2 \bmod N$, garantendo che u sia un residuo quadratico.
4. Pubblicazione dei parametri pubblici (N, u) , eliminando irreversibilmente i fattori primi p e q per garantire la sicurezza del sistema.

La distruzione dei fattori p e q è cruciale per la sicurezza dell'accumulatore, poiché la loro conoscenza permetterebbe di falsificare facilmente le prove di appartenenza. Questa caratteristica distingue gli accumulatori RSA da altre costruzioni crittografiche e richiede una cerimonia di generazione dei parametri particolarmente robusta.

9.3.2.2 Accumulazione (Accumulate)

La fase di accumulazione aggrega un insieme di elementi in un singolo valore:

1. Dato un insieme di commitment $(c_1, \dots, c_n)$, si calcola l'accumulatore come:

$$A = u^{c_1 \cdot c_2 \cdot \dots \cdot c_n} \bmod N \quad (9.2)$$

2. Questa operazione può essere eseguita incrementalmente, aggiornando l'accumulatore per ciascun nuovo elemento aggiunto:

$$A_{i+1} = A_i^{c_{i+1}} \bmod N \quad (9.3)$$

L'operazione di accumulazione può essere interpretata come l'applicazione ripetuta dell'esponentiazione modulare, dove ciascun commitment contribuisce all'esponente finale. In pratica, poiché l'esponente cresce rapidamente, l'implementazione richiede ottimizzazioni come l'uso dell'aritmetica modulare per gestire l'esponente.

9.3.2.3 Generazione del Testimone (GenWitness)

La generazione del testimone costituisce l'operazione cruciale per consentire prove di appartenenza efficienti:

1. Per un elemento specifico c_j nell'insieme, si calcola il testimone:

$$w_j = u^{c_1 \cdot c_2 \cdot \dots \cdot c_{j-1} \cdot c_{j+1} \cdot \dots \cdot c_n} \bmod N \quad (9.4)$$

2. In termini procedurali, il testimone può essere calcolato come:

$$w_j = \text{Accumulate}((c_1, \dots, c_n) \setminus \{c_j\}) \quad (9.5)$$

ovvero l'accumulatore di tutti gli elementi eccetto c_j .

Il testimone w_j rappresenta essenzialmente una prova che c_j è stato incluso nell'accumulatore A . La sua generazione è computazionalmente costosa (lineare rispetto alla dimensione dell'insieme), ma questa operazione viene eseguita una tantum dal proprietario dell'elemento quando intende dimostrarne l'appartenenza.

9.3.2.4 Verifica dell'Appartenenza (Acc.Verify)

La verifica dell'appartenenza di un elemento all'accumulatore è notevolmente efficiente:

1. Dati un accumulatore A , un elemento c e un testimone w , si calcola:

$$A' = w^c \bmod N \quad (9.6)$$

2. Si verifica se $A = A'$. L'uguaglianza conferma l'appartenenza di c all'accumulatore A .

Questa verifica richiede essenzialmente una singola esponenziazione modulare, indipendentemente dalla dimensione dell'insieme accumulato. Questa asimmetria tra il costo di generazione del testimone (operazione una tantum eseguita dal proprietario dell'elemento) e il costo di verifica (operazione frequente eseguita dalla rete) rende l'approccio particolarmente adatto al contesto blockchain.

9.3.3 Integrazione in Zerocoin: Ottimizzazione delle Prove di Riscatto

Nel contesto di Zerocoin, l'integrazione degli accumulatori RSA permette di riformulare le prove di riscatto in modo sostanzialmente più efficiente. La prova di riscatto si trasforma in una dimostrazione zero knowledge strutturata come segue:

“Conosco (c, w, r) tali che: (9.7)

- c è un commitment valido incluso nell'accumulatore A (verificabile attraverso il testimone w)
- c si apre correttamente sul numero seriale S dichiarato, ovvero $c = H(S, r)$ ”

Questa formulazione comporta diversi vantaggi critici:

- **Efficienza computazionale:** La generazione e verifica della prova hanno complessità costante rispetto al numero di commitment nel sistema.
- **Dimensione della prova:** La prova stessa ha dimensione costante (tipicamente alcuni kilobyte), indipendentemente dalla dimensione dell'insieme dei commitment.
- **Scalabilità:** Il sistema può supportare un numero arbitrariamente grande di commitment senza degradazione significativa delle prestazioni.

9.3.4 Limitazioni e Considerazioni Pratiche

Nonostante i significativi vantaggi offerti dagli accumulatori RSA, la loro implementazione in Zerocoin presenta alcune limitazioni pratiche:

- **Cerimonia di setup trusted:** La generazione dei parametri RSA richiede una fase di inizializzazione che deve garantire l'eliminazione sicura dei fattori p e q , creando un punto di fiducia potenzialmente problematico.
- **Dimensione delle prove:** Sebbene costanti rispetto alla dimensione dell'insieme, le prove non interattive zero knowledge basate su RSA rimangono relativamente grandi (alcuni kilobyte) rispetto alle transazioni standard Bitcoin (centinaia di byte).
- **Overhead computazionale:** Le operazioni crittografiche coinvolte, specialmente nella fase di verifica, comportano un overhead significativo rispetto alle transazioni standard, limitando il throughput complessivo del sistema.

Queste limitazioni hanno motivato sviluppi successivi, come Zerocash, che utilizza costruzioni più avanzate come zk-SNARKs per ottenere prove più compatte ed efficienti, pur mantenendo il principio fondamentale dell'accumulazione crittografica.

9.4 Le Sfide dell'Integrazione: Limiti e Problematiche di Zerocoin

Nonostante l'eleganza matematica, l'integrazione di Zerocoin con Bitcoin presenta sfide significative:

9.4.1 Incompatibilità con il Modello di Script di Bitcoin

L'implementazione richiederebbe modifiche sostanziali al protocollo Bitcoin, introducendo nuovi tipi di transazione per le operazioni di minting e spending con relative prove zero knowledge. Il linguaggio di scripting di Bitcoin non è Turing-completo e non supporta le operazioni crittografiche avanzate necessarie.

Questa incompatibilità richiederebbe una revisione fondamentale del modello di validazione delle transazioni, rappresentando un hard fork con tutte le complessità sociali, economiche e tecniche associate.

9.4.2 Divergenza dai Paradigmi di Sicurezza di Bitcoin

Le transazioni Bitcoin sono protette tramite firma digitale EC-Dsa, associando ciascuna transazione a una chiave pubblica specifica. Le transazioni Zerocoin invece si basano su prove zero knowledge, creando una dissonanza con il modello di sicurezza esistente e complicando l'integrazione.

9.4.3 Il Problema del Setup Fidato

La limitazione più significativa è la necessità di un "setup fidato" per generare i parametri iniziali dell'accumulatore, in particolare il modulo RSA $N = pq$. I fattori primi p e q sono estremamente sensibili: chiunque li conosca potrebbe creare false prove, compromettendo l'integrità del sistema. In altre parole esporre i parametri p e q farebbe perdere la soundness del sistema Zero coin.

Gli utenti devono fidarsi che i generatori dei parametri abbiano distrutto queste informazioni, un requisito in contrasto con la filosofia trustless di Bitcoin. Questo elemento fiduciario ha rappresentato un ostacolo principale all'adozione di Zerocoin come estensione diretta di Bitcoin.

9.5 Zerocash: Evoluzione e Superamento dei Limiti di Zerocoin

Zerocash, presentato nel 2014 al simposio Usenix Security da E. Ben-Sasson et al., rappresenta un'evoluzione fondamentale rispetto a Zerocoin, superandone le limitazioni strutturali e introducendo un paradigma completamente nuovo per la privacy nelle criptovalute. A differenza di Zerocoin, che operava come un'estensione del protocollo Bitcoin richiedendo una valuta base, Zerocash è stato concepito come un sistema autonomo e completo, svincolato dalla necessità di una valuta sottostante.

Le innovazioni principali di Zerocash rispetto al suo predecessore sono duplici. In primo luogo, l'implementazione di primitive crittografiche sostanzialmente più efficienti per la generazione e verifica delle prove di conoscenza zero. In secondo luogo, lo sviluppo di un'architettura indipendente che funziona interamente attraverso transazioni anonime, eliminando la necessità di una valuta base come nel caso di Zerocoin.

9.5.1 zk-SNARKs: Fondamento Crittografico

L'avanzamento tecnologico più significativo introdotto da Zerocash è l'adozione di zk-SNARKs (Zero-Knowledge Succinct Non-interactive Arguments of Knowledge), una classe avanzata di prove zero knowledge caratterizzate da proprietà eccezionali rispetto alle precedenti costruzioni crittografiche:

La dimensione delle prove risulta estremamente ridotta (circa 288 byte) e, aspetto fondamentale, rimane costante indipendentemente dalla complessità della computazione attestata. Questa caratteristica contrasta nettamente con le prove basate su accumulatori RSA utilizzate in Zerocoin, la cui dimensione risultava significativamente maggiore.

Il processo di verifica richiede un tempo dell'ordine dei millisecondi, con complessità computazionale notevolmente inferiore rispetto alle precedenti costruzioni. Questa efficienza rende le prove praticamente utilizzabili anche in contesti ad alta frequenza transazionale, superando una delle limitazioni fondamentali di Zerocoin.

Le prove sono completamente non interattive, potendo essere generate autonomamente dal possessore dell'informazione senza richiedere alcuna interazione con il verificatore. Questa proprietà risulta particolarmente adatta all'implementazione in sistemi blockchain, dove la natura distribuita e asincrona del consenso rende problematici i protocolli interattivi.

L'implementazione di zk-SNARKs ha permesso di superare definitivamente la principale limitazione di Zerocoin, ovvero l'elevato overhead computazionale e di spazio associato alle prove basate su accumulatori RSA, aprendo la strada a un'effettiva implementazione pratica dei principi di privacy crittografica nelle criptovalute.

9.5.2 Transazioni Completamente Anonime

A differenza di Zerocoin, dove l'anonimità riguardava principalmente la dissociazione tra origine e destinazione dei fondi, Zerocash implementa un modello di anonimato integrale per tutte le componenti della transazione.

I valori transazionali sono completamente nascosti, a differenza di Zerocoin dove le denominazioni erano pubblicamente verificabili. Questa proprietà elimina la possibilità di tracciamento basato sull'analisi dei flussi di valore, una tecnica di de-anonimizzazione particolarmente efficace in contesti dove gli importi sono visibili.

Il sistema introduce supporto nativo per operazioni complesse di divisione e fusione dei valori, permettendo transazioni con input e output multipli mantenendo rigorosi standard di anonimato. Questa flessibilità operativa supera una limitazione significativa di Zerocoin, dove la rigidità delle denominazioni predefinite comprometteva l'utilizzabilità pratica del sistema.

Il risultato è un protocollo in cui il registro distribuito registra esclusivamente l'esistenza delle transazioni, senza rivelare alcuna informazione sul loro contenuto effettivo. Metaforicamente, il valore della transazione viene inserito all'interno della "busta" crittografica, rendendo impossibile qualsiasi forma di analisi basata sui flussi di valore o sul volume transazionale.

9.5.3 La Problematica del Setup Fidato

Nonostante i significativi avanzamenti tecnici, Zerocash eredita e per certi versi amplifica la problematica del "setup fidato" già presente in Zerocoin. La generazione dei parametri pubblici per le prove zk-SNARK richiede input casuali e segreti che devono essere irrecuperabilmente distrutti dopo la creazione dei parametri pubblici.

Questa fase rappresenta un punto critico nel sistema: chiunque entrasse in possesso di questi valori segreti potrebbe compromettere irrimediabilmente l'intero protocollo, generando fondi dal nulla senza possibilità di rilevamento. Le implicazioni di questa vulnerabilità sono particolarmente severe:

- **Compromissione totale:** A differenza di Zerocoin, dove la compromissione avrebbe consentito principalmente la falsificazione di prove, in Zerocash comprometterebbe l'intero sistema monetario, permettendo la creazione illimitata di valuta.
- **Difficoltà di rilevazione:** La natura delle prove rende potenzialmente impossibile rilevare l'avvenuta compromissione, in contrasto con altri attacchi crittografici che lasciano tracce identificabili.
- **Complessità della cerimonia:** La generazione dei parametri richiede una "cerimonia multi-parte" estremamente sofisticata, in cui diversi partecipanti contribuiscono alla creazione dei parametri in modo che tutti debbano essere compromessi per danneggiare il sistema.

Questa problematica rappresenta il principale ostacolo all'adozione di Zerocash e ha stimolato ulteriori ricerche sulla generazione di parametri in contesti a fiducia distribuita o minimizzata.

9.6 Il Continuum dell'Anonimato nelle Criptovalute

Zerocash si posiziona all'estremità del continuum dell'anonimato nelle criptovalute, rappresentando il massimo livello di privacy attualmente raggiungibile. Analizzando i diversi approcci alla privacy nelle criptovalute, è possibile identificare cinque livelli progressivi di anonimato, come illustrato nella tabella seguente:

Sistema	Tipo	Attacchi all'anonimato	Implementabilità
Bitcoin	Pseudonymous	Tx graph analysis	Default
Single mix	Mix	Tx graph analysis, bad mix	Usable today
Mix chain	Mix	Side channels, bad mixes/peers	Bitcoin-compatible
Zerocoin	Cryptographic mix	Side channels (possibly)	Altcoin
Zerocash	Untraceable	None	Altcoin, tricky setup

Table 9.1: Livelli di anonimato nei sistemi di criptovalute

Questa progressione evidenzia il trade-off fondamentale tra livello di anonimato e complessità implementativa/computazionale, con Zerocash che rappresenta la frontiera attuale della privacy crittografica, al costo di significative sfide tecniche e di adozione.

Chapter 10

Lezione 10

10.1 Alternative Mining Puzzles

I puzzle crittografici costituiscono il nucleo fondamentale del sistema Bitcoin, rappresentando non solo un meccanismo di sicurezza, ma anche il principio cardine del sistema di incentivi che guida il comportamento dei partecipanti alla rete. Il sistema di mining di Bitcoin si basa su un delicato equilibrio: da un lato, il puzzle deve essere sufficientemente complesso da rendere costosi eventuali tentativi di attacco, garantendo così la sicurezza della rete; dall'altro, deve mantenere un livello di difficoltà tale da assicurare che i miner onesti ricevano compensi adeguati per il loro contributo. Questo equilibrio è essenziale per la sostenibilità a lungo termine dell'ecosistema Bitcoin.

Il design di questi puzzle crittografici non è tuttavia limitato alla configurazione attualmente implementata in Bitcoin. Esistono infatti numerose proposte alternative che perseguono obiettivi diversi, come la resistenza agli ASIC (Application-Specific Integrated Circuits), la resistenza ai mining pool e la generazione di benefici intrinseci oltre alla sicurezza della blockchain. Questo capitolo analizza tali alternative, esaminandone le caratteristiche, i vantaggi potenziali e le sfide implementative.

10.2 Requisiti Essenziali dei Mining Puzzles

Indipendentemente dalle specifiche implementazioni, qualsiasi puzzle crittografico adeguato all'utilizzo in un sistema di criptovalute deve soddisfare alcuni requisiti fondamentali. Questi criteri rappresentano il minimo necessario affinché il meccanismo possa garantire la sicurezza e l'equità del sistema.

10.2.1 Verifica Economica

Un requisito imprescindibile è che la verifica della soluzione del puzzle debba essere computazionalmente economica. Questo aspetto è cruciale poiché, mentre il mining è un'attività svolta da un sottoinsieme relativamente piccolo di nodi della rete, la verifica delle soluzioni deve essere eseguita da tutti i nodi per convalidare la blockchain. Un processo di verifica oneroso rallenterebbe significativamente la rete e aumenterebbe i requisiti hardware per la partecipazione, compromettendo la decentralizzazione del sistema.

10.2.2 Difficoltà Adattiva

Il puzzle deve consentire un aggiustamento parametrico della difficoltà. Questa caratteristica è essenziale per mantenere un tempo medio costante tra i blocchi nonostante le fluttuazioni nella potenza di calcolo totale della rete. In Bitcoin, la difficoltà viene ricalibrando ogni 2016 blocchi (circa due settimane) per mantenere un tempo medio di 10 minuti tra i blocchi. Senza questo meccanismo adattivo, l'aumento della potenza di calcolo porterebbe a una generazione sempre più rapida di blocchi, alterando l'emissione monetaria prevista e potenzialmente destabilizzando il consenso della rete.

10.2.3 Proporzionalità alla Potenza di Calcolo

Un aspetto fondamentale di un puzzle equo è che la probabilità di trovare una soluzione valida deve essere direttamente proporzionale alla potenza di calcolo impiegata. Questo principio garantisce che i partecipanti ricevano ricompense commisurate al loro contributo, senza distorsioni che favoriscano indebitamente determinati attori.

10.2.4 Assenza di Vantaggio Sproporzionato per Grandi Operatori

Il puzzle deve essere progettato in modo che gli operatori di maggiori dimensioni ottengano un vantaggio solo proporzionale alla loro potenza di calcolo. In altre parole, non dovrebbero esistere economie di scala significative che rendano il mining profittevole solo per entità di grandi dimensioni. Questo requisito è essenziale per preservare la natura decentralizzata del sistema.

10.2.5 Compenso Proporzionale anche per Piccoli Operatori

Complementare al punto precedente, è necessario che anche gli operatori con risorse computazionali limitate ricevano compensi proporzionali al loro contributo. Questo aspetto è particolarmente importante per garantire l'accessibilità del sistema e prevenire una centralizzazione eccessiva.

10.3 L'Inefficacia dei Puzzle Sequenziali

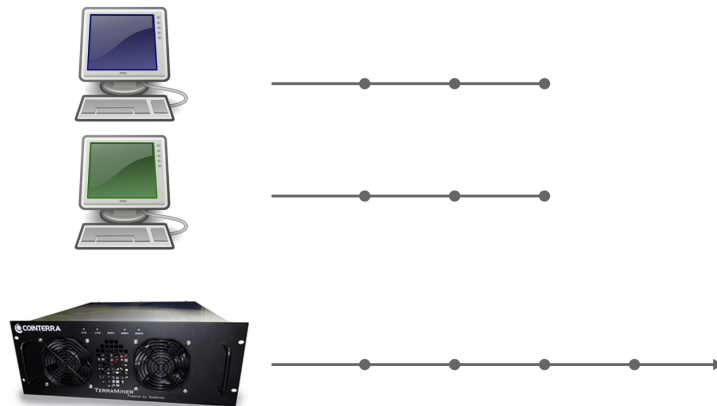


Figure 10.1: Rappresentazione di un puzzle sequenziale dove il miner più veloce completa tutti gli N passaggi per primo, trovando invariabilmente la soluzione.

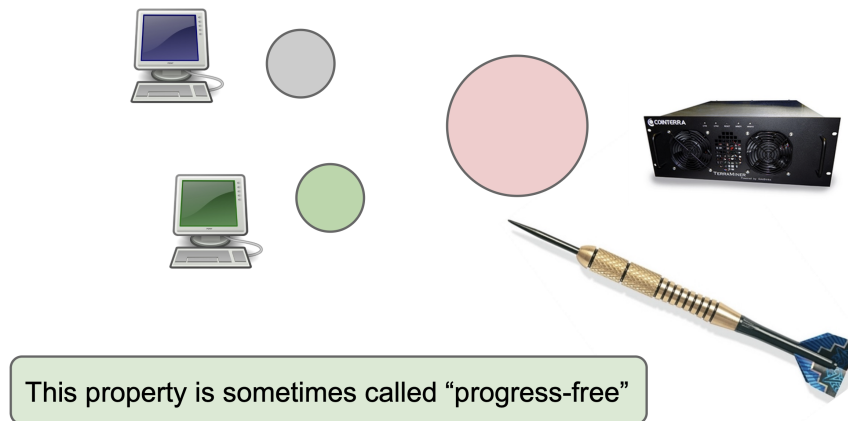
Per illustrare l'importanza di alcuni dei requisiti sopra elencati, è utile esaminare un esempio di puzzle crittografico inadeguato: il puzzle sequenziale. In tale configurazione, trovare una soluzione richiede l'esecuzione di N passaggi computazionali in sequenza, dove ciascun passaggio dipende dal risultato del precedente.

Questo approccio risulta problematico perché il miner con la maggiore potenza computazionale avrà sempre un vantaggio sproporzionato. In pratica, il miner più veloce completerebbero sempre per primo la sequenza di N passaggi, aggiudicandosi invariabilmente la ricompensa. In un simile scenario, i miner con risorse limitate non avrebbero praticamente alcuna possibilità di ottenere ricompense, anche se proporzionalmente al loro contributo.

10.3.1 L'Importanza della Proprietà "Progress-Free"

Un puzzle efficace dovrebbe invece implementare un campionamento ponderato, in cui ogni tentativo di soluzione sia indipendente dai precedenti e abbia una probabilità di successo proporzionale alla potenza di calcolo impiegata. Questa caratteristica, talvolta definita "progress-free" (priva di progresso), garantisce che non vi sia un vantaggio accumulativo nell'avere maggiore potenza di calcolo se non in termini strettamente proporzionali.

Il puzzle SHA-256 utilizzato in Bitcoin soddisfa questa proprietà: ogni hash calcolato ha una probabilità indipendente di produrre una soluzione valida, indipendentemente dal tempo trascorso o dal numero di tentativi precedenti. Questa caratteristica rende il mining un processo simile a una lotteria, in cui la probabilità di vincita è proporzionale al numero di biglietti acquistati (hash calcolati), senza altri vantaggi strutturali.



10.4 Puzzle Resistenti agli ASIC

Con l'evoluzione dell'ecosistema Bitcoin, l'attività di mining ha subito una progressiva professionalizzazione, con la transizione da hardware general-purpose (CPU, GPU) a circuiti integrati dedicati (ASIC), specificamente progettati per eseguire l'algoritmo SHA-256 con efficienza superiore di diversi ordini di grandezza. Questa evoluzione ha sollevato preoccupazioni riguardo alla centralizzazione del mining e all'accessibilità del sistema.

10.4.1 Motivazioni per la Resistenza agli ASIC

La ricerca di puzzle resistenti agli ASIC è motivata da due obiettivi principali:

10.4.1.1 Democratizzazione del Mining

Il primo obiettivo è rendere il mining accessibile alle persone comuni con hardware commodity, come laptop, personal computer o persino dispositivi mobili. Riducendo il divario di efficienza tra hardware specializzato e general-purpose, si abbasserebbe la barriera d'ingresso al mining, potenzialmente aumentando la decentralizzazione del network. Questo approccio mira a ridurre il gap prestazionale tra hardware custom e dispositivi di uso generale.

10.4.1.2 Prevenzione della Dominanza Manifatturiera

Il secondo obiettivo è prevenire che pochi grandi produttori di hardware dominino l'ecosistema del mining. Attualmente, la produzione di ASIC per Bitcoin è concentrata in un numero limitato di aziende, che godono di vantaggi significativi:

- **Vantaggio da rodaggio (burn-in):** I produttori possono testare e utilizzare i propri ASIC prima di metterli in vendita, ottenendo un vantaggio temporale non replicabile.
- **Progettazione interna:** L'accesso a competenze di progettazione avanzate consente lo sviluppo di chip sempre più efficienti, non accessibili ai concorrenti minori.

L'approccio per contrastare questo fenomeno consiste nel ridurre il gap prestazionale tra gli ASIC più recenti e quelli già disponibili sul mercato, limitando il vantaggio competitivo derivante dall'essere all'avanguardia tecnologica.

10.4.2 Puzzle Memory-Hard

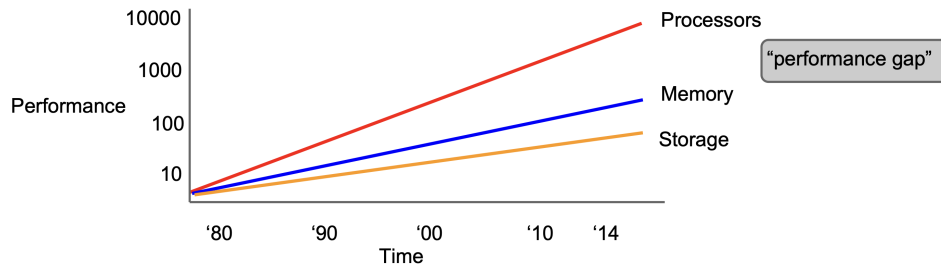


Figure 10.2: Evoluzione della performance di processori e memoria nel tempo, evidenziando come il divario prestazionale sia maggiore per i processori rispetto alla memoria.

Una delle strategie più promettenti per ottenere resistenza agli ASIC è l'utilizzo di puzzle "memory-hard", ovvero algoritmi la cui esecuzione richiede quantità significative di memoria. Questo approccio si basa sulla premessa che il costo e le prestazioni della memoria sono più stabili nel tempo rispetto a quelli dei processori.

Come illustrato nella Figura 10.2, mentre le prestazioni dei processori hanno registrato miglioramenti di diversi ordini di grandezza dagli anni '80 ad oggi, la memoria ha avuto un'evoluzione molto più contenuta. Di conseguenza, la progettazione di ASIC specifici per algoritmi memory-hard offrirebbe vantaggi meno marcati rispetto a quelli osservabili per algoritmi computazionalmente intensivi come SHA-256.

10.4.2.1 Scrypt: Fondamenti e Implementazione

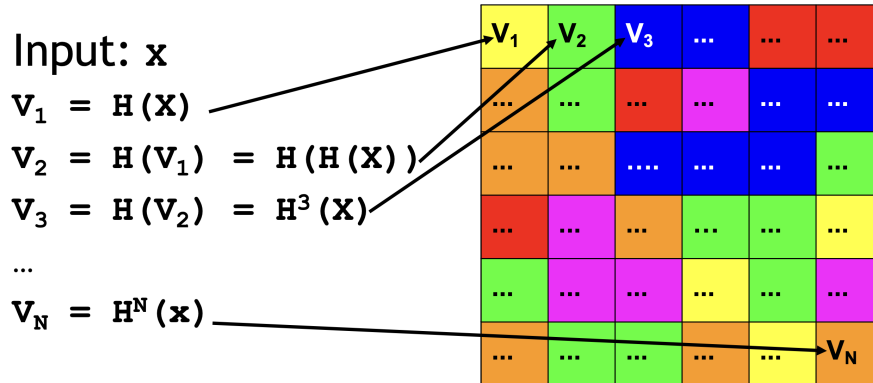
Scrypt rappresenta una delle più significative innovazioni nell'ambito delle funzioni memory-hard, sviluppata nel 2009 da Colin Percival con l'obiettivo specifico di contrastare gli attacchi hardware specializzati. A differenza delle funzioni di hash tradizionali come SHA-256, che richiedono risorse computazionali ma poca memoria, Scrypt è stata progettata per necessitare simultaneamente di entrambe le risorse, imponendo un trade-off costoso tra tempo di esecuzione e memoria utilizzata. L'algoritmo è stato originariamente concepito per rafforzare la derivazione di chiavi crittografiche da password, un contesto in cui la resistenza agli attacchi hardware (come FPGA e ASIC) risulta particolarmente preziosa. Successivamente, grazie alle sue proprietà anti-ASIC, Scrypt è diventata la funzione di hash predominante in numerose criptovalute alternative, con Litecoin come implementazione di riferimento.

Architettura a Due Fasi Il funzionamento di Scrypt si articola in due fasi distinte e complementari:

1. **Fase di Inizializzazione e Riempimento della Memoria:** Nella prima fase, Scrypt costruisce una struttura dati di grandi dimensioni attraverso un processo iterativo. Partendo dall'input iniziale (X) (che in contesto di mining include l'header del blocco candidato), l'algoritmo genera una sequenza di valori ($V_1, V_2, \dots, V_N$) tramite applicazioni successive di una funzione di hash (H):

$$V_1 = H(X) \quad V_2 = H(V_1) = H^2(X) \quad V_3 = H(V_2) = H^3(X) \quad \vdots \quad V_N = H(V_{N-1}) = H^N(X)$$

Questi valori vengono memorizzati in un buffer di dimensione N , occupando uno spazio proporzionale al parametro di costo dell'algoritmo. Questa fase richiede $O(N)$ operazioni di hash e $O(N)$ spazio di memoria.

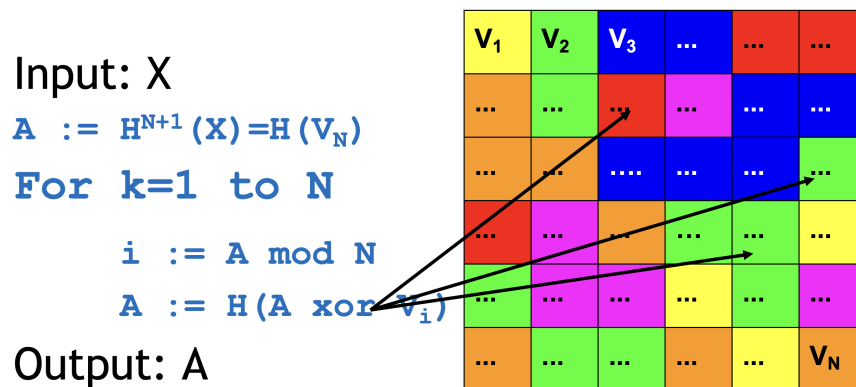


2. **Fase di Mixing con Accesso Pseudocasuale:** Nella seconda fase, Scrypt effettua accessi in lettura alla memoria precedentemente inizializzata, seguendo un pattern pseudocasuale determinato dagli stessi dati. L'algoritmo inizializza un accumulatore $A := H(V_N)$ e procede con N iterazioni dove, ad ogni passo:

$$i := A \bmod N \quad (10.1)$$

$$A := H(A \oplus V_i) \quad (10.2)$$

Questa fase è cruciale per la proprietà memory-hard: ogni iterazione dipende da un accesso casuale alla memoria, rendendo impossibile ottimizzare il processo senza mantenere l'intero buffer in memoria. L'output finale A costituisce il risultato dell'algoritmo.



Complessità Computazionale e Spaziale L'aspetto fondamentale di Scrypt è il rigido vincolo tra memoria e tempo di esecuzione. Con accesso completo alla memoria, l'algoritmo ha complessità temporale ($O(N)$) dove (N) è il parametro di dimensionamento. Tuttavia, se l'implementazione tenta di risparmiare memoria, incorrerà in penalità computazionali significative:

- **Implementazione ottimale:** Con ($O(N)$) spazio di memoria, la complessità temporale è ($O(N)$).

- **Implementazione con memoria limitata:** Con memoria insufficiente, per calcolare on-demand un valore ($V[j]$) è necessario ricalcolare l'intera catena ($V[1], V[2], \dots, V[j-1]$), risultando in una complessità temporale di ($O(N^2)$) nel caso peggiore.

Questa relazione quadratica significa che tentativi di implementare Scrypt con memoria ridotta comportano un incremento del tempo di esecuzione che cresce molto più rapidamente rispetto al risparmio di memoria ottenuto.

10.4.2.2 Compromesso Tempo-Memoria in Scrypt

Il design di Scrypt induce un compromesso tempo-memoria (time-memory trade-off) non lineare e sfavorevole, rappresentando la sua principale innovazione rispetto ad algoritmi precedenti. Matematicamente, questo compromesso può essere analizzato considerando strategie di implementazione con memoria ridotta. Consideriamo uno scenario in cui si memorizzano solo i valori (V_i) con indice dispari, dimezzando così i requisiti di memoria. Quando l'algoritmo richiede l'accesso a un valore (V_i) con (i) pari, l'implementazione deve:

1. Recuperare il valore (V_{i-1}) dalla memoria (disponibile poiché ha indice dispari)
2. Calcolare dinamicamente ($V_i = H(V_{i-1})$)

Questa strategia aumenta il numero totale di operazioni da ($2N$) a circa ($5N/2$), poiché statisticamente metà degli accessi richiederanno un calcolo aggiuntivo. Generalizzando, se si utilizza solo una frazione ($1/k$) della memoria originale, il tempo di esecuzione cresce approssimativamente proporzionale a (k).

Questo compromesso sfavorevole è fondamentale per l'efficacia di Scrypt come puzzle proof-of-work resistente agli ASIC, poiché:

- Ostacola l'implementazione di pipelines di esecuzione parallela che potrebbero accelerare drasticamente l'algoritmo
- Rende meno attraenti i compromessi che sacrificano memoria per aumentare la densità di unità di calcolo
- Forza i produttori di hardware specializzato a includere significative quantità di memoria on-chip, riducendo il vantaggio rispetto all'hardware general-purpose

10.4.3 Critica e Limitazioni di Scrypt

Nonostante le sue eleganti proprietà teoriche, l'adozione di Scrypt come alternativa a SHA-256 presenta alcune limitazioni significative che ne hanno parzialmente compromesso gli obiettivi originali:

- **Costo computazionale della verifica:** A differenza di SHA-256, dove la verifica delle soluzioni richiede una singola operazione di hash, la verifica di una soluzione Scrypt necessita dell'esecuzione completa dell'algoritmo, con i medesimi requisiti di memoria e complessità computazionale della fase di mining. Questa caratteristica impone un overhead significativo ai nodi della rete durante la validazione dei blocchi, potenzialmente limitando la scalabilità del sistema.

- **Parametrizzazione e calibrazione:** La configurazione ottimale dei parametri di Scrypt (dimensione della memoria, numero di iterazioni) rappresenta un compromesso delicato tra sicurezza e prestazioni. Valori troppo elevati renderebbero la verifica proibitivamente costosa, mentre valori troppo bassi ridurrebbero l'efficacia della resistenza agli ASIC.
- **Resistenza agli ASIC compromessa:** Contrariamente alle aspettative iniziali, l'industria hardware ha progressivamente sviluppato ASIC ottimizzati per Scrypt, come quelli prodotti da Zeus Miner, Innosilicon e altri. Questi dispositivi integrano quantità sufficienti di memoria on-chip per eseguire efficientemente l'algoritmo, erodendo il vantaggio teorico dell'approccio memory-hard. Sebbene il divario di efficienza tra ASIC e hardware general-purpose rimanga inferiore rispetto a SHA-256, la promessa di una democratizzazione radicale del mining non si è pienamente realizzata.

10.4.4 Approcci Alternativi alla Resistenza agli ASIC

Oltre a Scrypt, sono stati proposti altri approcci per ottenere resistenza agli ASIC:

10.4.4.1 Funzioni di Hash Complesse

Una strategia consiste nell'utilizzare funzioni di hash più complesse, combinando diversi algoritmi. Un esempio è X11, utilizzato inizialmente da Darkcoin (ora DASH), che concatena 11 diverse funzioni di hash. L'idea è che la progettazione di ASIC per una catena così complessa di algoritmi sarebbe più costosa e tecnicamente impegnativa.

10.4.4.2 Target Mobile

Un approccio più radicale consiste nel modificare periodicamente l'algoritmo di puzzle, rendendo obsoleti gli ASIC esistenti. Questa strategia, sebbene teoricamente efficace, presenta notevoli sfide implementative, tra cui la necessità di aggiornamenti frequenti del software e il rischio di fork della blockchain.

10.4.5 Contrargomentazione: SHA-256 è Adeguato

È interessante notare che esiste una corrente di pensiero che sostiene che l'algoritmo SHA-256 di Bitcoin sia già sufficientemente adeguato. Questa posizione si basa sull'osservazione che l'evoluzione degli ASIC per Bitcoin sta rallentando, con miglioramenti marginali tra generazioni successive. In questa prospettiva, i grandi ASIC più costosi offrono vantaggi prestazionali sempre più ridotti rispetto ai modelli più economici:

Un ASIC economico contiene un numero inferiore di circuiti SHA-256 standard rispetto a un ASIC costoso, ma l'architettura fondamentale rimane la stessa. Questa standardizzazione limita il vantaggio derivante da economie di scala o da progetti proprietari avanzati, potenzialmente stabilizzando l'ecosistema del mining nel lungo periodo.

10.5 Proof-of-Useful-Work

Una critica frequentemente rivolta a Bitcoin riguarda l'apparente "spreco" di energia elettrica per il mining. Con un consumo energetico stimato tra 150 MW e 900 MW (a metà del 2014), e significativamente aumentato negli anni successivi, sorge spontanea la domanda: è possibile riciclare questa potenza computazionale per scopi utili?

10.5.1 Il Concetto di Computazione Utile

Il proof-of-useful-work (prova di lavoro utile) si propone di risolvere questo problema modificando il puzzle crittografico in modo che la sua soluzione produca risultati con valore scientifico, sociale o computazionale al di là della semplice sicurezza della blockchain.

10.5.1.1 Candidati per Computazioni Utili

Tra le applicazioni potenzialmente adatte a questo approccio vi sono problemi del tipo "ago nel pagliaio", che richiedono l'esplorazione di vasti spazi di ricerca. Esempi promettenti includono:

- **Folding proteico:** La ricerca di configurazioni a bassa energia per proteine, con applicazioni in biologia e medicina.
- **Ricerca di segnali extraterrestri:** L'analisi di regioni anomale in segnali astronomici, utilizzata nella ricerca di intelligenze extraterrestri.

Questi problemi hanno dimostrato di essere adatti a progetti di calcolo distribuito come i progetti @Home, suggerendo una potenziale compatibilità con un sistema di mining distribuito.

10.5.1.2 Sfide Implementative

L'implementazione di proof-of-useful-work deve affrontare diverse sfide:

- **Difficoltà controllata:** Le istanze del problema scelte casualmente devono mantenere un livello di difficoltà prevedibile.
- **Selezione imparziale:** Si pone il problema di chi debba scegliere i problemi da risolvere, con il rischio di introdurre centralizzazione o bias.

10.5.2 Primecoin: Un Caso di Studio

Un esempio concreto di proof-of-useful-work è Primecoin, una criptovaluta lanciata da Sunny King nel 2013. Primecoin sostituisce il tradizionale puzzle basato su SHA-256 con la ricerca di catene speciali di numeri primi.

10.5.2.1 Catene di Cunningham

Il puzzle di Primecoin richiede di trovare sequenze particolari di numeri primi chiamate "catene di Cunningham". Una catena di Cunningham è una sequenza di numeri primi $p_1, p_2, \dots, p_n$ dove $p_i = 2p_{i-1} + 1$, ovvero ogni numero primo della sequenza è pari al doppio del precedente più uno.

10.5.2.2 Algoritmo del Puzzle

Il processo di mining in Primecoin è parametrizzato da tre valori m , n e k e si articola come segue:

1. Si calcola $x = H(\text{prev} \parallel$

La difficoltà del puzzle può essere regolata modificando i parametri m , n e k .

10.5.2.3 Risultati e Utilità

Primecoin ha ottenuto alcuni risultati tangibili in ambito matematico: molte delle catene di Cunningham più lunghe conosciute sono state scoperte da miner di Primecoin. Questo dimostra che il sistema ha effettivamente prodotto risultati potenzialmente utili.

Tuttavia, il valore pratico di queste scoperte rimane oggetto di dibattito. Da un lato, il problema è studiato anche da altri progetti come PrimeGrid, suggerendo un certo interesse matematico; dall'altro, l'applicabilità pratica di queste catene di numeri primi non è immediatamente evidente rispetto ad altre computazioni con impatti più diretti in ambiti come medicina o informatica.